

UNIVERSITY OF GDAŃSK

Krzysztof Torończak
Wiktor Szymulewicz
Bartosz Urbanowicz
Tomasz Wojciechowski

Passed Pawn educational chess platform – development with DevOps practices and the use of AI

Field of study: Computer Science
Level of study: first-cycle (bachelor's) studies
Study profile: practical
Form of study: full-time

Bachelor's thesis
written under the supervision of
dr Jakub Neumann, prof. UG

Gdańsk, 2025

Contents

1. Introduction	7
1.1. Introduction to the challenges of learning chess	7
1.2. Goal of the thesis	7
1.3. Existing similar solutions on the market	8
1.3.1. Chessable	8
1.3.2. Chessly	8
1.3.3. lichess.org	9
1.3.4. Summary of existing solutions	9
2. System requirements analysis	11
2.1. Functional requirements	11
2.1.1. User registration and login	11
2.1.2. Browsing courses	11
2.1.3. Course details	11
2.1.4. Learning from a course	11
2.1.5. Writing reviews	12
2.1.6. Creating courses	12
2.2. Non-functional requirements	12
2.3. Description of actors	12
2.4. Coach use case diagram	13
2.5. Student use case diagram	14
2.6. Use case diagram of features shared by the coach and the student	16
2.7. External systems the application relies on	16
2.7.1. Stripe	16
2.7.2. Cloudinary	17
2.7.3. MistralAI	17
3. User interface design	18
3.1. The UI design process	18
3.2. Selected user interface design tools	19
3.3. Limitations of UI libraries	20
3.4. Custom UI components and views	20
3.5. The importance of an original user interface design	29
3.6. Problems and lost resources	30
4. Passed Pawn architecture	32
4.1. Examples of commonly used types of system architectures	33
4.1.1. Monolith	33
4.1.2. SOA (Service-Oriented Architecture)	33

4.1.3. Microservices	34
4.1.4. Big Ball of Mud – an anti-pattern	36
4.2. Choice and design of the Passed Pawn architecture	37
4.3. Single Page Application (SPA) vs Multi Page Application (MPA)	42
4.3.1. Speed of interaction	42
4.3.2. SEO (Search Engine Optimization)	42
4.3.3. Initial loading time	42
4.3.4. Use of client resources	43
4.3.5. Server load	43
4.3.6. Rich user experience	43
4.3.7. Efficient development	43
4.4. Communication protocols used	43
4.4.1. SSE	43
4.4.2. HTTP, HTTP + TLS	44
5. SPA browser application	45
5.1. Comparison of the most popular SPA frameworks/libraries – table	45
5.2. Technology choices related to the user interface	46
5.2.1. Comparison of frameworks in terms of styling – table	46
5.2.2. Approach to styling	46
5.2.3. Style encapsulation	47
5.2.4. PrimeNG as the chosen UI library	47
5.3. Features of Angular and their impact on the implementation	48
5.3.1. Strongly imposed design assumptions	48
5.3.2. TypeScript	48
5.3.3. Dependency Injection system	48
5.3.4. Extensive support for forms	49
5.3.5. Attribute directives	49
5.3.6. Advanced and expressive data binding	50
5.3.7. Support for reactive programming – RxJS	50
5.3.8. Change detection	51
5.3.9. Built-in HTTP client	51
5.4. Technology choices related to access control	52
5.4.1. KeycloakJS, Keycloak-Angular	52
5.4.2. Guards	53
5.5. Implementation structure	53
5.6. Chess logic	55
6. Main server application	57
6.1. Purpose and role of the server application	57
6.2. ASP.NET Core — the reason for choosing the technology	57

6.3. Application architecture	57
6.3.1. Design patterns	57
6.3.2. N-tier architecture	57
6.4. Application layers	58
6.4.1. API	58
6.4.2. Business Logic	58
6.4.3. Data Access	58
6.4.4. Models	58
6.4.5. Layer diagram	58
6.5. API interface	59
6.5.1. REST	59
6.5.2. Swagger	59
6.6. Examples of operations	60
6.6.1. Fetching a lesson by ID	60
6.7. Error handling and validation	61
6.7.1. Global exception handler	61
6.7.2. Validation	61
6.8. Communication with the database — Entity Framework Core	61
7. AI module as a tool for improving UX	63
7.1. Purpose of introducing the module	63
7.2. RAG system – a solution that makes it possible to use an LLM	63
7.3. Implementation of the AI server application	64
7.3.1. Python	64
7.3.2. Using LangChain to create a RAG chain	64
7.3.3. Vector database	65
7.3.4. API	66
7.4. Analysis of available AI models	66
7.5. Sequence diagram of the AI module	66
8. System infrastructure	68
8.1. Division of infrastructure into traditional/on-premises and cloud	68
8.2. Choice of an OVH dedicated server and a domain name	69
8.2.1. Dedicated server offer and specification	69
8.2.2. Domain name offer and specification	70
8.3. Dedicated server software	70
8.4. Kubernetes – the chosen system orchestration tool	70
8.4.1. Flexibility of Kubernetes	71
8.4.2. Reliability and scalability of Kubernetes	72
8.4.3. Justification for using Kubernetes	72
8.5. K3s as the chosen K8s distribution	72

8.6. The most important custom components of the cluster	73
8.6.1. Longhorn	73
8.6.2. Nginx Ingress Controller	74
8.6.3. Cert-manager	74
8.6.4. Other custom elements of the cluster	74
8.7. Application development and deployment infrastructure	75
8.7.1. Jenkins server	75
8.7.2. GitHub services	75
8.8. Keycloak server	76
8.9. Use of Infrastructure as Code (IaC) and GitOps good practices	76
8.9.1. Origins of IaC	76
8.9.2. Definition and significance of IaC	76
8.9.3. Scope of IaC practices used	77
8.9.4. Origins of GitOps	77
8.9.5. Definition and significance of GitOps	78
8.9.6. Scope of GitOps practices used	78
9. Process automation, the DevOps approach	80
9.1. Methodology of developing and maintaining the Passed Pawn application	80
9.1.1. Application of DevOps practices	81
9.1.2. Application of CI/CD practices	81
9.2. Jenkins – the main process automation tool	82
9.2.1. The most popular alternatives to Jenkins	83
9.2.2. Scope of Jenkins usage	83
9.2.3. Justification for choosing Jenkins	83
9.3. Division of responsibility for pipelines	84
9.4. Jenkins — Kubernetes integration	87
9.5. GitHub Actions – test automation	88
9.6. Git hooks – code validation	89
9.6.1. Justification for using a pre-commit hook	89
9.6.2. Husky – a platform-agnostic wrapper for git hooks	89
9.6.3. ESLint – static code analysis	90
9.6.4. Stylelint – static code analysis	90
10. System security	91
10.1. Keycloak central authorisation server	91
10.2. System security methodology	92
10.3. General security schemes and mechanisms	93
10.3.1. Same Origin Policy	93
10.3.2. HTTP + TLS	93
10.4. Client application security	94

10.4.1. Protection against XSS	94
10.4.2. Protection against video theft	94
10.4.3. Access token management by the Keycloak server and the Keycloak adapter	95
10.5. Server application security	96
10.5.1. SQL injection	96
10.5.2. Data validation	96
10.5.3. RBAC validation	96
10.6. System infrastructure security	96
11. Organisation of work on the project	98
11.1. Pair programming	98
11.2. Code review	99
11.3. Kanban	99
11.4. Brainstorming	100
11.5. Communication in the team	100
12. System testing and validation	102
12.1. API tests	102
12.1.1. Manual tests	102
12.1.2. Automated tests	102
12.1.3. GitHub Actions	103
12.1.4. Rider	103
12.2. End-to-end tests	104
13. Project summary and potential for further development	106
Bibliography	107

Attachments

The code from the main branches (main) of the git repositories used in the thesis was attached to the thesis in the form of a zip file. In addition, the repositories can be viewed on the GitHub platform: <https://github.com/passed-pawn-dev>

The development version of the web application is available at: <https://pp-dev.passed-pawn.com>

1. Introduction

1.1. Introduction to the challenges of learning chess

Chess has been one of the most popular board games in the world for centuries. Its origins date back to the 6th century AD, but in recent years the game has been experiencing a true renaissance. This is due to the growing popularity of playing online. According to estimates, as many as 600 million people around the world may play chess.¹ As the popularity of this strategy game grows, so naturally does the demand for effective and easily accessible ways of learning chess. The potential of this market is further amplified by the fact that a player needs education and development at every level of skill. Complete beginners have to learn the basic rules, such as how the pieces move and the general principles of the game. Once they master the basics, it is time to learn more complex strategies and motifs. Advanced players, in turn, memorise openings and endgames. Even at the competitive level, players must constantly study new variations and learn how to respond to them, depending on what their opponent might play.

Many novice chess players wrongly assume that it is enough to improve their skills by playing games. Such an approach can lead to reinforcing bad habits, a growing number of defeats and, ultimately, giving up the game. Players can also find a wide range of private lessons given by chess coaches. This form of learning, although it can prove very effective, requires a commitment of time and considerable financial resources. There is therefore a need for a platform that enables less budget-straining learning over the Internet, on which a player can spend as much time as they themselves consider necessary. At the same time, the platform should offer comprehensive and effective chess education.

1.2. Goal of the thesis

The goal of this thesis is to create a web application for learning chess effectively through chess courses created by experienced coaches registered in the system. The application is meant to meet the needs of two types of users — coaches and students. For the student, it should make it possible to purchase the courses that best suit their needs. The student should also be able to search precisely for a course that interests them, preview selected parts of it and acquire it. In a purchased course the student should have access to various kinds of interactive elements dedicated to chess that support learning and allow them to absorb and consolidate knowledge effectively at their own pace. They should also be able to rate the course in order to share their opinion with other users.

The coach should be able to create courses with a price, size and scope of material set by themselves. The tools provided for creating interactive elements should be simple and intuitive, but also flexible and rich in functionality. In addition, enabling the coach to edit and modify the created courses, lessons and elements is a desirable part of the application.

Apart from fulfilling the functional assumptions presented above, an equally important goal of the thesis is to carry out the project in a way that allows easy maintenance, free modification and further

development. Achieving this goal requires making conscious and well-considered technological and architectural decisions, while making use of modern, good software development standards. This refers above all to applying the DevOps approach and making heavy use of the practices of Infrastructure as Code (IaC), GitOps and CI/CD.

The last important topic that the thesis aims to explore is the effective use of AI mechanisms to improve the functioning of the designed web application. Nowadays artificial intelligence has already influenced the functioning of many industries and is a large field of rapidly advancing innovation. One cannot remain indifferent to the current AI revolution, which is why adopting AI-based features in the application is a nod towards the current direction and subject matter of ongoing innovation.

1.3. Existing similar solutions on the market

1.3.1. Chessable

Chessable is an educational chess platform created in 2016, acquired by Play Magnus Group in 2019 and incorporated into the Chess.com ecosystem in 2022. The platform enables learning chess through interactive courses with exercises on the chessboard and video materials, and it also allows tracking of the user's progress and statistics.

The main drawback of the Chessable platform is its unintuitive user interface, in particular the panel for creating and managing the user's courses. Logical errors are visible in the implementation and structure of the portal, an example of which is adding lessons to a course. In addition, an unpolished presentation layer worsens the perception of the application — there is no consistency of the user interface between the view of available chess courses and the form used to create them. The Passed Pawn chess platform operates on a unified visual layer in which using the platform is as simple and approachable for new users as possible. On the other hand, a major advantage of Chessable is its direct integration with and support from the Chess.com platform.

Chessable has an elaborate monetisation strategy that allows purchasing a single course, or a freemium model based on using the platform free of charge with access to a limited pool of the portal's functionality. Another option is a periodic PRO subscription, which offers advanced statistics and progress analysis. The subscription also provides access to more options in the course editor. Passed Pawn, in turn, allows purchasing individual courses. The revenue strategy is simple and clear: the customer knows what they are paying for and what they get.

1.3.2. Chessly

Chessly is an educational chess platform created by the American chess player and internet personality Levy Rozman. Since the platform launched in 2021, more than 900,000 users have registered on it.² The service currently offers more than 50 courses, and its monetisation model is based on a monthly subscription. The courses consist of interactive elements, descriptions and videos.

The fundamental difference between Passed Pawn and Chessly is that the latter does not offer ordinary registered users the ability to create and sell courses. Courses are created only by the founder

and his employees. This results in a much smaller potential for the number of courses available on the platform. The solution proposed in Passed Pawn, which allows anyone to create their own course, means that their number can grow quickly. The drawback of such a solution is that quality control is harder, but this problem can be partially solved by the course review system available on the described platform, in which any user can leave a review.

A monetisation model based on a monthly subscription may not suit many users. A single payment gives access to all courses, but there is no way to buy access to a specific course. This means that if a user is interested in, for example, a particular opening or strategy, they still have to pay the full price, which includes courses they are not interested in. Another problem is that after cancelling the subscription the user loses access to the courses. In Passed Pawn, a course once bought will always remain available to the user.

1.3.3. lichess.org

lichess.org is a free, open-source chess platform. Although its main functionality is playing games between players, it also has several educational tools. These tools can be divided into those offered directly by the platform and those that all users can add. The platform directly offers a wide set of chess puzzles, divided into motifs, at various levels of difficulty. There are also simple exercises that help in learning piece movement and basic tactical motifs. Users can add so-called studies. These are sets of examples with an interactive chessboard and descriptions, as well as puzzles that the player has to solve on their own.

Although all studies are free, they are usually not very extensive. Unlike courses in Passed Pawn, they usually consist of a few examples describing a single variation or strategic motif. The fact that all content is free is an advantage for students, but may be a big disadvantage for coaches. A coach can earn money only by offering private lessons. In Passed Pawn, coaches will earn directly from the courses they upload.

lichess.org is a non-profit association. The application displays no advertisements and is based on open-source software. Funding comes exclusively from voluntary donations from the platform's users. Such a business model differs fundamentally from the Passed Pawn model, which is focused on making a profit.

1.3.4. Summary of existing solutions

Existing educational chess platforms offer many possibilities for coaches and students, but each of them also has significant drawbacks. The choice looks particularly unattractive for chess tutors. The available solutions for creating one's own courses are unpolished. The Chessable platform, which has one of the most extensive sets of features for managing one's own courses, is unintuitive to use. The Chessly application does not offer the ability to create one's own courses, and lichess only offers a way to advertise private lessons. Passed Pawn, on the other hand, fills a gap in the chess application market by offering equally extensive tooling support for both the coach and the student. The platform is clear and simple to use. The environment created enables the business growth of tutors as well as

the deepening of knowledge by enthusiastic chess players, fostering the development of a passionate and mutually supportive community.

Notes

1. Chess.com. *How Many Chess Players Are There In The World?*
<https://www.chess.com/article/view/how-many-chess-players-are-there-in-the-world>.
Accessed: 2025-06-23. 2017.
2. Chessly. *Chessly main page*. <https://chessly.com/>. Accessed: 2025-06-23. 2025.

2. System requirements analysis

2.1. Functional requirements

The Passed Pawn platform was designed with both beginners and more advanced players who want to develop their chess skills in mind. To this end, the system offers a range of features that enable learning, practising and tracking progress. Other users can create and share learning materials.

2.1.1. User registration and login

The application allows creating a user account and logging in with a username and password. Users have roles, Student or Coach, and the user session is stored securely in the browser.

2.1.2. Browsing courses

The student can browse the courses available for learning. The user can clearly see how much they will pay for a course and what it is about. They can also see the recommended skill level of the student.

The application offers a wide selection of filters to make this activity as simple as possible. The filters are very responsive and react as soon as the user selects something.

2.1.3. Course details

The user also has the option of viewing the details of a selected course. Here the elements included in the course, its author and its reviews are visible. Here the user can also buy the course or add it to their list if it is free.

2.1.4. Learning from a course

If the course has been bought or added to the list, the user can start studying. Each course contains lessons, and each lesson contains its own elements. The platform supports four different types of elements, and each lesson can contain different elements in any quantity.

- **Puzzle** — A chess problem that has only one solution. The user gets access to an interactive chessboard and has to play the sequence of moves that seems best in the given position. If a move is incorrect, it is automatically taken back and the user has to make another attempt. It is an ideal way of learning specific openings or finding interesting tactical patterns.
- **Example** — The user sees a chessboard on which a position has already been prepared, optionally with arrows and coloured squares. In addition, the user receives a text explaining what is happening on the board.
- **Video** — The user gets access to a video, which makes it possible to explain any concept in a format convenient for both sides.
- **Quiz** — A question that the user must answer by choosing one of several options. It can be a simple text question, but it is also possible to display a chessboard and ask a question about it, for example "what is the best move in this position?".

2.1.5. Writing reviews

Every student can leave a review of a course they have learned from. Each review requires a rating on a scale of 1–5 and, optionally, a comment. Other users can browse the comments in the course details view, and the average rating is clearly displayed.

2.1.6. Creating courses

Users registered as coaches gain the ability to create courses. The application provides an advanced and intuitive editor that makes it simple to create any type of lesson element.

2.2. Non-functional requirements

Non-functional requirements are requirements that do not define the functionality of the system itself. Instead, they concern issues such as performance, security, reliability, usability, ease of use and scalability. Functional requirements can affect non-functional requirements and vice versa. For this reason, defining both functional and non-functional requirements is important in order to properly analyse and assess the idea for a system before its implementation begins or before it is fully implemented. A system may (but does not have to) be able to work even if it does not meet the non-functional requirements set for it.

- Security — the system should ensure that users' sensitive data is properly protected and that paid courses cannot be stolen
- Usability — the user interface should be intuitive, but at the same time simple and efficient to use. Both the student and the tutor should easily understand and master the functionality available to them
- Scalability — the system should allow new features to be developed easily, without the need to introduce major changes to the existing system.
- Portability — moving the system deployment from one hardware environment to another should not be a problem and should take place without any interruption to the service
- Reliability — the platform should be available at any time of day or night. The quality of service (loading times, absence of errors) must always be at a similar level that is satisfactory for the user.

2.3. Description of actors

As the application is a platform for learning from courses, there are 2 main actors — the Coach and the Student. The only actors of the application are users, since no external program uses Passed Pawn.

Coach

The **coach** is a user who has advanced chess knowledge and acts as a mentor to students. The application provides many tools for the coach that can be used to build all kinds of chess courses. The user is given complete freedom — it is they who decide what the course should look like and what elements it should consist of. The coach can also charge for their courses, but may also publish them for free.

Most important features available to the coach:

- Creating and editing courses
- Creating and editing lessons
- Creating lesson elements
- Managing and monitoring their courses and their content

Characteristics of the coach:

- Advanced knowledge of chess
- Willingness and experience in teaching others
- Desire to earn money from courses (optional)

Student

The **student** is a user who uses the application to broaden their chess knowledge and skills. This can be either a complete novice who is only learning the rules of the game or an intermediate player who wants to develop their strategic and tactical skills and deepen their knowledge of openings, the mid-dlegame and endgames. The student uses materials prepared by the coach and can contact the mentor in case of any doubts.

Most important features available to the student:

- Browsing and buying courses
- Studying based on the course content
- Creating lesson elements (if allowed)
- Managing and monitoring their courses and their content

Characteristics of the student:

- Desire to learn and broaden their knowledge of chess

2.4. Coach use case diagram

The diagram presents the use cases of the Passed Pawn system from the perspective of the coach, who can create and manage chess courses. The coach adds lessons and their elements (e.g. quizzes, videos, examples) and is also able to delete them.

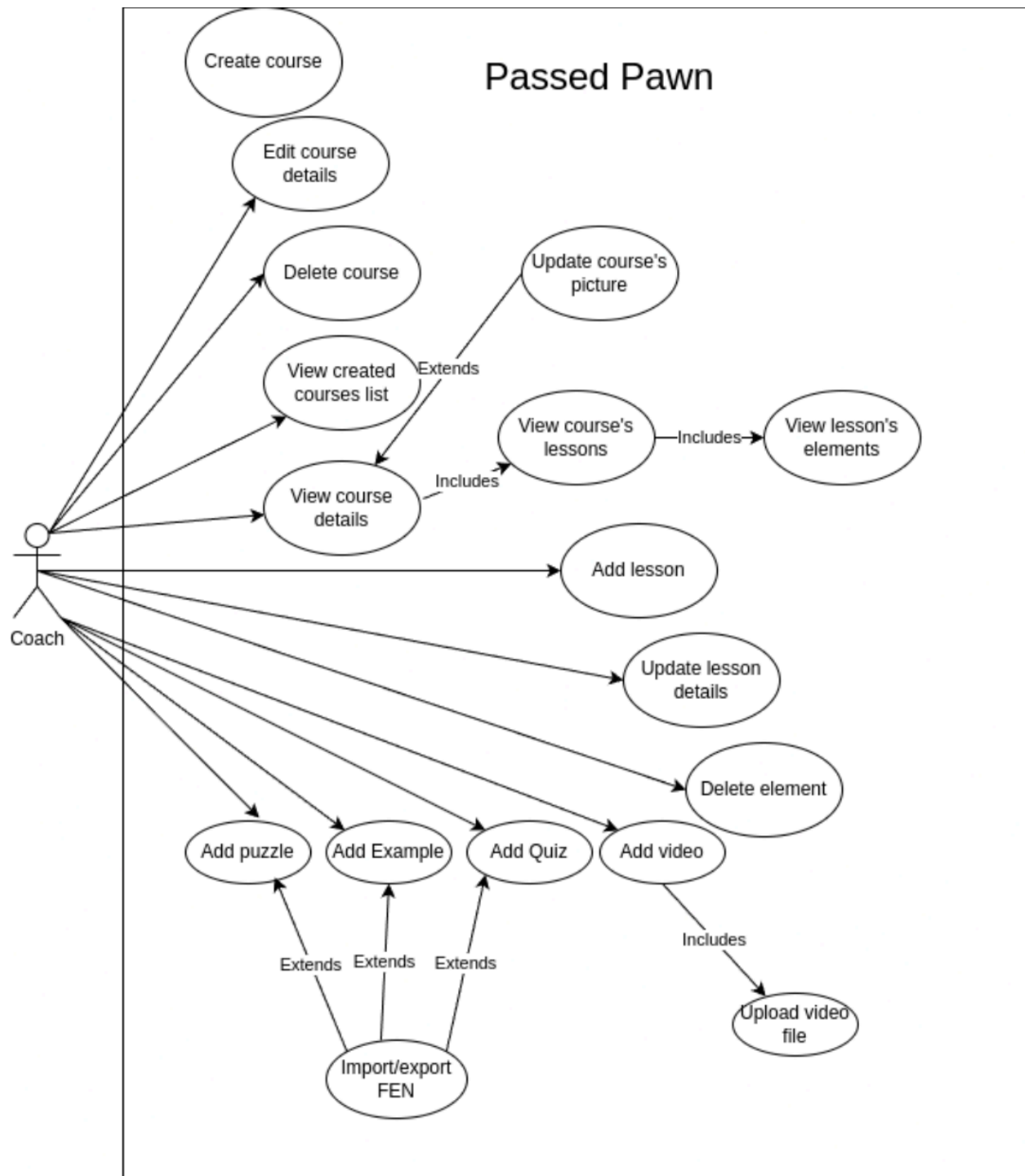


Figure 1: Coach use case diagram

2.5. Student use case diagram

The diagram presents the use cases of the Passed Pawn system from the perspective of the student, who can browse and buy courses and also leave reviews after completing them. The student has access to the lesson content (quizzes, videos, examples and puzzles) and to information about the

coach. The system also makes it possible to filter available courses and preview their details before purchase.



Figure 2: Student use case diagram

2.6. Use case diagram of features shared by the coach and the student

The diagram presents the features that are available to both the coach and the student. There are not many of them; they mainly concern the processes of creating and using accounts, as well as the built-in AI assistant.

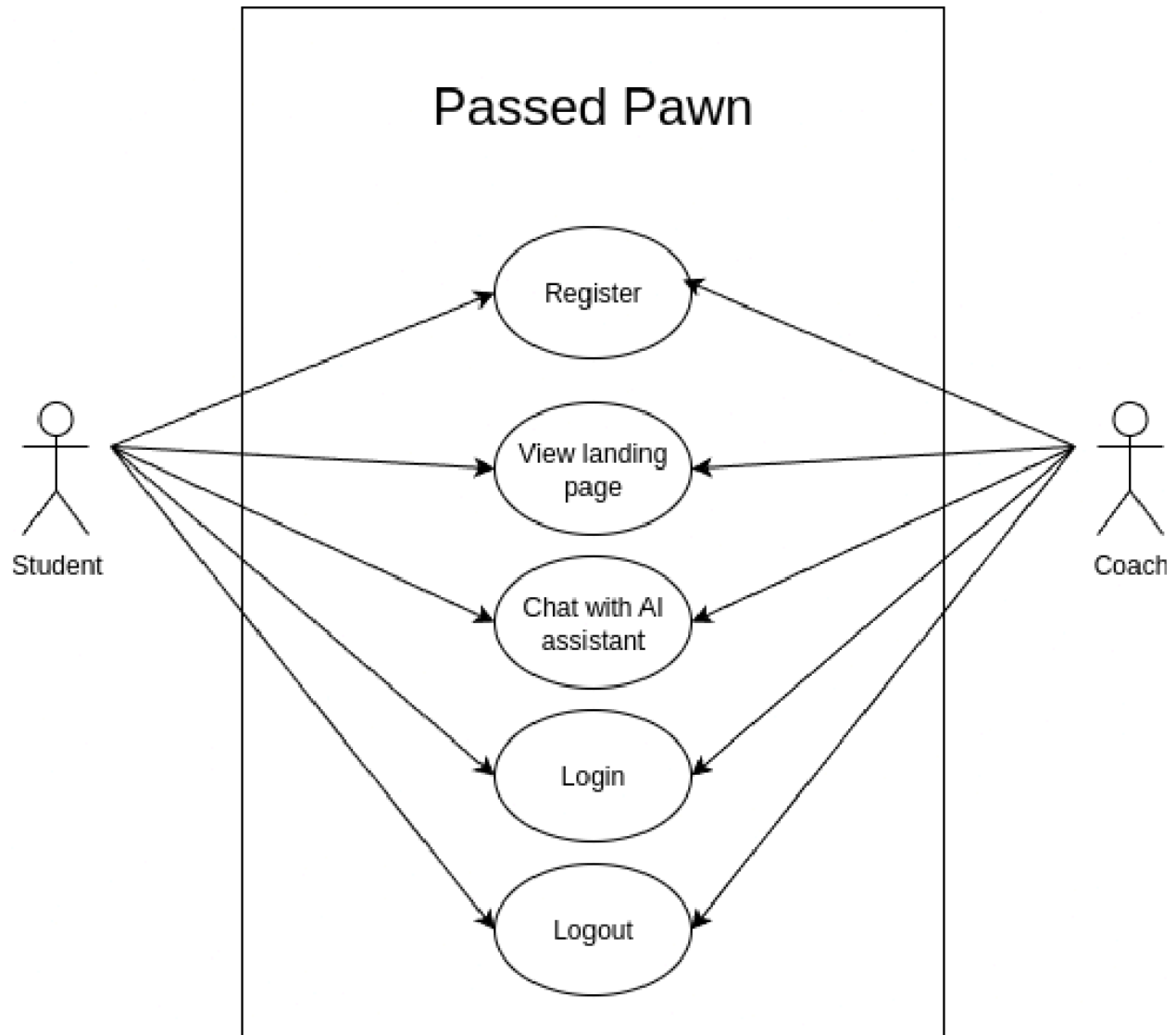


Figure 3: Use case diagram of features shared by the coach and the student

2.7. External systems the application relies on

2.7.1. Stripe

Stripe is a platform for handling online payments that makes it possible to accept payments by card, bank transfer and other methods. Apart from a clear and convenient admin panel, Stripe also offers tools for many technologies that greatly simplify developers' work. Passed Pawn uses Stripe to receive payments for course purchases.

The tools offered by Stripe greatly supported the design of both the client and the server application. For example, Stripe is able to send requests to the Passed Pawn API using webhooks, and its tools ensured security and a guarantee that the requests are not sent by third parties.

2.7.2. Cloudinary

Cloudinary is a cloud service for managing images and videos. Passed Pawn uses both of these offered services, and Cloudinary provides security and performance for these resources.

The tools provided by Cloudinary enabled the developers to design the entire logic of saving and modifying resources without the need to use the API manually. This applies to both the server and the client application.

2.7.3. MistralAI

MistralAI is a company offering access to cloud-hosted large language models (LLMs). Requests to these models can be made using the provided API. Unlike many alternatives, MistralAI offers very generous free features, which turned out to be sufficient for the needs of the Passed Pawn application.

3. User interface design

User interface design is a foundation in the world of today's web applications. It is a direct factor influencing the "clickability" and conversions of an application, which are a measure of business success.

The graphical interface should be aesthetically pleasing, and on the other hand intuitive and efficient, which may be complicated by the business logic being implemented. Building a coherent whole is a difficult task, which is why it is extremely important to analyse and design the UI thoroughly so that it generates as much traffic on the site as possible. The web application market is very demanding. In keeping up with today's competition, the appearance of the interface is a very important factor — it is the proverbial book cover, on the basis of which we have on average 10 seconds to convince the user to stay on our website. A polished UI is also the quality that we directly present to the user and, consequently, trust, which is a key piece of the puzzle.

3.1. The UI design process

The design of the visual layer began with examining competitors' interfaces and extracting from them the best elements and aesthetic techniques worth using. On the other hand, errors and visual aspects that should be avoided in order to interest the user in the product were analysed. After the team had set out and approved the values and patterns it would follow, it was time to design the general outline of the site.

Before starting the aforementioned 2nd stage, a set of key features, the so-called MVP, to be implemented was established in order to identify the individual elements and their arrangement on the page. The work began with the general skeleton of the views with the chessboard, and then moved on to designing less important elements, such as the components of the available courses and the coach profile. After the entire process of modelling the skeleton was complete, a simplified outline of the site emerged.

The next stage was the detailed design of individual components and placing them on the previously prepared model. After applying everything, the first version of the finished interface appeared, which was naturally subjected to analysis and possible corrections.

The next stage focused on selecting the icons and graphics visible on the site. The niche character of the chess application market forced the team to design some of the icons themselves, as ready-made UI libraries were unable to provide them.

The final stage of the UI design process, after combining all the elements, was an in-depth analysis of the created model. The final appearance of the interface was assessed and simple user experience (UX) tests were carried out on the site model. After a team consultation and the introduction of final corrections, the UI design process was officially completed. It is worth adding that over the course of the chess application's development the UI design underwent slight changes and corrections. When a

new feature was added, it was necessary to prepare a visual concept for it and integrate it with the existing model.

3.2. Selected user interface design tools

The UI design of the chess application was prepared in Canva. It is a popular online platform for creating graphics and visual designs. The intuitiveness and simplicity of the program allowed the team, which had little experience in user interface design, to get started immediately.

The work began with designing the independent views mentioned earlier, which in the final phase came together into a complete UI design. The project made use of Canva's rich graphics library, which allows efficient searching and advanced editing of icons and multicoloured graphics. For the needs of the chess application, custom icons and graphics composed of geometric shapes and chess piece icons were created.

In hindsight, Canva does not seem to be the best design software for web application developers. The main drawback of the program is that its available tools do not fully reproduce the capabilities of Cascading Style Sheets (CSS). Canva does not offer shadows (the equivalent of box-shadow in CSS) and has limited capabilities for creating gradients, which significantly affected the efficiency of creating the UI model and forced the team to rely on additional software.

An alternative solution would have been to use Figma, an online interface (UI) design tool that allows rapid prototyping of websites and applications. In building web applications, it facilitates collaboration between designers and developers. The proposed software fills the tooling gaps left by Canva; it would also have allowed a more accurate design of the model and catching elements that needed to be rebuilt already at the design stage, instead of making corrections during the implementation of the client part of the application.

Another problem with Canva is its lack of scalability. The template offered for websites resembles the one for creating presentations and performs poorly for software UI design. The linear layout (one slide below another) is unreadable and inefficient in larger projects. Figma has an excellent solution here, as it allows the project to be developed in the form of an interactive board, enabling a visual arrangement of the individual application views, which promotes scalability and convenient navigation.

The problem with Figma is its learning curve, which is much slower than in the case of Canva, which stands out for its simplicity. The key factor in choosing the tool, however, turned out to be previous experience with Canva, which cannot be said of Figma, in which the team had no experience. In the final reckoning, it is difficult to estimate the time lost due to the tooling limitations encountered in Canva, because learning to navigate and use Figma would also have required an undetermined amount of work.³⁴

Table 1: Tool comparison: Canva and Figma

Feature	Canva	Figma
Use in web design	Basic – good for simple mockups or landing pages	Professional tool for creating complete UI/UX designs

Feature	Canva	Figma
Ability to create interactions	Limited (linking between pages)	Advanced prototyping with animations, interactions and states
Grid and layout precision	Limited control, no advanced grid tools	Precise grid tools, auto layout, components
Team collaboration	Possible, but simplified	Advanced real-time collaboration
Export to code (e.g. HTML/CSS)	No direct support	Integration with tools such as Zeplin, Anima, Figma to Code
Creating reusable components	Limited possibilities	Full support for components, variants and libraries
Responsive designs	No support for responsiveness	Auto layout and scaling support – responsive design
Learning curve	Very easy, intuitive	Medium – requires time to master advanced features
Target group	Beginners, marketers, graphic designers for simple projects	UI/UX designers, developers, product teams

The table was prepared on the basis of an external source in the form of an online article.⁵

3.3. Limitations of UI libraries

PrimeNG is a great solution that makes it possible to use ready-made UI components in one's project, and despite its huge base of high-quality elements, the library has its limitations.

- The chess application contains a form for rating chess courses. The platform's priority is clarity and simplicity; using the portal is meant to be a pleasure, which is why the form field for rating a course should follow the idea of convenience and simplicity. Unfortunately, the library does not offer a ready-made component (stars) for giving a rating that would allow reviews to be submitted intuitively.
- The form field for giving a rating is half the battle; we also need a component that will represent the average rating of a course in the form of stars. PrimeNG has such a component, but it does not support half-stars, which are needed to represent the average score more accurately, as it is usually a floating-point number.
- PrimeNG also does not have a component (form field) that would clearly present the difficulty level of a course. Here, too, a custom implementation of such an element has to be used.

3.4. Custom UI components and views

Using the chess application is meant to combine pleasure with learning, which is why the portal's interface is as simple and intuitive as possible. The chosen colours and style of the elements on the page are meant to interest the user in using the product and help them develop their chess skills. The key view is the first page seen by a potential customer, i.e. the landing page. The user scans the page for a few seconds on average, and then makes an important decision about whether they are interested in exploring our product further. The goal of the landing page is to present the offer in a quick and understandable way.

The home page consists of thematic graphics, a concise headline and 2 distinctive buttons (calls to action), which are meant to attract the attention of a potential customer.

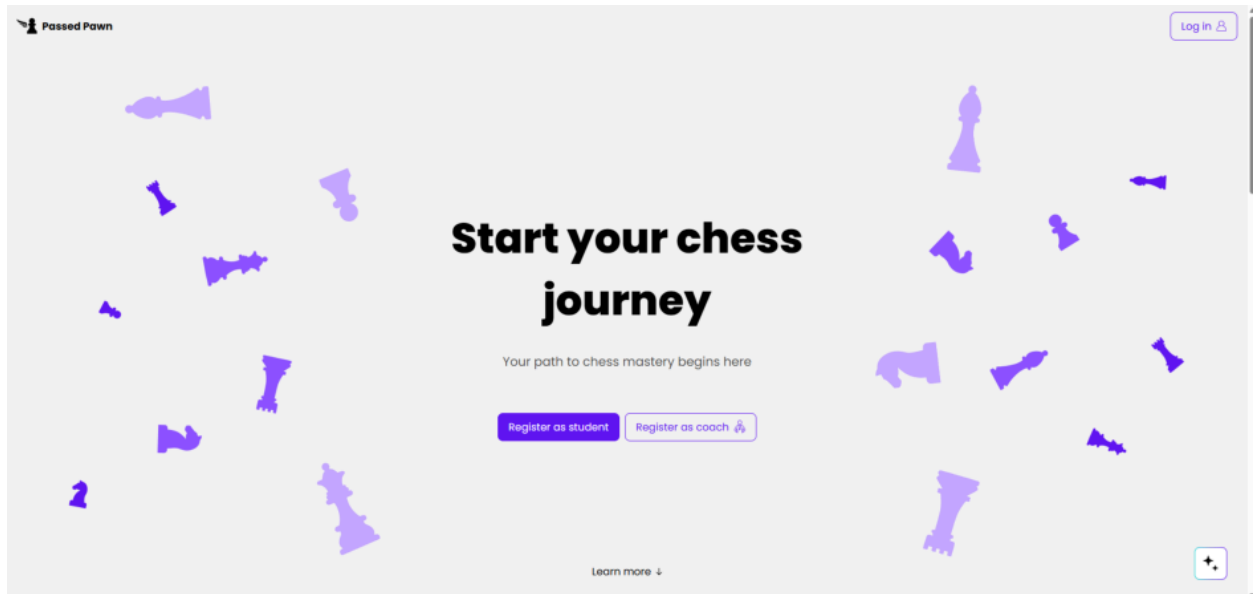


Figure 4: Landing page of the chess application

Below it is a section showing the content of the course and the recommended learning model, from which the user can learn the details of the services offered and the structure of the chess courses. The sections are minimalistic and contain little text, replaced with icons, in order to present the process visually and not overwhelm the user with unnecessary information.

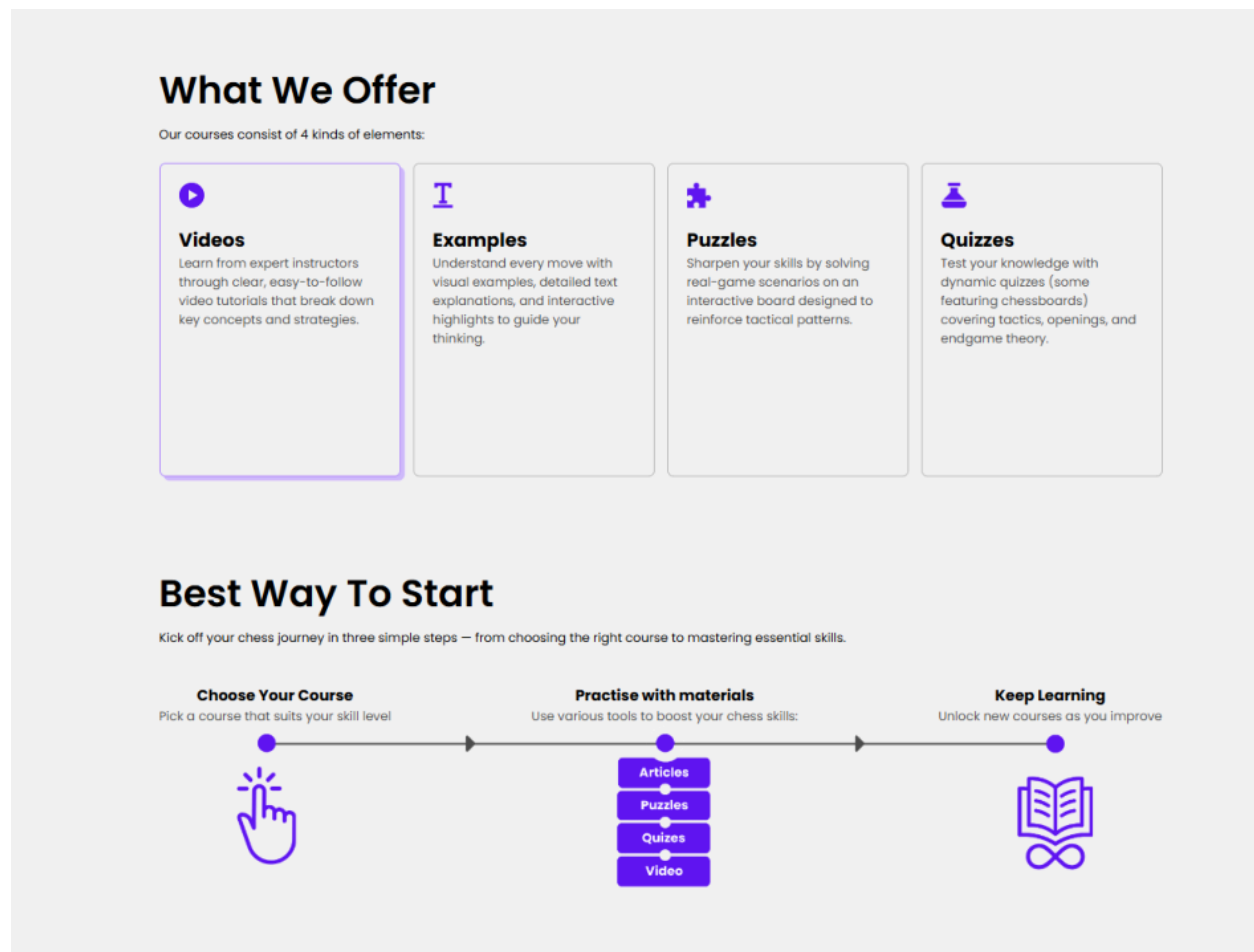


Figure 5: View of the course offer and the proposed learning model

At the very bottom there is a section of questions asked by users interested in the product, which address important and popular topics related to the software offered and automate the communication needed to learn the details the user is interested in.

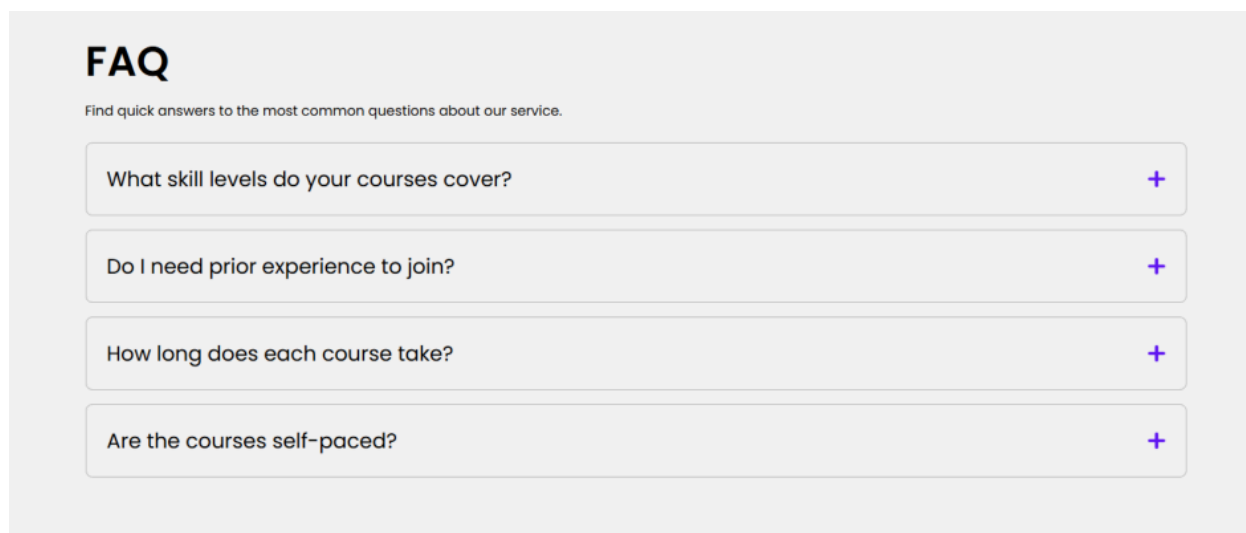


Figure 6: Landing page, questions section

Puzzle Builder component — a view in the coach account that allows creating puzzles for chess courses. The component is divided into 4 sections (logical units):

- **Info and Board Operations** — contains text fields for basic information about the puzzle and a button that allows operations on the chessboard.
- **Chessboard** — the centre, the most important element of the component; the other sections are subordinate tools.
- **Fen** — an element for loading a FEN onto the chessboard in its basic version (there is a second, extended version). FEN (Forsyth–Edwards Notation) is a notation that makes it possible to represent the position on the chessboard at a given moment of the game in text form. The notation consists of a part representing the placement of the pieces and a part presenting other data such as castling rights, en passant, move counters, etc. The element contains a text field and a button that applies the entered FEN to the board. The component signals an invalid FEN format to the user by visibly deactivating the button (the button loses its colour).
- **Puzzle** — the most important subordinate section, which works directly with the chessboard. The element presents the move history and allows setting both the starting and the final position of the puzzle. The coach can also interact with the move history (go back to a selected position). The interface of this section is divided into 3 main elements showing the puzzle creation process. Going from the top, the 1st element is the place where the state of the starting position is shown. A square with an icon means that the starting position has been selected, while an empty square means that no position has been applied; the same applies to the element with the state of the final position (at the very bottom). In the middle there is an interactive table with the history of moves made, together with an arrow showing the course of the puzzle creation process (selecting the starting position → moves on the board → selecting the final position). The whole thing visualises the state of the parameters of the puzzle being created in a simple way.



Figure 7: View of the puzzle creation component

Quiz/Question and answers component — the component of the 3rd stage of creating a quiz for a chess course, located in the coach view. The element is divided into 4 sections (logical units):

- **Board Pieces and Board Operations** — contains a list of pieces that can be dragged onto the board and a set of buttons that allow operations on the chessboard. The section gathers tools that interact directly with the chessboard and its pieces.
- **FEN** — the extended version of the FEN component, enriched with a panel that allows configuring the notation. The section contains custom-implemented form fields without the use of the PrimeNG library (checkbox fields).
- **Chessboard** — the centre of the component; the other sections are subordinate tools.
- **Quiz and Addings** — divided into 2 tabs. The first of them is Quiz, containing form fields — the question and answers, which are highly interactive. The coach can delete (the red colour of the button emphasises the criticality of the operation), edit and assign a board to answers (similarly to the puzzle builder). A square indicates the state of the answer (whether it has a board assigned). On the left side of the answer panel there are form fields (radio inputs), which the coach selects when choosing the correct answer. The second tab is Addings, containing text fields with the explanation of the solution and a hint. The content of the tabs separates the tools into the more important and more frequently used ones — Quiz — and the less important ones — Addings. This division creates an intuitive and efficient interface for the coach.

3. Question and answers

Back

Next →

Board Pieces

Board Operations

Starting position

Base position

Flip Board

Clear

8

7

6

5

4

3

2

1

g

f

e

d

c

b

a

FEN

Side to move

White short

White long

Black short

Black long

Half move clock

Full move number

White

0

1

rlbqkbnr/ppppplpp/2n2p2/8/3PPN2/PPP2PPP/RNBQKB1R w KQkq - 0 1

Quiz Addings

Question

What should white do?

Answers (2 - 10)

Correct Answer

Applied Board

☒

Move pawn to C3

☐

Move bishop to D4

☐

Enter answer text

+ Add Answer

Figure 8: View of the quiz creation component, stage 3

An input field component used in the form for submitting and editing a user's review of a chess course. The PrimeNG library does not have a ready-made equivalent. The intuitive interface allows a rating to be given simply and quickly. After hovering over a star, the user sees the corresponding rating, which is confirmed after clicking. A rating can be given with an accuracy of half a star. When editing a rating, the component displays the previously confirmed value, while at the same time allowing the rating to be changed to a new one.

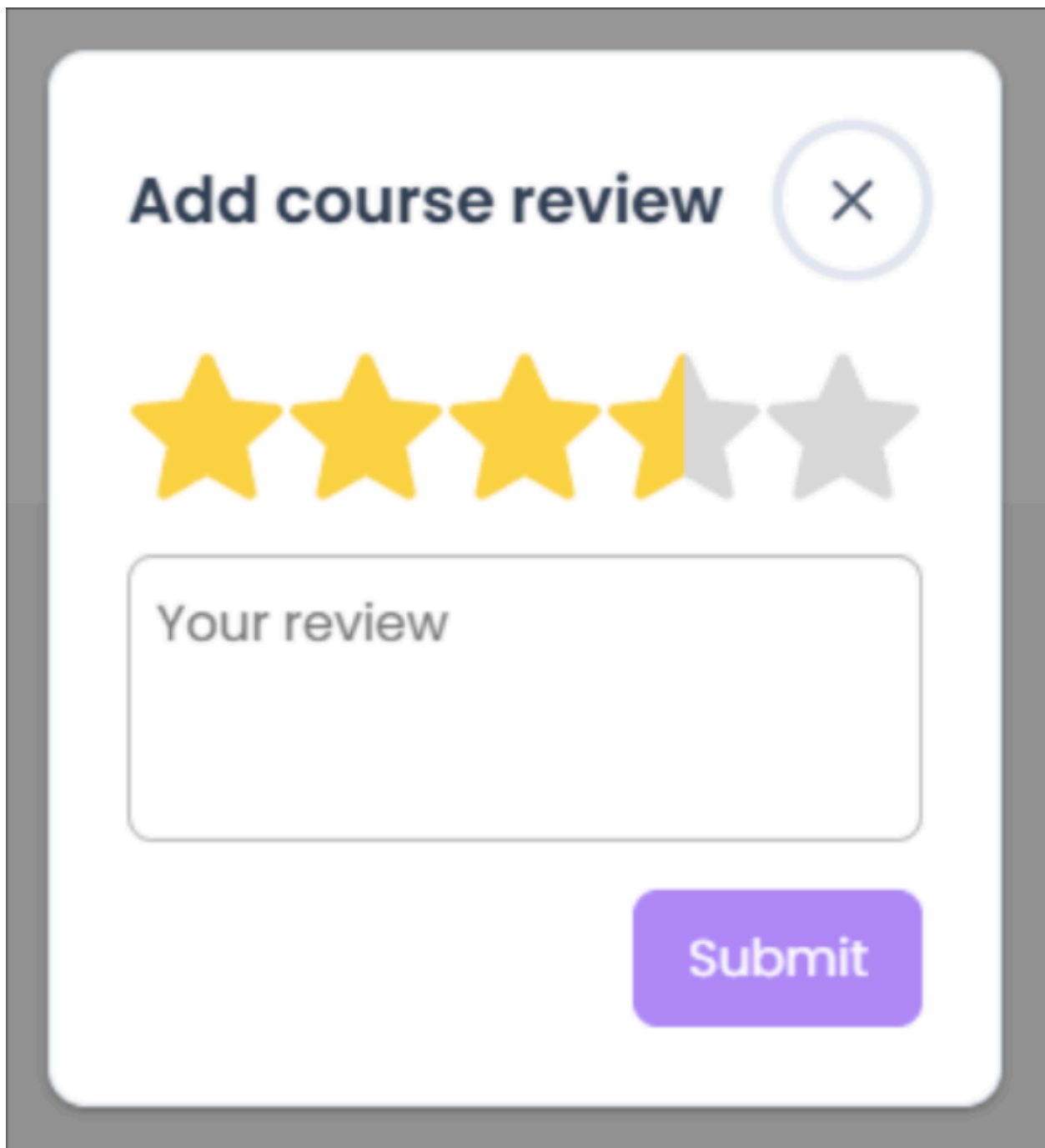
The image shows a user interface for adding a course review. It features a title 'Add course review' in a bold, dark blue font. To the right of the title is a circular close button with a dark blue 'X' icon. Below the title is a row of five stars. The first three stars are solid yellow, the fourth is half-yellow and half-grey, and the fifth is solid grey. Underneath the stars is a large, rounded rectangular text input field with the placeholder text 'Your review' in a light grey font. At the bottom right of the form is a purple rounded rectangular button with the word 'Submit' in white text.

Figure 9: Course review component

The component presents the rating given to a chess course. The PrimeNG library offers a ready-made element that does not support the half-stars used in the application. Therefore, a custom component was implemented in the chess application that allows ratings to be displayed with an accuracy of half a star. Thanks to this, full integration of the system for submitting and presenting reviews is possible.



Figure 10: Component presenting the average course rating

An input field component for the difficulty level of a chess course. PrimeNG does not offer a suitable element in its collection that meets these requirements. The component is used in the chess course search, where the user can filter the results based on the difficulty level, presented as a range of values. The element makes it easy to set the limits of this range, with the possibility of editing them. The module has been equipped with a button for resetting the settings, because editing only allows the difficulty range to be extended, without the possibility of narrowing it. At the bottom there is purple text informing the user about the selected range (in this case [from beginner to expert]).



Figure 11: Input field component for the course difficulty level

A diagram presenting the content of a chess course, located on the details page. The component visualises the number of individual course elements through the length of the corresponding bar of the chart. The diagram presents the data on a relative scale, in which the bar with the largest value is the reference point (100% of the scale), and the remaining bars are adjusted proportionally to it. The chart informs the user about the variety of materials and allows them to assess the attractiveness of the course.



Figure 12: Component, diagram presenting the content of a chess course

Interactive arrows on the chessboard that make it possible to visualise piece moves. They appear in the view for creating examples (1 of the 4 atomic course elements) and are available in 4 colours at the coach's disposal. An arrow is created by holding down the right mouse button, dragging the cursor to the target square and releasing the button. To delete a specific arrow, this process must be repeated, while clearing the board of all arrows is possible thanks to the "Clear arrows" button.

The PrimeNG library does not offer an implementation of this component; even if a ready-made model existed, integrating it with an independently created chessboard would be problematic.

The arrow display mechanism is based on simple vector calculations in a Cartesian coordinate system. Knowing the [X, Y] coordinates of 2 points (the starting and the target point), we are able to calculate the length and angle of the arrow on the chessboard.

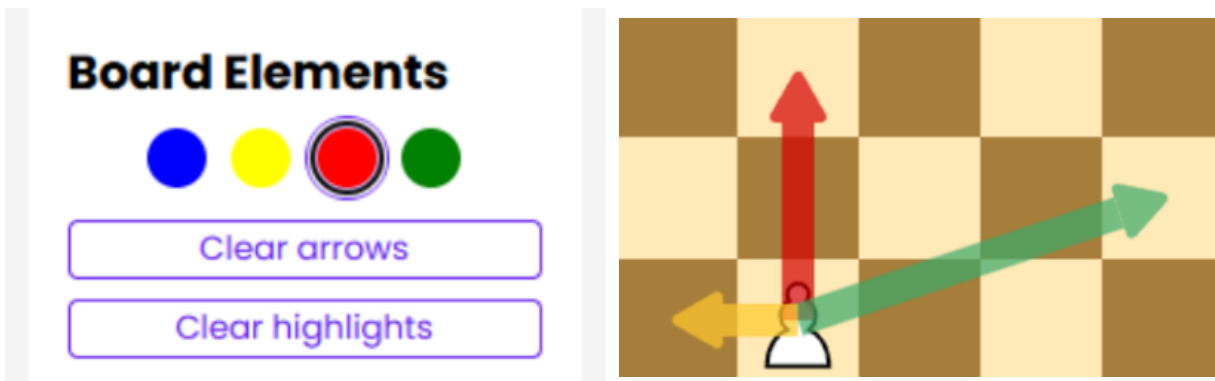


Figure 13: Control panel and the system of interactive arrows on the chessboard

The AI assistant is an expandable component that allows communication with Bob the elephant (the mascot of the chess platform) in order to get help with using and navigating the application. Bob is a helper who answers customers' questions. The AI is available to logged-in and anonymous users in the bottom right corner of the chess application.

The component's interface consists of a chat, in which the conversation with the artificial intelligence is displayed, and a text field for asking questions. The component lets the user control the visibility of the content by expanding and collapsing it.

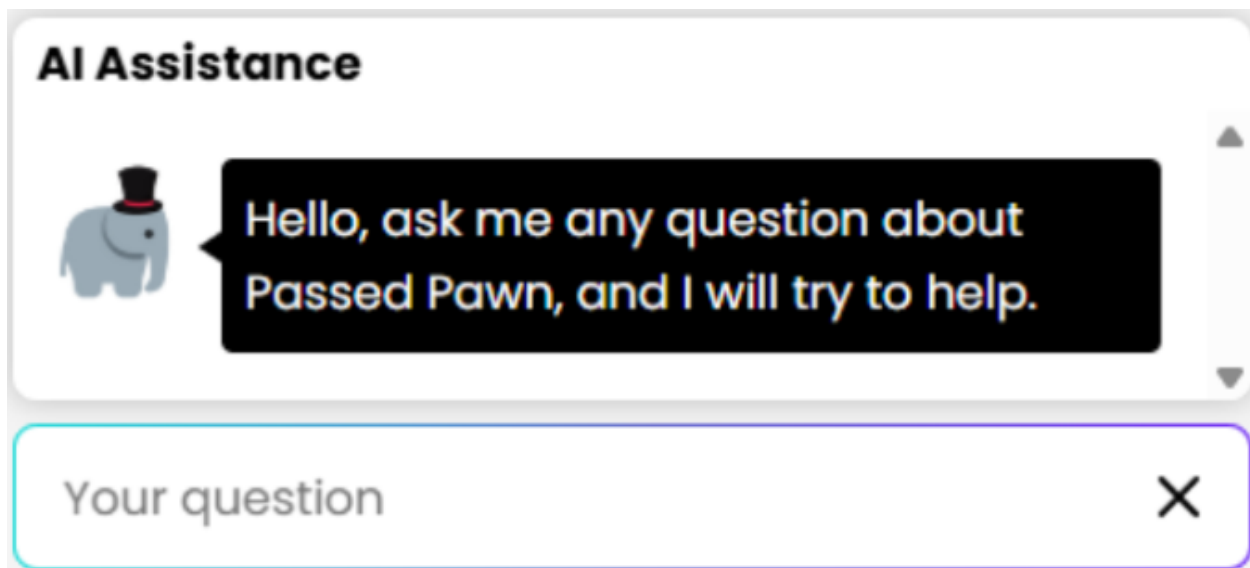


Figure 14: AI assistant

3.5. The importance of an original user interface design

When creating a website, the development team focuses on producing a clear implementation of the project that follows clean code principles. From the point of view of the developers responsible for the development of the project, this is very important, as it affects the efficiency of their work.

However, if we look at applications from the customer's perspective, the key is the design, which is responsible for ease of use and clarity of information. The user does not judge the quality of the server application's code; it has no significance for them. The hierarchy of priorities for a website's customer can be simplified to 2 main aspects: the application supports the user in achieving their goals and allows them to achieve them in a quick, intuitive and friendly way.

An original design is the product's calling card, which distinguishes it on a market of growing competition. The appearance and behaviour of the interface are the main factor influencing how the user will remember the site. Another aspect of originality is the minimal use of external UI libraries. The advantage of this approach, apart from a unique interface, is full independence and creative freedom. The team is not limited by a set of ready-made, partially configurable components used in thousands of other applications.

The absence of external libraries also means no additional heavy dependencies, which generate redundant code and make optimisation more difficult. The advantage of the originality of our design is flexibility in modifications. Developers are not limited by the capabilities of the aforementioned UI libraries and can adapt to the user's needs without any problem.

Table 2: Comparison: custom UI design vs ready-made UI library

Criterion	Custom UI design	Ready-made UI library
Uniqueness and originality	Fully unique look and style	Limited individuality, template-like look

Criterion	Custom UI design	Ready-made UI library
Fit to project needs	Maximum flexibility and control	Limited to the available options and configuration
Performance and bundle size	Fewer dependencies, lighter code	Larger bundle due to excess unused code
Speed of delivery (MVP)	Requires more time and work	Faster prototyping
Scalability and modification	Easy to adapt to changes and project growth	Modifications often difficult or requiring workarounds
Technical requirements / experience	Requires knowledge of CSS, UX and front-end	Lower entry barrier for the developer
Design consistency (team/larger project)	Standardisation must be taken care of yourself	Built-in component consistency
Community support / documentation	None – you create it yourself	Strong support and ready-made examples

3.6. Problems and lost resources

Decisions regarding the project were always subject to discussion and validation by all team members, which effectively increased the chances of taking the right and rational steps regarding the further development of the project.

Unfortunately, designing version 2.0 of the user interface was not a justified action and contributed to the loss of dozens of hours of work that the team should have devoted to building key features. The project ended with only the landing page model being developed, with "styled" components and newly chosen colours, replacing the monochromatic purple style with new colours (blue, orange and red). The model as a whole looks very good and meets the assumptions set for it. Version 2.0 will be expanded in the future and eventually applied in the chess application, but this does not change the fact that undertaking this work during the period of intensive development of the application's MVP was ill-considered.

Another mistake was designing the user interface in a poorly suited tool, namely Canva. Melanie Perkins' application is poorly suited to modelling the UI of websites, because it is unable to reflect the capabilities of CSS (gradients and shadows). As the size of the project grows, the efficiency and ease of navigating the application decrease. During the development of the chess application, a drop in Canva's performance was also noticed (the application froze and needed a lot of time to load the project).

A further complication was the use of simple PrimeNG components that the team was able to implement itself (buttons and some form fields). The choice of the UI library was justified by saving time on implementing already available components, which resulted in the loss of valuable minutes trying to modify ready-made elements and ultimately replacing them with hand-written code.

To sum up, most of the problems encountered concerned the loss of a single resource, namely time, which resulted in a longer construction time for the chess application.

Notes

3. Canva. *How to Use Canva: A Beginner's Guide*. <https://www.canva.com/learn/how-to-canva-beginners-guide/>. Accessed: 2025-06-19. 2025.
4. Figma. *Browse component properties*. <https://www.canva.com/learn/how-to-canva-beginners-guide/>. Accessed: 2025-06-19. 2025.
5. Hitesh Sahni. *Canva vs Figma comparison: which design tool is right for you?* <https://hypegig.com/canva-vs-figma-comparison/>. Accessed: 2025-06-19. 2025.

4. Passed Pawn architecture

Architecture in relation to IT projects can mean very different things, and various types of architectures designed within a given undertaking can be named. It is therefore worth quoting a general definition of the concept of architecture itself, which will make it possible to understand the universal sense, meaning and characteristics that connect different types of architectures. Dr hab. inż. Jan Werewka, professor at WSEI, defines architecture as "[...] the art and science of planning, designing and organising spaces, structures or systems so that they meet specific functional, aesthetic and technical requirements."⁶ Professor Werewka lists the following most important basic characteristics of architecture:⁷

- **Structure:** The way in which the elements of the system are arranged and how they are connected to each other.
- **Building Blocks:** In every system there are units that perform specific functions in building the whole system.
- **Design principles:** A set of principles governing the process of design and development, and the monitoring and maintenance of the built object.
- **Goals:** Architecture has specific goals, which are: functionality, aesthetics, scalability, performance, security, compliance with user requirements, etc.

In the IT context, on the other hand, he distinguishes the following types of architectures, which will help to precisely determine the type of architecture discussed in this section:⁸

- **Business Architecture** — Concerns the business structure of the organisation, including business processes, work organisation, business models and products. It sets business goals, motivation and the operating strategy.
- **Information Architecture** — Focuses on the management and flow of information in the organisation, covering the modelling, organisation and life cycle of data.
- **Application Architecture** — Defines how applications in the organisation are designed, integrated and managed, including the division into layers (presentation, business logic, data).
- **Software Architecture** — Focuses on the structure and organisation of code, covering the division into modules, classes, components and design patterns (e.g. microservices, MVC).
- **Data Architecture:** Covers the structure, management and storage of data in the organisation, including data models and storage strategies (e.g. Big Data).
- **Technology Architecture** — Defines the technological infrastructure, including hardware, software and networks, needed to achieve the organisation's goals.
- **Integration Architecture** — Concerns the ways of integrating systems, applications and data in the organisation, covering various tools and methodologies (e.g. middleware, ESB).
- **System Architecture:** Covers the design and organisation of entire IT systems, their components and the way they are integrated.
- **Security Architecture** — Focuses on securing information, systems and processes in the organisation through the use of security policies and protection mechanisms.
- **Network Architecture** — Deals with the design and management of the network infrastructure that enables communication between systems and devices (e.g. LAN, WAN).

- **Cloud Architecture** — Covers the design and organisation of IT resources in the cloud, including the management of SaaS, PaaS and IaaS services and the integration of the cloud with on-premises infrastructure.

The thesis focuses on the technical analysis of the Passed Pawn project, so Business Architecture and Information Architecture are not presented in it.

4.1. Examples of commonly used types of system architectures

4.1.1. Monolith

Monolithic architecture is a traditional approach in which the whole system is developed and deployed as a single unit. All components, such as the user interface, business logic and data layer, are tightly connected with each other and share a code base. The key element of monolithic architecture is the centralisation of all system functions in a single application.

Characteristics of a monolith

- **Compactness** — despite the division into modules, the whole application is still deployed as a single application, which requires coordination of module development
- **Modularity** — a well-built monolith is divided into logical, independent modules that have clearly defined interfaces
- **Loose coupling of modules** — in a well-designed monolith, changes in one module have minimal impact on the others, which makes managing and developing the system easier

Advantages

- **Simple deployments** — the existence of a single deployment artefact makes deploying the system easier
- **Easier testing** — concentrating the code in one place and in a single unit that just needs to be run can make small and medium-sized systems easy to test
- **Easier analysis** — concentrating the code in one place and in a coherent technology spectrum makes small and medium-sized systems easier to analyse
- **Low performance overhead** — no communication delays between modules, which results from in-process communication instead of, e.g., network communication

Disadvantages

- **Costly and inflexible scaling** — scaling individual modules is impossible or difficult, so the default way of scaling is to scale the whole, which may be undesirable and generate unnecessary costs
- **Increased vulnerability and cascading failures** — an application error in one module can affect the whole system
- **Low flexibility** — less flexibility in choosing technologies for individual parts of the application

4.1.2. SOA (Service-Oriented Architecture)

SOA (Service-Oriented Architecture) is an approach to system design in which the system is built as a set of loosely coupled and autonomous applications/services. By design, each service performs a

specific business task, and communication takes place via standard network communication protocols and styles, such as SOAP or REST. A key aspect of this architecture is the ability to reuse the functionality contained in the services.

Characteristics

- **Modularity and reusability** — services are designed as independent modules that can be reused in different applications and contexts
- **Autonomy and flexibility** — changes in one service should not require modifications to others, which enables independent deployment and provides flexibility
- **Encapsulation of implementation details** — the implementation of each service is hidden and only the interface is exposed to consumers, which allows, for example, the selection of different technologies and programming languages suited to the needs of the service
- **Composition instead of integration** — applications are created by composing autonomous services

Advantages

- **Flexibility and scalability** — easier addition, modification and removal of services without disrupting the operation of the system, which allows adaptation to changing business requirements. Services can be deployed and scaled independently
- **Reuse of components** — reduced development time and costs thanks to the use of existing, proven components.
- **Easy integration** — makes it easier to connect various internal and external systems thanks to standardised communication protocols
- **Reduced complexity** — dividing systems into smaller services lowers maintenance costs and increases clarity
- **Resilience and reliability** — the failure of one service does not have to affect the whole system, which increases fault tolerance
- **Security** — the ability to manage security in detail at the service level, which is crucial in distributed applications

Disadvantages

- **Management complexity** — requires appropriate architectural oversight and an effective operating model⁹
- **Greater infrastructure requirements** — infrastructure is needed to manage the services and their communication
- **Increased network latency** — communication between services carries additional overhead related to sending information over the network
- **Difficulty of proper implementation** — poorly thought-out, inorganic and poorly managed service contracts and boundaries can reduce their autonomy, while still leaving the drawbacks in the form of greater management complexity and additional network overhead that negatively affects performance

4.1.3. Microservices

Microservice architecture consists in building a system as a set of small, independent services/applications that most often communicate with each other over the network and have clearly

defined logical and physical boundaries. The key element of this architecture is the autonomy and independence of individual microservices, which allows them to be deployed on their own without coordination with other microservices.

Characteristics of microservices

- **Independence and autonomy** — each microservice is by design independent and can be developed, deployed and scaled separately
- **Single responsibility** — each microservice is by design specialised to achieve a single goal with a well-defined scope
- **Own database** — in a microservice architecture, microservices often have their own databases, which is related to the pursuit of as much independence as possible from other parts of the system and other microservices
- **Lightweight communication mechanisms** — microservices usually communicate with each other through lightweight communication channels such as REST, gRPC or via message queues and event brokers, e.g. RabbitMQ or Apache Kafka, in order to reduce the performance overhead of network communication as much as possible and to reduce coupling between microservices (applies to the use of message queues and event brokers)
- **Technological diversity** — teams can choose different technologies and programming languages to work with individual microservices

Advantages

- **Scalability** — individual microservices can be scaled independently depending on demand
- **Flexibility and speed of development** — independent teams can work on different microservices, which makes it easier to develop large, complex systems that would otherwise have to coordinate many teams and the work of dozens or even hundreds of programmers
- **Fault tolerance** — the failure of one microservice does not affect the whole system
- **Technological freedom** — the ability to choose the best technology for each microservice
- **Easier maintenance** — smaller and more specialised components are easier to understand and maintain

Disadvantages

- **Management complexity** — managing a large number of independent microservices can make analysing the system as a whole significantly more difficult and introduces a whole range of problems related to network traffic, which may not pay off in small and medium-sized projects
- **Difficulty in maintaining data consistency** — managing data consistency in a distributed system can be much more difficult and often requires the use of complicated architectural patterns, e.g. a system of distributed transactions
- **Increased network latency** — communication between microservices carries additional overhead related to sending information over the network
- **Difficulty of proper implementation** — poorly thought-out, inorganic microservice boundaries can eliminate their independence and autonomy, while still leaving the drawbacks in the form of greater complexity in management and debugging and additional network overhead that negatively affects performance

4.1.4. Big Ball of Mud – an anti-pattern

Big Ball of Mud is a system architecture anti-pattern that describes a system with no clear structure, consisting of components connected in a thoughtless way, without maintaining high standards and conscious conventions in the application code. The key element of this type of architecture is the lack of a well-thought-out design and applied conventions, which leads to chaos and difficulties in management.

Characteristics of a Big Ball of Mud

- **No clear structure** — no logical division into modules and layers
- **Strongly tangled and coupled code**: there are strong connections between many places in the code and its dependencies, which makes it difficult to analyse and modify
- **Unclear component boundaries** — functionality is scattered in many places, without clear assignment to specific components
- **Lack of documentation** — usually no or insufficient documentation of the system.
- **Cyclic dependencies** — components depend on each other, which makes testing and maintenance difficult

Why a Big Ball of Mud is a counter-example

- **Difficulties in maintenance and development** — modifying and extending the code is risky and carries a high probability of breaking existing functionality.
- **Poor scalability** — the lack of proper structure makes effective scaling of the system difficult.
- **Low code quality** — inconsistency, code duplication and a lack of standards lower the quality of the software.
- **Low flexibility and difficult refactoring** — due to the strong coupling of parts of the code and the lack of a well-thought-out structure, refactoring and adapting the system to changing needs is much more difficult and carries an increased risk of breaking existing functionality
- **High operating cost** — the drawbacks of the system presented above result in a high cost of maintaining and developing the system

Feature \ Architecture	Monolith	SOA	Microservices	Big Ball of Mud
Structure	A single, integrated application	A set of services that can be used and reused to support applications	Small, independent services, each focused on one business function	No clear structure, chaotic, strongly coupled and tangled code
Development	Development, testing and deployment are simple at a small project scale	More complex than a monolith, simpler than microservices	Complex, requires coordination between services	Difficult and risky, lack of modularity makes changes difficult
Scalability	Scales as a whole	Services can be scaled independently	Services can be scaled independently	Very low, the system is hard to split and scale

Feature \ Architecture	Monolith	SOA	Microservices	Big Ball of Mud
Deployment	Easy deployment, a single deployment artefact	Requires careful management of service versions and dependencies	Continuous deployment possible, requires orchestration tools	No clear deployment process, often chaotic
Technology stack	Uniform across the whole application	May vary between services, but rarely due to integration challenges	May vary significantly between services	No consistency, a mixture of technologies without control
Data management	Centralised database	Shared databases or separate databases for each service	A separate database for each service, promotes decentralisation of data	No clear boundaries, data often global or duplicated
Communication	Method calls within the application	Communication between services over the network (SOAP, REST)	Communication between services often via REST or message queuing systems	No well-maintained interfaces, strongly coupled and tangled code

Table 3: Comparison of architectures

The table was prepared on the basis of an external source in the form of an online article.¹⁰

4.2. Choice and design of the Passed Pawn architecture

The architecture of the Passed Pawn system can be characterised as lying somewhere between a typical SOA architecture and a monolithic architecture. The most accurate name for it is probably **Service-Based Architecture (SBA)**, due to its service-oriented character. What distinguishes it from SOA is the fact that it does not fulfil the goals and assumptions of SOA. The division into services does not in this case result from the need for or a plan of their reuse in different systems and applications. Nor is it intended to ensure their absolute autonomy and independent deployment. The division results from pragmatic strategic and tactical motives in the design of the system and from a proactive approach to potential challenges of its development, maintenance and support. The designed architecture **provides high flexibility in the development, maintenance and support** of the system at an early stage of its life, while strongly **minimising the various kinds of overhead in work, resources and performance** associated with SOA and microservices. In the current state of the project there is no need for strongly formal and rigid management of contracts between services, and the current division into services provides a number of advantages. With the current requirements of the project and the competences of the team, deploying the whole system as a single monolithic application could be more complicated and, in addition, less flexible.

More detailed analyses with implementation details, fitting specific types of architectures, e.g. security, deployment and application architecture, are presented in later sections of the thesis. Below, on the other hand, the division of the general architecture into the 14 most important parts/components of the project is presented.

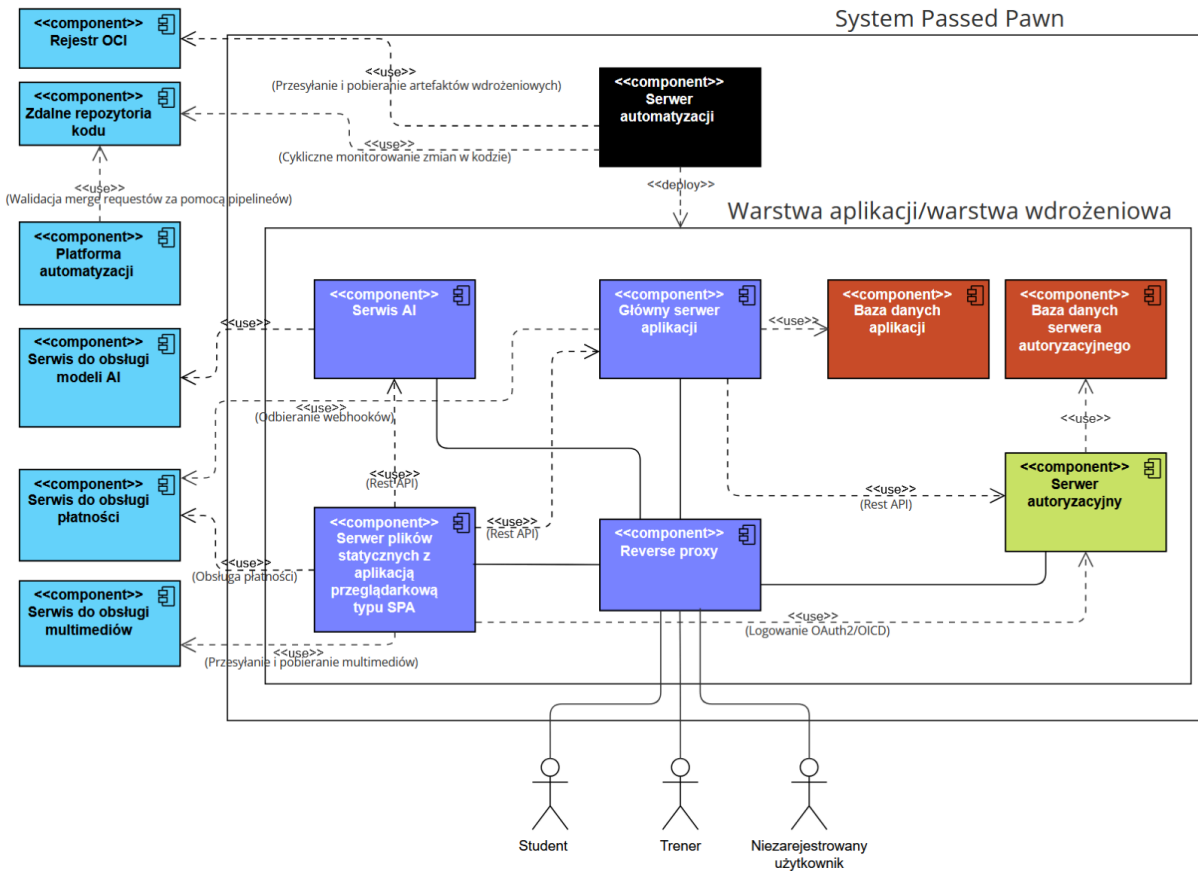


Figure 15: UML component diagram of the general architecture

1) SPA browser application (TypeScript, Angular, Nginx) The role of the browser application, which provides end users with a graphical interface for interacting with the system, is fulfilled by an SPA (Single Page Application) written in the Angular framework in TypeScript. The application is delivered to users visiting the website in the form of static files by an Nginx server. The main task of the application is to provide users with the best possible experience of interacting with the system.

Dependencies:

- **Main application server** — using the exposed REST programming interface to handle the business logic related to chess courses
- **AI service** — using the exposed REST programming interface to handle the business logic related to the AI assistant
- **Authorisation server** — handling OAuth2/OIDC login
- **Payment service** — handling payments in the application
- **Multimedia service** — handling the download and upload of multimedia

2) Main application server (REST API, C#, ASP.NET Core) The role of the main application server is fulfilled by a server designed in ASP.NET Core technology in C# with a REST programming interface. Its main tasks include handling all of the application's business logic related to cour-

ses, enforcing access control to application resources with respect to the user's role and identity (coach/student/anonymous user) and communicating with the database for data persistence.

Dependencies:

- **Payment service** — receiving webhooks containing the statuses of completed payments
- **Application database** — communication with the database via the PostgreSQL protocol for data persistence

3) AI service (REST API, Python, FastAPI) The role of the AI service is fulfilled by a server designed in FastAPI technology in Python with a REST programming interface based on HTTP + JSON communication. Its main tasks include handling the business logic related to the operation of the AI assistant in the application.

Dependencies:

- **Reverse proxy** — handling users' questions (not to be confused with requests) in the form of HTTP requests coming from end users' browsers
- **AI model service** — using the service's REST API to handle users' questions with selected AI models

4) Authorisation server (Keycloak) The role of the authorisation server is fulfilled by an internally managed Keycloak server. Its main task is to be a central point providing functionality for identity and access management in the system in accordance with modern security standards.

Dependencies:

- **Authorisation server database** — communication with the database via the PostgreSQL protocol for data persistence

5) Application database (PostgreSQL) The role of the application database is fulfilled by a PostgreSQL database, which provides persistence of the data needed to handle the application's business logic. Below is an ERD diagram of the database generated by the pgAdmin tool.

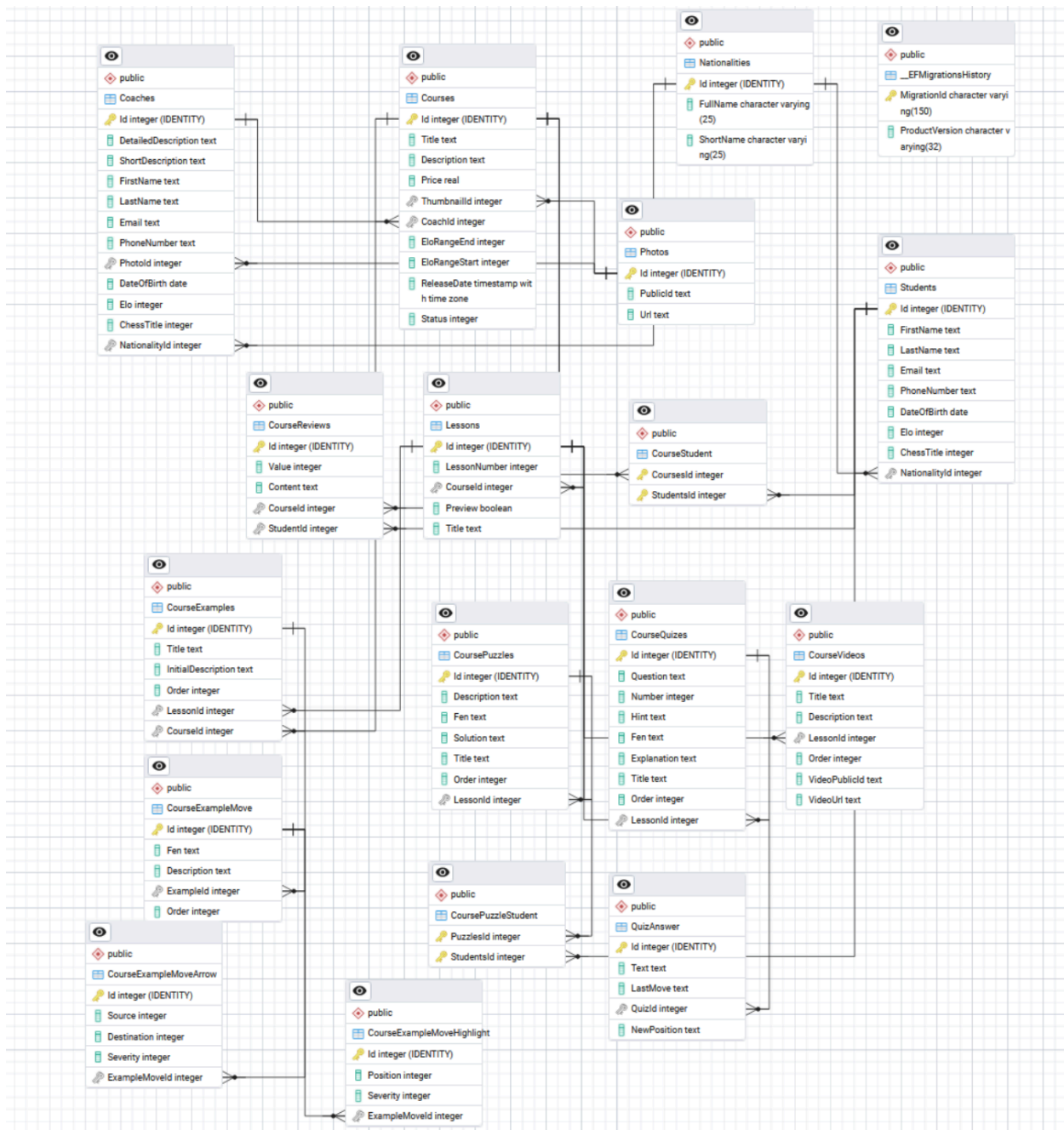


Figure 16: ERD diagram of the application database

6) Authorisation server database (PostgreSQL) The role of the authorisation server database is fulfilled by another PostgreSQL database, which provides persistence of the data needed to handle the functionality provided by the authorisation server. It contains, among other things, the data needed for user authorisation and authentication. The database schema is managed automatically by the authorisation server.

7) Reverse proxy (Nginx) The role of the reverse proxy is fulfilled by an Nginx server, which acts as an intermediary in communication with the application, forwarding users' requests to the appropriate

services of the application layer inside the system's private network. It is the only internally managed component of the system that is accessible from the Internet. Its main role is to act as a central access point to the system that adds security mechanisms (e.g. TLS/HTTPS) and performance optimisations to communication with the system.

Dependencies:

- **Static file server with the SPA browser application** — mediating users' communication with the SPA application via the HTTP protocol
- **Main application server** — mediating users' communication with the main application server via the HTTP protocol
- **Main application server** — mediating the sending of notifications to users using the SSE (Server-Sent Events) protocol
- **AI service** — mediating users' communication with the AI service via the HTTP protocol
- **Authorisation server** — mediating users' communication with the authorisation server via the HTTP protocol

8) Automation server (Jenkins) The role of the internally managed automation server is fulfilled by a Jenkins server deployed as part of the Jenkins Operator for Kubernetes. Its main task is to automate the processes of building, deploying and versioning the system's deployment artefacts.

Dependencies:

- **Code repositories (GitHub Repositories)** — periodically monitoring the code to detect changes and automatically build new deployment artefacts, push them to the registry and then deploy them
- **OCI registry (GitHub Package Registry)** — pushing the built deployment artefacts for storage in an OCI registry in the form of GitHub Package Registry

9) AI model service (MistralAI) The role of the AI model service is fulfilled by the MistralAI service, which exposes a REST API for querying various AI models.

10) Payment service (Stripe) The role of the payment service is fulfilled by the Stripe service, which provides functionality for carrying out payments in the web application.

11) Multimedia service (Cloudinary) The role of the multimedia service is fulfilled by the Cloudinary service, which provides infrastructure for storing, uploading and downloading multimedia.

12) Remote code repositories (GitHub Repositories) The role of the remote code repositories is fulfilled by GitHub Repositories. Apart from storing code, they provide a range of features that make it easier to manage code changes, strengthen the security of code management and allow the creation of rules, restrictions and enforced code management conventions.

- **Automation platform** — updating the main branch of the main application server repository is reserved for candidates (Merge Requests) that meet the validation requirements performed by automated tests on the GitHub Actions platform

13) Automation platform (GitHub Actions) The role of the externally managed automation platform is fulfilled by GitHub Actions, whose main task is to validate candidates for new versions of the system presented in so-called Merge Requests using automated tests.

14) OCI registry (GitHub Package Registry) The role of the OCI registry used to store and share deployment artefacts is fulfilled by GitHub Package Registry.

4.3. Single Page Application (SPA) vs Multi Page Application (MPA)

Two main architectural solutions were considered for building the client application: **Single Page Application (SPA)** and **Multi Page Application (MPA)**. MPA is a more traditional approach in which navigation between different views on the site takes place by rendering a new page every time the user wants to change the view. In an SPA everything is rendered on the same page, which eliminates the need for frequent reloading. This makes the user experience smoother and closer to a native application.

Both approaches have their uses depending on factors such as the complexity of the application or its interactivity. That is why it was very important to analyse the requirements before choosing the appropriate solution. The most important aspects that influenced the decision are presented below:¹¹

4.3.1. Speed of interaction

In an SPA there is no need to load a new page from the server on every view change (it is enough to fetch the necessary data), thanks to which transitions between views are much faster and smoother (no page reload effect). In the Passed Pawn application both the student and the coach have many views between which they must navigate frequently, which is why an SPA seems a natural choice.

4.3.2. SEO (Search Engine Optimization)

In a traditional SPA, search engine positioning is a big challenge, because the *crawlers* used by search engines work best when the HTML is immediately filled with the relevant data. In an SPA the data is available only after the JavaScript code has been loaded. MPA performs better here, but modern SPA frameworks offer two techniques that solve this problem:

- SSR (Server-Side Rendering): pages are rendered on the server after receiving a request
- SSG (Static Site Generation): rendering the page on the server when building the application

Unfortunately, these techniques can only be used for some pages. In the described application, SEO matters only for the home page (landing page), because the rest of the content is available only after logging in and purchasing a course. Therefore, positioning is not crucial.

4.3.3. Initial loading time

Bounce rate and conversion rate are the 2 most important metrics determining user interest. The bounce rate refers to the percentage of users who visited the site and left it without any interaction. The conversion rate, in turn, determines the number of users who performed a specific action (e.g. registration on the platform). Without SSR/SSG, an SPA can have a longer loading time and a greater

delay before the page becomes interactive (TTI – Time To Interactive). Reducing TTI from 5.5 s to 3.85 s can reduce the bounce rate by 10% and increase the conversion rate by 57%.¹² MPA performs better in this category, but an SPA can minimise this problem by using SSR/SSG on the most important pages.

4.3.4. Use of client resources

An SPA runs much more JavaScript code in the client's browser, which can cause a drop in performance and, consequently, a worse user experience. However, the described application does not use complicated animations or intensive computations, so this problem is not very significant.

4.3.5. Server load

An MPA has to fetch all the data from the server on every view change, which increases the server load. In an SPA, on the other hand, some data can be retained between views. This problem can become significant with a large number of users, but it is worth taking into account already at the planning stage.

4.3.6. Rich user experience

An SPA offers a richer, more native user experience than an MPA. It is easier to implement animations, interactive elements or real-time updates. This is very important for us, because the user interface (of both the student and the coach) is based on interactive chessboards, forms and other dynamic elements.

4.3.7. Efficient development

SPA frameworks (e.g. React, Angular, Vue) offer many tools supporting efficient development and code organisation. Thanks to component-based architecture (CBA), it is possible to create independent, reusable components. By applying good practices (e.g. the single responsibility principle — SRP), avoiding excessive dependencies and writing reusable components, the development process becomes simpler and more efficient.

Taking all of the above aspects into account, it was decided to use a **Single Page Application**. This choice significantly facilitated the implementation process and provided a better user experience.

4.4. Communication protocols used

4.4.1. SSE

SSE (Server-Sent Events) is a protocol based on listening for messages. Unlike similar alternatives such as WebSockets or MQTT, SSE is completely one-way. As the name suggests, the client listens for messages coming from the server.

In the Passed Pawn application such one-way communication has one important use. Because all payments are processed by an external service, Stripe, after a course is bought there is a slight delay before it is actually added to the student's list of courses in the database. For this reason, after buying

a course the client starts listening for SSE messages, and the server sends such a message when the course has been added to the list. Thanks to this, the user is able to see a notification.

This use case does not need two-way communication at all, which is why SSE turned out to be the ideal protocol for it.

4.4.2. HTTP, HTTP + TLS

HTTP is by far the most frequently used protocol in the Passed Pawn system. With the exception of the one example where SSE was used, all communication between the client and the server is based on the HTTP protocol. The main reason for choosing this protocol is adherence to REST conventions and strong support from the frameworks used by the system.

Apart from client-server communication, HTTP is also used exclusively for communication with external services such as Stripe or Cloudinary.

All communication sent from and to the application server uses HTTP + TLS (HTTPS) to ensure confidentiality, integrity and authentication.

The Passed Pawn application uses HTTP 2.0, because tests revealed that version 1.1 causes unexpected behaviour when using SSE. In addition, version 2.0 offers other advantages, such as faster processing of requests and responses.

Notes

6. prof. WSEI dr hab. inż. Jan Werewka. *Rola i znaczenie architektury w obszarze IT* (The role and importance of architecture in IT). <https://magazyn.wsei.edu.pl/rola-i-znaczenie-architektury-w-obszarze-it/>. Accessed: 2025-06-21. 2024.
7. Ibid.
8. Ibid.
9. Marcin Sobieraj. *Service Oriented Architecture - SOA*. <https://drogaarchitektait.pl/2020/08/service-oriented-architecture-soa/>. Accessed: 2025-06-21. 2020.
10. Pramitha Jayasooriya. *Monolithic vs. SOA vs. Microservices Architecture: A Java Perspective*. <https://medium.com/@lpramithamj/monolithic-vs-soa-vs-microservices-architecture-a-java-perspective-6d3d9fb26ac7>. Accessed: 2025-06-21. 2024.
11. Sehban Alam. *Single Page Applications (SPAs) vs. Multi-Page Applications (MPAs): Advantages and Challenges*. <https://medium.com/@sehban.alam/single-page-applications-spas-vs-multi-page-applications-mpas-advantages-and-challenges-df06bee3fed1/>. Accessed: 2025-06-17. 2024.
12. Edgemesh. *Why Time to Interactive (TTI) is the Most Consistent and Valuable Metric*. <https://edgemesh.com/blog/time-to-interactive-and-conversion-rate#heading-1>. Accessed: 2025-06-17. 2025.

5. SPA browser application

The client application is the interface for the user and the only way to interact with its functionality. For an educational platform, interaction with the user is a key element of the system, because without an appropriate presentation of the materials they have no value. When creating the client application, it was necessary to make sure that the experience was as good as possible both for the student and for the person creating the materials. It was also necessary to ensure an appropriate choice of technologies and solutions that would enable efficient development and good scalability of the solution.

5.1. Comparison of the most popular SPA frameworks/libraries – table

Table 4: Framework comparison: Angular, React and Vue

Criterion	Angular	React	Vue
Release date	2016 (Angular 2+)	2011	2014
Backed by	Google	Meta	Community
Type	Framework	Library	Framework
Approach	Opinionated (strongly imposed design assumptions)	Flexible (lets you decide on the organisation of the project and specific solutions yourself)	More flexible than Angular, but not as flexible as React
Routing	Built-in (Angular Router)	External (React Router, Next.js)	External, but officially supported (Vue Router)
Forms	Template-driven: a declarative approach using Angular directives; Reactive Forms: model-based, using FormControl and FormGroup	Manual implementation using useState/useRef, external dependencies (Formik, React Hook Form)	Manual implementation using v-model, external dependencies (vee-validate, vuelidate)
State management	None built in, services or NgRx, NGXS can be used	None built in, Redux, Zustand can be used	None built in, Vuex, Pinia can be used
Component model	Class-based, with an HTML template	Currently recommended: functional	SFC (Single File Components)
Testing support	Built-in: Karma, Jasmine	External: Jest, React Testing Library	External: Vitest, Vue Test Utils
Dependency injection	Built-in DI	No built-in DI	Built-in DI, but very limited

The table was prepared on the basis of an external source in the form of an online article.¹³

It compares the 3 most frequently used SPA frameworks.¹⁴ Basic information about their history and support, their general specifics and, in more detail, technological aspects are presented. The following sections describe in more detail the factors behind the choice of a specific framework.

5.2. Technology choices related to the user interface

Before starting the project, the team has to make an important decision about the choice of technologies that will enable efficient construction of the project's UI. It is an equally important stage as implementation, because its consequences will carry through the rest of the application's development. After careful analysis, the team decided on Angular as the frontend of our chess portal. The chosen framework is well suited to building large applications. Angular enforces a strong structure and architecture of the project, which in turn translates into clear rules for organising the code. The application is divided into components, which split the code into smaller logical units, which as a result makes the code clearer and easier to navigate.

5.2.1. Comparison of frameworks in terms of styling – table

Table 5: Comparison of component styling: Angular and React

Feature	Angular	React
Project structure	Styles per component (.component.scss/css)	Any: CSS Modules, inline styles, Tailwind
Scope (style scope)	Encapsulation by default (scoped styles)	No scoping by default — requires CSS Modules or CSS-in-JS
Styling methods	CSS / SCSS; Angular Material; Tailwind (optional)	CSS / SCSS; CSS Modules; Styled-components; Tailwind CSS; Emotion
Framework integration	Angular CLI handles styles automatically	Complete freedom, depends on the config/bundler
UI libraries	Angular Material, NG-ZORRO, PrimeNG	Material UI, Ant Design, Chakra UI, Tailwind UI
Dynamic styling	Harder (via classes/ngStyle)	Natural – styles can be set directly in JS
Philosophy	Separation of layers: logic, HTML, style	Component = logic + JSX + styles (holistic approach)

The table was prepared on the basis of an external source in the form of an online article.¹⁵

5.2.2. Approach to styling

The choice of a style language is key from the perspective of implementing the look of the user interface. There are 3 main technologies popular among developers: CSS, SCSS and Tailwind. Classic CSS is a style sheet language used to describe the presentation of elements in HTML documents, SCSS is defined as an extension of the CSS language, and Tailwind is a modern CSS framework based on styling with ready-made classes. In the case of the chess project, defining a large number of Tailwind style classes leads to mixing the HTML structure with many selectors, which as a result produces chaotic and unreadable code. Primitive CSS, on the other hand, represents a nested HTML structure in an unreadable way. The most suitable option turned out to be SCSS, which allows styles to be defined in a way that visualises the nesting of the given classes. The CSS extension allows variables to be defined conveniently, enabling styles to be unified across the whole application. Using variables reduces the risk of errors when writing style values from memory (e.g. colour, component

width) and allows values to be replaced efficiently throughout the application. SCSS is also equipped with a for loop, which automates writing repetitive classes, making it convenient to style the chess-board component.

Table 6: Comparison of frontend styling systems

Feature / Technology	CSS	SCSS (Sass)	Tailwind CSS
Syntax	Standard, simple	CSS extension with additional features	Utility classes used in HTML
Code organisation	Manual	Variables, nesting, mixins, functions	Everything through utility classes – zero CSS files
Abstraction / logic	No logic — everything written by hand	Yes – variables, conditions, loops	No logic – everything through ready-made classes
Ease of use	Simple for beginners	Requires learning the extensions	Requires knowledge of the classes and configuration
Performance	Depends on hand-written code	Optimised, but larger CSS	Very efficient – the classes used are concise and short
Reusability	Has to be written separately	High thanks to mixins and functions	High thanks to utility classes, but less flexible
Configuration	None	Minimal (preprocessor)	Requires a Tailwind config and a build tool (PostCSS, Webpack)

The table was prepared on the basis of an external source in the form of an online article.¹⁶

5.2.3. Style encapsulation

Angular supports style encapsulation – this refers to the way CSS styles are applied to components, making it possible to hide them from the outside world. The framework offers 3 encapsulation modes: the default Emulated – styles are limited to the component; None – no encapsulation; ShadowDom – the style works within the real ShadowDOM, a web technology that makes it possible to create isolated DOM trees inside HTML elements. The default encapsulation mode used in our project – Emulated – makes the written SCSS clear and easier to maintain. The chess application uses the default encapsulation mode, which reduces the risk of CSS class conflicts and accidental overriding of styles.

5.2.4. PrimeNG as the chosen UI library

PrimeNG is a UI library offering a wide selection of ready-made user interface components, dedicated to Angular. It makes it possible to choose a theme that suits the application. It significantly speeds up the development process, which makes it possible to focus on key features.¹⁷ As described earlier, the components for which a specific look had to be achieved were implemented by hand, but it was still decided to use ready-made solutions from the library to create, among other things, "toast" notifications, modal windows, some types of inputs and pagination.

5.3. Features of Angular and their impact on the implementation

Angular is a framework for building web applications created on the basis of AngularJS. It is developed by Google, and new versions are released about every 6 months. The described application uses version 19, i.e. the latest at the time of writing. Angular is based on TypeScript and makes it possible to build complex single-page applications (SPA). It is used for building medium and large web applications. It offers a component-based architecture. This section describes selected features of Angular and the solutions built into it that influenced the implementation of the browser application.

5.3.1. Strongly imposed design assumptions

Angular is an *opinionated framework*, which in practice means that its creators impose certain good practices, divisions, conventions and architectural solutions in advance. In large applications, in whose development many developers are involved, this makes it possible to avoid problems resulting from the individual approaches and decisions of single people. It also helps to avoid unnecessary debate about the superiority of a given solution and to focus on the essential aspects of how the application works.

5.3.2. TypeScript

TypeScript is a high-level, statically typed programming language used in Angular. It is a superset of JavaScript, to which it adds support for static typing and some object-oriented features such as decorators and interfaces. The whole Angular framework was designed with the use of TypeScript in mind. Compared to JavaScript, using this language increases the safety and readability of the code, makes developing large applications easier and provides good integration with Angular's object-oriented architecture.

5.3.3. Dependency Injection system

Angular offers an extensive, yet simple to use, Dependency Injection system. Using this pattern offers the following benefits:

- Loose coupling: classes (in this case components or directives) do not create their dependencies themselves, which allows them to be easily replaced.
- Easier testing: easy replacement of dependencies allows simple *mocking* for unit tests
- Increased code modularity: services can be used in many components or directives
- Improved code quality: easier refactoring and better readability

The Dependency Injection system in Angular offers extensive control over where a dependency is available. It achieves this thanks to several levels of Injectors (the abstraction responsible for providing dependencies). These include, among others, the Element Injector, which works locally for a component or directive and their descendants, and the Environment Root Injector, which contains globally available dependencies.¹⁸

React has no built-in DI system. The Context API can be used for this purpose, but it comes with many limitations.

5.3.4. Extensive support for forms

Angular offers 2 approaches to forms, each of which has different advantages. The table below presents a comparison of these 2 approaches:¹⁹

	Reactive	Template-driven
Form model	Defined in the component class	Defined in the template using directives
Approach	Programmatic	Declarative
Validation	Validation in the class using Validators	Validation in the template using directives
Use cases	Complex or large forms with dynamically changing fields or extensive validation	Simple static forms with little validation

Unlike Angular, which offers as many as two ways of creating forms, each with built-in validation, testing and monitoring of the state of a given field (dirty, touched, valid etc.), React and Vue do offer the possibility of implementing simple forms using built-in mechanisms such as `useState/useRef` in React or `v-model` in Vue, but this creates the need to use many external packages, which can be difficult to choose and to maintain as dependencies.

5.3.5. Attribute directives

Attribute directives are classes created using the `@Directive` decorator. They make it possible to act directly on elements of the DOM tree. Thanks to this, we can operate at a lower level of abstraction, which can be useful in many scenarios, e.g. when we want to dynamically add and remove elements from the tree or change their styles or other properties. There are also built-in directives, such as `ngClass`, which dynamically assigns CSS classes, `ngStyle`, which sets styles directly, and `ngModel`, which enables two-way data binding. Using custom attribute directives offers numerous advantages:

- Separation of Concerns: the logic responsible for directly manipulating the DOM tree is separated from the component logic
- Reusability: directives can very easily be assigned to many components or DOM elements. For example, in the Passed Pawn client application, directives were created for displaying multicoloured arrows and square highlights on the chessboard. They are used in several chessboards available in the application. It is enough for the chessboard to have a specific structure of elements for the directive to be usable.

```
<div class="chessboard"
  [appChessboardArrows]="arrows"
  [appChessboardHighlights]="highlights"
>
  @for (field of chessboardView; track $index) {
    <div class="chessboard-field">
      ...
    </div>
  }
</div>
```

- Easier testing: thanks to separating the directive's logic from the component, it is easier to write unit tests, because there is no need to create the whole component.
- A controlled and safe way of manipulating the DOM tree
- Clean and more readable HTML: directives allow a concise and semantic description of behaviour, which makes the code more readable and declarative

5.3.6. Advanced and expressive data binding

Data binding is a concept present in most modern web frameworks. It consists in connecting the user interface with data from the code. In the case of Angular, these are the template and the component class. In Angular there are four types of data binding, of which the first three are one-way and the last one two-way:²⁰

- *Interpolation* — used for simply displaying data in the view, for example `<h3>{{ course.title }}</h3>`
- *Property binding* — This kind of data binding can be used for 2 purposes. It is used to dynamically assign a value to a property of a DOM element: this can be, e.g., the source of an image ``. We can also use it to pass a value to a component — for this we have to declare an appropriate attribute with the `@Input` decorator in the component class, e.g. `@Input() public rating: number = 0;`. Now we can pass a value to the component: `<app-star-rating [rating]="givenRating" />`. This kind of data binding can also be used on directives.
- *Event binding* — used to pass data in the opposite direction to property binding, i.e. to react to DOM events or events emitted from components. We can also use it in two ways, on a DOM element: `<button (click)="onDelete($event)">` and on a component: `<app-coach-course-tile (delete)="deleteCourse($event)" />`. In order to use event binding on a component, we have to declare an attribute with the `@Output` decorator and assign an instance of the `EventEmitter` class to it: `@Output() public delete = new EventEmitter<string>();`. Then, to emit an event, it is enough to call the `emit` method: `this.delete.emit(this.course.id.toString());`
- *Two-way binding* — this makes it possible to combine event binding and property binding by using two kinds of brackets: `<input [(ngModel)]="currentSeverity"/>`. It is most often used with `ngModel`, which provides a ready-made implementation. This mechanism is the basis of Template-driven Forms.

The data binding syntax in Angular is very declarative and makes the code easier to read. Seeing the appropriate brackets in the template, we immediately know what kind of binding we are dealing with and in which direction the data is sent.

5.3.7. Support for reactive programming – RxJS

RxJS (Reactive Extensions for JavaScript) is a library that offers support for reactive programming in JavaScript. The most important element of RxJS is the `Observable`, which represents a collection of future values or events. It also offers operators such as `map`, `filter`, `reduce` and others, which make it possible to handle asynchronous events in a similar way to classic JavaScript collections. All of this enables an alternative and modern approach to asynchronous programming, replacing JavaScript's built-in `Promise` mechanism.²¹

5.3.8. Change detection

A change detection mechanism is needed so that when some data changes in a component (for example, an element is added to an array, a numeric value changes, etc.), these changes are reflected in the component's view. Angular offers an advanced change detection system that is effective and efficient. This system works automatically, but understanding how it works can be useful for avoiding errors and making use of its full potential.

Angular intercepts various asynchronous events in the browser and triggers the change detection mechanism on each of them. These can be user interactions, such as clicks or inputs, and asynchronous operations, such as `setTimeout()`. The mechanism of adding change detection calls to the default implementations is provided by the Zone.js library. Change detection can also be triggered programmatically in 2 ways: checking for changes in the current component by calling the `ChangeDetectorRef.detectChanges()` method, or triggering the whole change detection cycle with the `ApplicationRef.tick()` method. After change detection is triggered, the `ChangeDetector`, which is instantiated in every component when it is created, checks all the data that has been used in the template. A change is detected if the current value differs from the previous one.

It is also possible to change the way change detection works within the scope of a component. We can set the detection strategy to "OnPush" by setting the following property in the `@Component` decorator: `changeDetection: ChangeDetectionStrategy.OnPush`. As a result, detection will be triggered automatically only in 3 cases:

- When the reference of the data passed via `@Input` changes
- When an `EventEmitter` emits an event
- If we use the *async pipe* in the template and the `Observable` emits a new value

We also have the option of disabling automatic change detection completely. Such a need may arise when it is of great importance for performance. It is then enough to call the `ChangeDetectorRef.detach()` method, after which changes can only be detected programmatically.

The change detection mechanism offered by Angular is efficient, effective and reliable. The framework also allows quite extensive control over how it works.²²

5.3.9. Built-in HTTP client

Communication with the server application is a key function of the Passed Pawn client application. A significant part of this communication takes place over the HTTP protocol, which is why its handling and the related functionality are an important part of the chosen framework. Angular has a built-in HTTP client in the form of the `HttpClient` singleton. It supports all available HTTP methods. To use the client, it is enough to call the appropriate method, e.g.: `return this.httpClient.get<{lessonCount: number}>('/api/Course/...')`.

The client offers many useful features that make working with this protocol in large projects much easier:

- Access via Dependency Injection — `HttpClient` can be made available as a singleton throughout the application thanks to the `provideHttpClient()` method. Thanks to this, we can use it in many services or directly in components.
- Integration with RxJS — The client's methods return an `Observable`. Thanks to this, the request is sent only when we subscribe to the returned value (calling the `subscribe` method). We also gain access to all the useful RxJS operators discussed earlier.
- Automatic JSON serialisation/deserialisation — the HTTP client automatically converts our JavaScript objects to JSON before sending a request, and the JSON received in the response to a JavaScript object. This significantly streamlines the process of communicating with the server application and makes the code more readable.
- Interceptors — offer a very useful mechanism for intercepting requests and responses. An interceptor is called before a request is sent or a response is delivered to its recipient. In it, one can, for example, modify the request or response, handle errors or add headers. In *Passed Pawn* an interceptor was created that adds a header with a JWT token to every request for authorisation on the server. It is possible to register many interceptors, creating a "chain" in which they are executed in the order of their declaration.

Angular's HTTP client is a useful, proven and complete solution. Other frameworks such as Vue or React force the user to rely on JavaScript's built-in `fetch` tool or to use the external `axios` dependency. However, these solutions are much poorer in functionality, which is why developers have to implement it themselves, which reduces efficiency and means that the code can differ significantly from project to project.²³

5.4. Technology choices related to access control

All security mechanisms are implemented on the server, but in the client application we need a mechanism that integrates with the identity provider and ensures that requests contain the appropriate tokens needed for authentication. This section describes the chosen solutions and the access control mechanisms available in Angular.

5.4.1. KeycloakJS, Keycloak-Angular

Keycloak JS is a JavaScript library that makes it possible to secure web applications by acting as an adapter to Keycloak. It uses the OpenID Connect protocol to connect to the Keycloak instance on the server.²⁴ `Keycloak-Angular` is a "wrapper" around Keycloak JS that makes the library easier to use by providing various abstractions and tools. It allows easy configuration and creation of an instance of the keycloak service. This service can then be used as a singleton throughout the application, thanks to support for Angular's Dependency Injection.²⁵ It is used, among other places, in the `AuthService`, where methods are needed for logging the user in and out and for extracting the username or the token. It is also essential in the guards, where the role of the currently logged-in user is checked.

5.4.2. Guards

All endpoints are properly secured on the server, but from the user's point of view we still cannot allow a situation in which they end up on a page intended for a different role. This is where *guards* come to our aid — a mechanism offered by the Angular Router. When implementing a guard, there are 2 options to choose from — a functional or a class-based guard. In Passed Pawn functions were used because of their conciseness and simplicity, but both approaches are widely used. The CanActivate guard (i.e. CanActivateFn in the functional version) was used to secure the page. It checks the currently stored JWT token, which is taken from the Keycloak instance. From this token we can extract information about the role of the given user and, consequently, decide whether they may have access to the given page. If the user has access, it is enough to return `true`; otherwise we return `false`. To use guards, it is enough to pass an array to the object describing the given route, e.g. `canActivate: [loggedInGuard, studentGuard]`.²⁶

Angular offers a wide range of other guards:²⁷

- CanActivateChild — similar to CanActivate, but also protects child routes
- CanDeactivate — protects against leaving a component. Useful, for example, to protect the user from accidentally losing unsaved data in a form.
- Resolve — used to start fetching data before navigation begins
- CanLoad — prevents the code of a module to which the user has no permissions from even being loaded
- CanMatch — a more modern alternative to CanLoad, more flexible and offering more control

Guards are an extremely helpful mechanism offered by the Angular Router, because they allow access control, optimisation and an improved user experience in a modular, simple and declarative way.

5.5. Implementation structure

The implementation of the client application was divided into 5 modules. The 2 most important modules, independent of each other, were separated: *student* and *coach*. Both of them contain their own components, models, resolvers, services etc. Each also defines its own routing. Thanks to this, it was possible to use *lazy loading* and to secure whole modules in one place.

```
{
  path: 'student',
  canActivate: [loggedInGuard, studentGuard],
  loadChildren: () =>
    import('./modules/student/student.routes').then((mod) => mod.STUDENT_ROUTES)
},
{
  path: 'coach',
  canActivate: [loggedInGuard, coachGuard],
  loadChildren: () =>
    import('./modules/coach/coach.routes').then((mod) => mod.COACH_ROUTES)
},
```

Dividing the essential part of the code into these 2 modules definitely made development easier at the point when many elements appeared that served a similar function, but for different types of users. For example, both the coach and the student need a course view, but these views are completely different. Adopting a naming convention in which the name of each component is prefixed with the name of the module (e.g. StudentCourseComponent) significantly improved navigation and helped avoid mistakes.

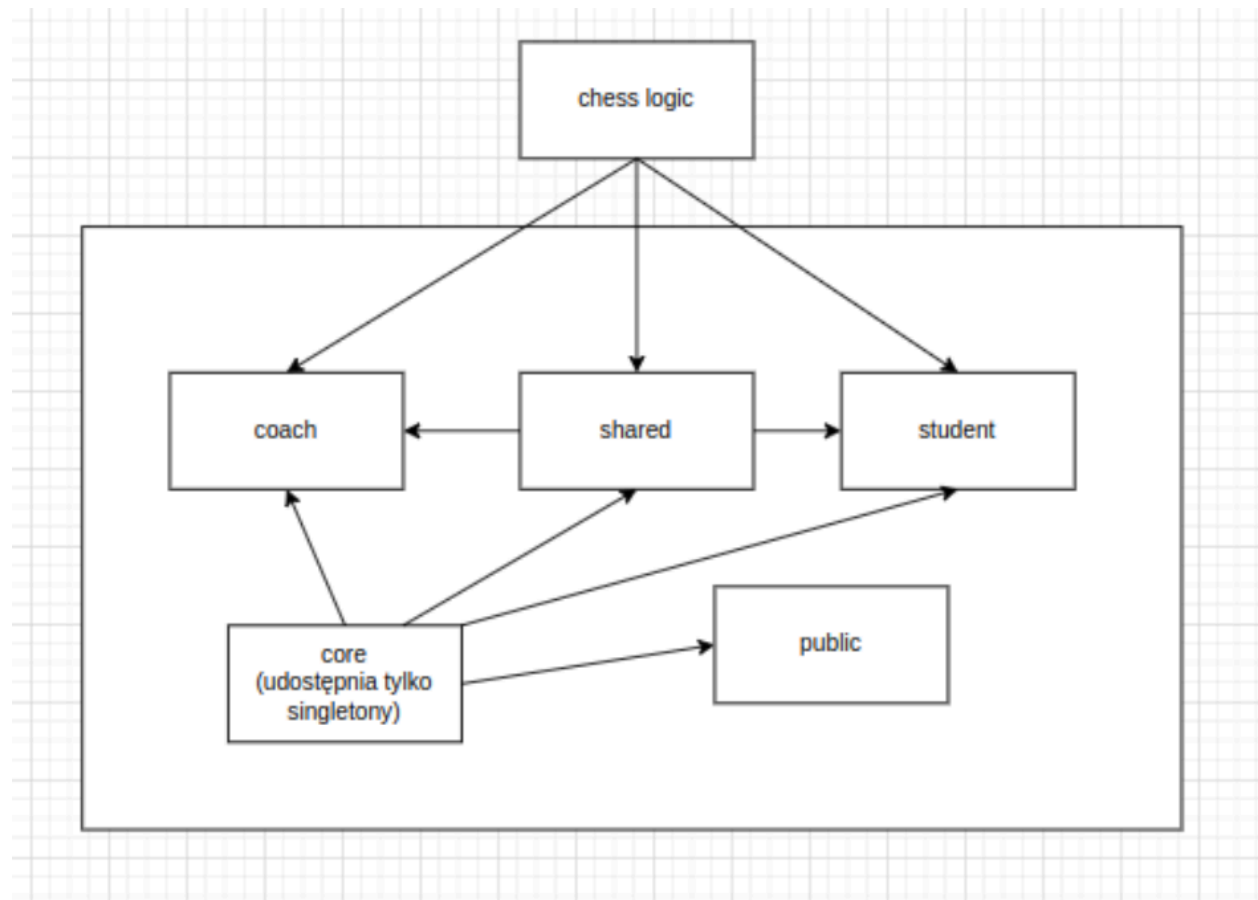


Figure 17: Diagram of the client application modules

Apart from the "main" modules, the following 3 modules were also separated:

- *shared* — this module contains all the code used by both the *coach* and the *student* module. It is important to be sure that the logic defined here will always be the same regardless of the user type. It contains, among other things, directives for displaying arrows and highlights on the chessboard, components and services related to the chatbot, and other minor features such as displaying the course difficulty level.
- *core* — this module is responsible for code that is not shared by the *coach* and *student* modules, but is an integral part of the whole implementation. It contains *guards*, *interceptors* and services that must always be *singletons* (in the case of the described application this is *authService*).
- *public* — all the code related to the anonymous user was placed here, i.e. the components of the *landing page* and the *not found* page, which is displayed when the user enters an invalid URL.

Apart from the main source code of the application divided into modules, one more part was separated, independent of Angular and written in plain TypeScript. This is the chess logic, defining all the necessary rules of chess such as legal moves, the logic of check, checkmate, determining the result of the game etc. Separating this part from the rest of the code introduces a clear division of responsibilities and makes navigating the project easier.

The simplified structure of the modules and the relationships between them are shown in the diagram. An arrow denotes a dependency of one module on another — the module it points to depends on the module it comes from.

5.6. Chess logic

The logic implementing the rules of chess and displaying the chessboard was separated from the rest of the application in order not to introduce unnecessary coupling and to adhere to the principle of separation of concerns. The code is based on a publicly available YouTube tutorial with an accompanying GitHub repository.²⁸²⁹ The original code was extensively changed and reworked for the needs of the described platform. New features and mechanisms were added, and the existing data structures and classes, together with their methods and attributes, were also modified. Attempts were made to contact the author of the code, but unfortunately they failed. The channels of communication with the author indicated in the description of the YouTube video are inactive or do not exist. The GitHub repository contains a question about how the code is licensed, but at the time of writing the thesis and designing the application it also remains unaddressed.³⁰ In view of the lack of any possibility of contacting the author of the code used, the lack of information about how the code is licensed, and the fact that the code was shared as part of a tutorial, it was assumed that it is intended for general and unrestricted use. Accordingly, in a modified and extended form, it was used to serve the purpose not of playing chess, but of supporting chess elements in courses. Such a solution gives full control over extending, modifying and implementing the code used to meet the functional requirements of the platform. This is a great advantage compared to using external libraries, none of which was found in a form that could satisfy all the functional needs of the platform, and which would involve much more difficult extension and modification. They would also introduce unnecessary, unused functionality that would worsen the application's performance. The chosen solution therefore introduces more initial complexity and effort, but allows better performance optimisation as well as greater scalability, potential for extension and overall control over the solution.

Notes

13. BrowserStack. *Angular vs React vs Vue: Core Differences*.
<https://www.browserstack.com/guide/angular-vs-react-vs-vue>. Accessed: 2025-06-18. 2025.
14. Stack Overflow. *Tag Trends*.
<https://trends.stackoverflow.co/tags=reactjs%2Cvue.js%2Cangular%2Csvelte%2Cangularjs%2Cvuejs3>. Accessed: 2025-06-19. 2025.
15. Gara Mohamed. *Angular vs. React (Part 1): style abstraction and encapsulation*.
<https://medium.com/@gara.mohamed/angular-vs-react-part-1-style-abstraction-and-encapsulation-f55b336e0eaa>. Accessed: 2025-06-19. 2018.

16. Sotiris Kourouklis. *SASS, CSS, or Tailwind: Which One Should You Choose?*
<https://dev.to/sotergreco/sass-css-or-tailwind-which-one-should-you-choose-3c0o>.
Accessed: 2025-06-19. 2025.
17. PrimeNG. *PrimeNG Documentation*. <https://primeng.org/>. Accessed: 2025-06-17. 2025.
18. Miłosz Rutkowski. *Wszystko co musisz wiedzieć o Dependency Injection w Angularze* (Everything you need to know about Dependency Injection in Angular). <https://angular.love/pl/wszystko-co-musisz-wiedziec-o-dependency-injection-w-angularze>. Accessed: 2025-06-18. 2024.
19. Angular. *Forms in Angular*. <https://angular.dev/guide/forms>. Accessed: 2025-06-18. 2025.
20. Angular Minds. *Data Binding In Angular: Everything You Need to Know*.
<https://www.angularminds.com/blog/data-binding-in-angular>. Accessed: 2025-06-18. 2025.
21. Angular. *The RxJS library*. <https://v17.angular.io/guide/rx-library>. Accessed: 2025-06-17. 2023; RxJS. *RxJS Overview*. <https://rxjs.dev/guide/overview>. Accessed: 2025-06-17. 2025.
22. Angular University. *Angular Change Detection - How Does It Really Work?* <https://blog.angular-university.io/how-does-angular-2-change-detection-really-work/>. Accessed: 2025-06-19. 2025.
23. Angular. *Understanding communicating with backend services using HTTP*.
<https://angular.dev/guide/http>. Accessed: 2025-06-19. 2025.
24. Keycloak JS. *Keycloak JS npm package page*. <https://www.npmjs.com/package/keycloak-js>.
Accessed: 2025-06-17. 2025.
25. Mauricio Vigolo. *Keycloak Angular npm package page*. <https://www.npmjs.com/package/keycloak-angular>. Accessed: 2025-06-17. 2025.
26. Angular. *CanActivateFn Documentation*. <https://angular.dev/api/router/CanActivateFn>.
Accessed: 2025-06-19. 2025.
27. Raghuvardhan Karanam. *Route Guards in Angular*.
<https://raghuvardhankaranam.medium.com/route-guards-in-angular-c2c01fe6167b>. Accessed:
2025-06-19. 2023.
28. Roberts Tech. *Code a Chess Game with Stockfish API – JavaScript Tutorial*.
<https://youtu.be/fJIsqZmQVZQ?si=6JttY01vB8CtVCzJ>. Accessed: 2025-06-30. 2024.
29. Roberts Tech. *Code a Chess Game with Stockfish API – JavaScript Tutorial*.
<https://github.com/awesomeCStutorials/chess-game>. Accessed: 2025-06-20. 2024.
30. davehorner. *license?* <https://github.com/awesomeCStutorials/chess-game/issues/4>. Accessed:
2025-06-20. 2025.

6. Main server application

6.1. Purpose and role of the server application

The server application is one of the more important elements of the architecture of the whole system, acting mainly as an intermediary between the frontend and the database. Its main purpose is to implement the business logic, handle requests coming from the client application and manage the data stored in the system. The application always makes sure that an operation is performed only and exclusively by an authorised user.

6.2. ASP.NET Core — the reason for choosing the technology

ASP.NET Core is part of the .NET platform managed by Microsoft and focuses on building web applications. This framework is cross-platform and open source.

This technology was chosen mainly because of the experience of many team members, but a big factor was also its rich ecosystem, which ultimately turned out to be a big advantage over similar alternative technologies such as Spring Boot.

The team is satisfied with this decision, as the application was built quickly and is easy to manage, largely thanks to the numerous packages installed from NuGet (the .NET package manager).

6.3. Application architecture

6.3.1. Design patterns

The application uses many design patterns to make management as simple as possible:

- **Dependency Injection** — The application consists of many services and repositories, and almost all of them are added to the DI container. This allows these objects to be injected automatically into the classes that need them. Its automatic nature makes managing the code easier, and the dependency injection mechanism itself makes testing easier thanks to the use of mocks.
- **MVC (Model–View–Controller)** — This is one of the more frequently used design patterns in ASP.NET Core applications. In the context of an API, the model represents the data and business logic, the controller handles HTTP requests, and the view is the data in JSON format returned to the user.
- **DTO (Data Transfer Object)** — a design pattern used to transfer data between application layers, e.g. between a controller and the business logic layer. Data returned to the user is also in DTO format, because this makes it possible to control what information is exposed externally.

6.3.2. N-tier architecture

N-tier architecture is an approach to application design that consists in logically dividing the system into separate parts with specific responsibilities, such as handling requests, business logic or data access.

This architecture was used in Passed Pawn because of its multi-layered approach. It makes managing the code easier, especially when working in a team.

6.4. Application layers

6.4.1. API

The layer responsible for receiving HTTP requests and returning responses. It contains controllers (in accordance with the MVC pattern), various middleware, and the entry file, which adds services to the DI container, configures authorisation and other middleware, and starts the application.

6.4.2. Business Logic

Many controllers do not perform many operations, so the logic is not moved anywhere. However, for more complicated methods, the logic is placed in services inside this project. This guarantees that the controllers are not heavily overloaded, which improves their readability.

6.4.3. Data Access

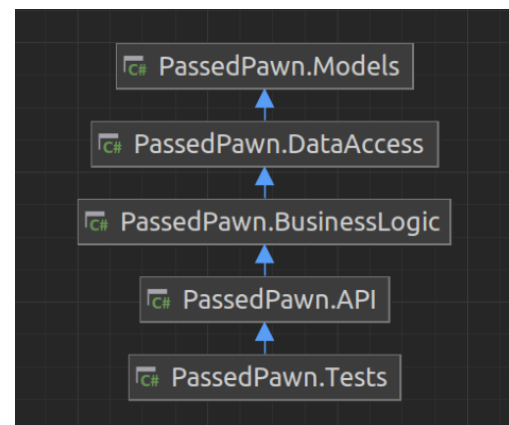
This project contains the so-called entities, i.e. the models from the MVC pattern. In addition, the project also contains the ApplicationDbContext, i.e. the class that represents the data and entities inside the database, and repositories, i.e. services responsible exclusively for modifying and retrieving data from the database.

6.4.4. Models

The project contains various classes that hold data. Most of the models used in the application are DTOs. The project also contains validators, which guarantee that the client does not send data that the API is unable to process.

6.4.5. Layer diagram

The diagram presents the layered structure of the application. The arrows illustrate the direction of access — which layer may use which. For example, the BusinessLogic layer has no access to the classes in the API layer, whereas the API may use the functionality of the BusinessLogic layer. This arrangement keeps the application orderly and readable.



6.5. API interface

6.5.1. REST

The API was designed on the basis of the REST (Representational State Transfer) architectural style. REST defines a set of rules that ensure consistent and predictable communication between client and server. Thanks to this, responses are easy for the client to interpret, and the whole API structure becomes more intuitive.

REST relies on the convention of HTTP response codes:

2xx (e.g. **200 OK**, **201 Created**) — the operation was performed successfully,

4xx (e.g. **400 Bad Request**, **404 Not Found**, **403 Forbidden**) — an error on the client side, for example data sent that the server is unable to process,

5xx (e.g. **500 Internal Server Error**) — an error on the server side, e.g. an exception in the server application or a temporary failure of an external service.

Thanks to the unambiguous use of these codes, the client can easily determine whether a request ended in success or encountered a problem, and what kind of problem it was. In addition, REST provides for an intuitive URL structure. For example:

GET	<i>/api/course</i>	<i>// Get all courses</i>
GET	<i>/api/course/1</i>	<i>// Get the course with ID = 1</i>
POST	<i>/api/course</i>	<i>// Add a new course</i>
PUT	<i>/api/course/1</i>	<i>// Update the course with ID = 1</i>
DELETE	<i>/api/course/1</i>	<i>// Delete the course with ID = 1</i>

Table 7: Example REST requests

Such a structure is intuitive and makes it easier for clients to use the API.

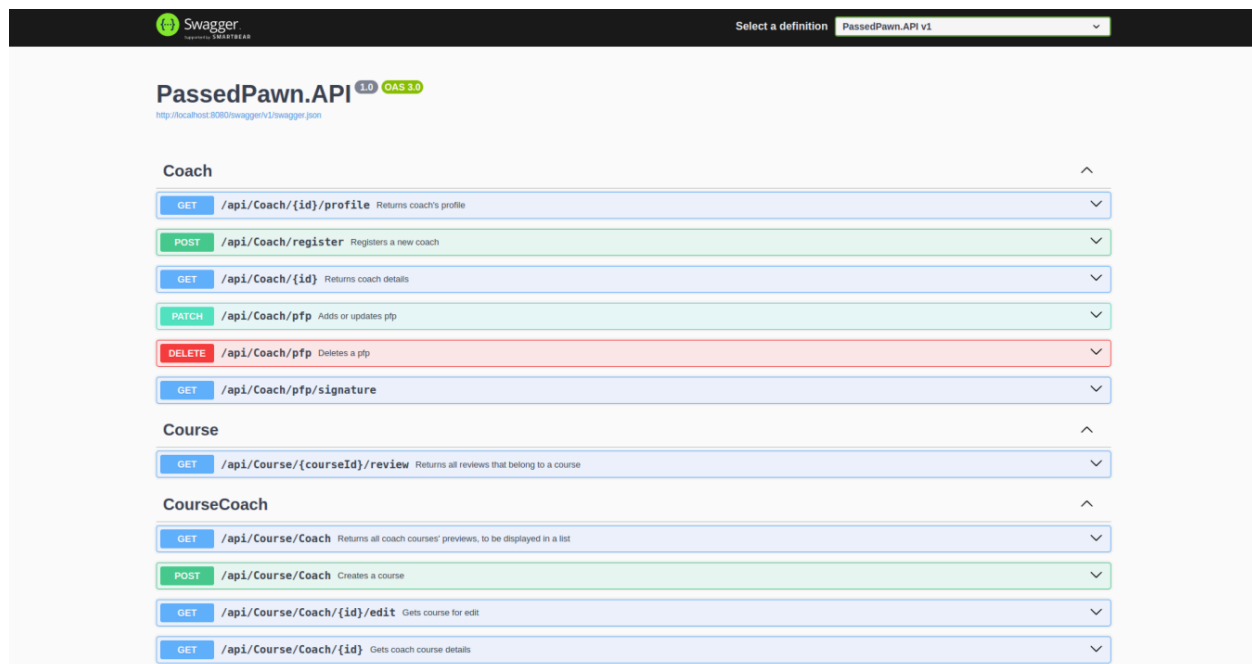
REST also assumes that the API should be stateless, which means that each request must contain all the information needed to process it. In other words, the server should not store any context between successive requests, which makes scaling easier and simplifies the architecture.

Although REST does not impose a specific data format, in practice, as in Passed Pawn, the most frequently used one is JSON (JavaScript Object Notation), due to its readability and lightness. Because all data sent from and to the server is in JSON format, communication is always intuitive.

6.5.2. Swagger

The documentation of a web application, especially of a REST API, should clearly present the available resources and their functionality. This makes it possible to control the development of the project and facilitates communication in the team, which is key to efficient software development. Swagger comes to the rescue — a set of tools and a specification for designing, describing, documenting and testing REST APIs. It is currently part of the OpenAPI ecosystem. Swagger is a popular tool that makes it possible to generate concise and consistent documentation (JSON). The technology also has

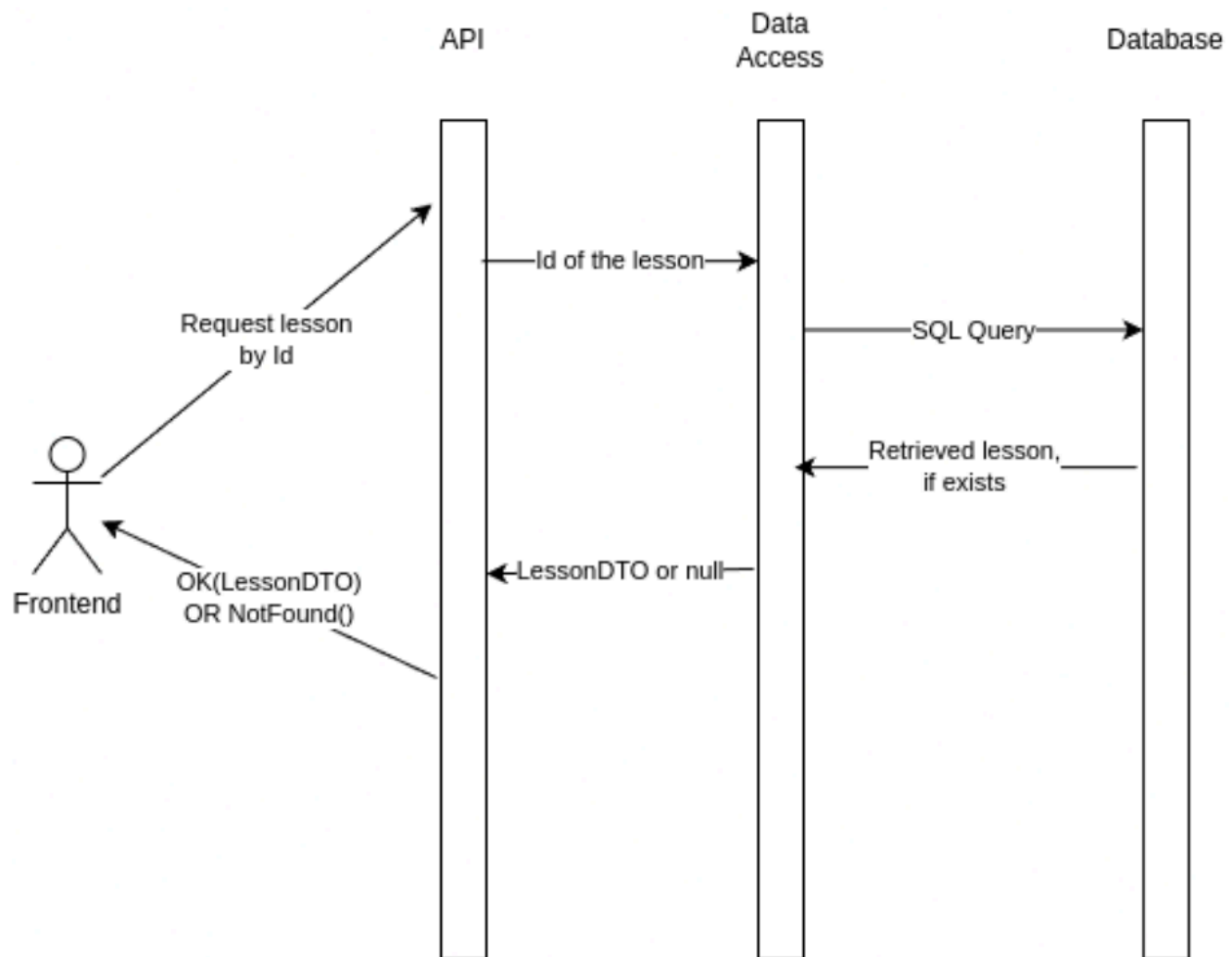
options for visualising the documentation by generating a UI (a web page) for the API resources. The OpenAPI solution enables collaboration at the technical level between the frontend and backend development teams, which have to integrate the software being built into a working whole. Swagger made it possible to define a specification of the API resources (endpoints and data models), which was the main point of reference for the teams responsible for the client and server layers, which eliminated discrepancies between the integrated projects. OpenAPI solved the problem of a lack of consistency between the layers of the chess application. The API documentation of the chess portal includes descriptions of the individual paths (endpoints) together with a defined list of returned HTTP response codes. Swagger takes into account the types of returned objects (schemas) used by the server.



6.6. Examples of operations

6.6.1. Fetching a lesson by ID

The UML diagram presents the operation performed when the frontend sends a request for a course with a specific ID. It is one of the more trivial operations; the diagram shows that only two layers of the application are used.



6.7. Error handling and validation

6.7.1. Global exception handler

Unexpected exceptions should not harm the whole application. For this reason, a middleware called `GlobalExceptionHandler` is used. It catches exceptions and returns information to the user that the application is temporarily unavailable. At the same time, the details of the exception are logged, which allows developers to analyse and fix the error.

6.7.2. Validation

All data sent by the client is validated to ensure that all the data can be processed by the application. ASP.NET Core provides many ready-made validators, but several validators specific to the Passed Pawn application were written.

6.8. Communication with the database — Entity Framework Core

Entity Framework Core is a modern, efficient ORM (Object-Relational Mapper) for the .NET platform. It makes it possible to map C# class objects to relational tables in the database, which greatly

simplifies accessing and modifying data, as using this tool completely eliminates the need to write SQL queries by hand.

In the project, EF Core served as the database access layer. Thanks to it, the following were possible:

- Automatic creation of the database structure based on model classes, requiring minimal configuration (Code First),
- Handling of relationships between entities (e.g. one-to-many, many-to-many),
- Executing LINQ queries against the database,
- Migrations that allow the database structure to evolve without data loss,
- Asynchronous, non-blocking database queries.

Another advantage of EF Core is its support for many database providers — in this case PostgreSQL was used, but one can easily switch to, e.g., SQL Server, SQLite or MySQL with minimal code changes.

7. AI module as a tool for improving UX

7.1. Purpose of introducing the module

When implementing the client application, the team paid great attention to designing a simple and intuitive user interface. However, although from the designer's perspective the interface is not complicated, some users may have trouble finding a given feature, or may not be aware of what functionality the system offers. In order to minimise this problem, it was decided to introduce an AI assistant in the form of a chatbot. It is available from every view in the corner of the window. The user can send it a question at any moment, and it, taking on the role of a friendly elephant, will answer according to its own knowledge of the application. Thanks to this solution, the user will always have quick access to information about the interface at their fingertips. The chatbot also opens up new possibilities for unregistered users who are unfamiliar with the purpose and functionality of the platform. If the information available on the landing page turns out to be insufficient for them, they can ask the assistant any question on this subject.

7.2. RAG system – a solution that makes it possible to use an LLM

An LLM (Large Language Model) is an advanced artificial intelligence model trained on huge data sets. Its aim is to understand and generate text in a way that is as close as possible to natural language.

A big drawback of LLMs is that these models learn from huge amounts of data, and once trained it is not easy to extend their knowledge. Although this is technically possible thanks to a process called fine-tuning, this process is expensive in terms of computing resources and time — in many cases as expensive as training the model from scratch, which is unrealistic for most people or businesses. That is why people using LLMs often have to add context to their queries, which is tedious and often insufficient. These problems are solved by the RAG (Retrieval-Augmented Generation) mechanism.

RAG is based on the idea that the LLM has access to its own private database. The database must be a vector database, because this enables very fast searching for related texts, mainly on the basis of sentence semantics.

With every query to the RAG system, the question is first redirected to the vector database. The vectors (i.e. the semantics of the sentences) of the query and of the stored data are compared, and the closest data is retrieved from the database. Then the query and the extracted data are passed together to the LLM, which, looking at the data, answers the user's question.

The process is intuitive, efficient and "invisible" to the person asking, since from the user's perspective the model simply answers the question. A big advantage is also the fact that the model does not have to be retrained every time in order to add new information; it is enough to store the relevant data in the database.

Another advantage is data privacy. The database can be hosted locally and is not shared anywhere.

Also important is the significantly reduced risk of hallucinations. RAG makes it possible to verify every answer. The LLM can be taught that if the answer to a question is not contained in the documents from the database, it should simply answer that it does not know. This gives much greater certainty that the user will not be misinformed.

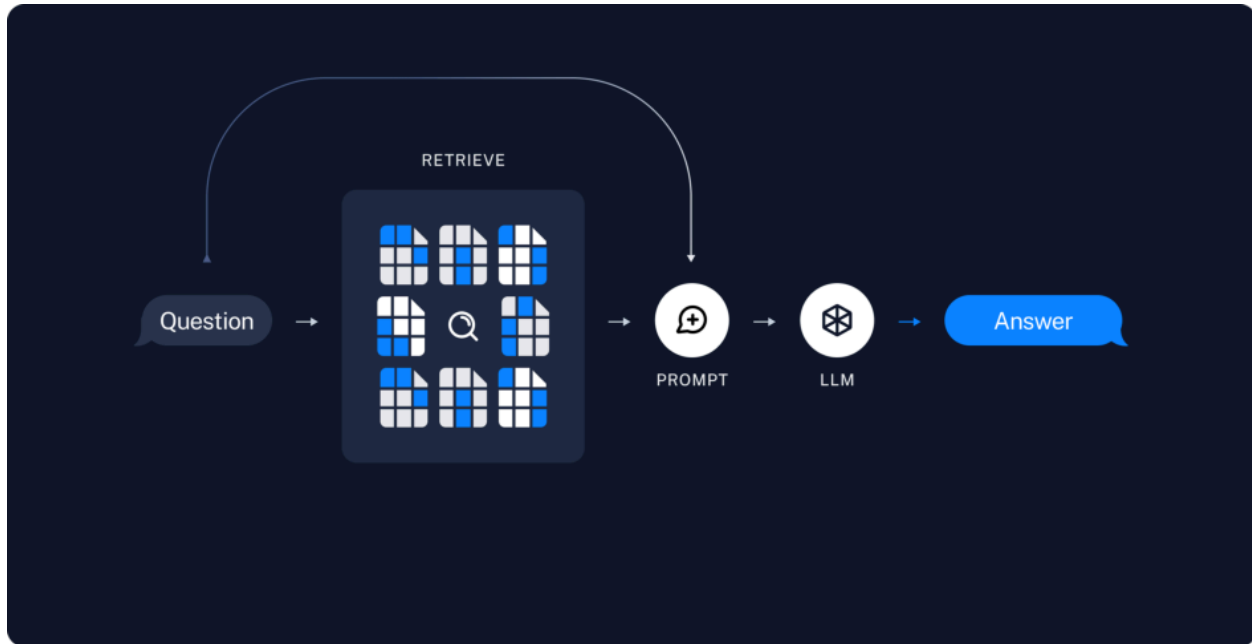


Figure 18: Diagram of how RAG works, published by LangChain

The diagram comes from the Python LangChain documentation.³¹

The database used by the RAG system of the Passed Pawn application contains knowledge about the site, mainly use cases, but also the appearance of the user interface. A user asking a question, for example "How can I register as a student?", receives an answer from the LLM that contains all the instructions on how the user can register. That is why it is important that the database contains as much information as possible and that it is always up to date.

7.3. Implementation of the AI server application

7.3.1. Python

A high-level, general-purpose programming language. Its syntax is characterised by simplicity and clarity. In Passed Pawn it was used to implement the chatbot because of the wide range of available libraries and tools related to artificial intelligence.

7.3.2. Using LangChain to create a RAG chain

LangChain is a framework for building applications based on LLMs. It enables the integration of LLMs with other technologies, such as vector databases or embedding models. It also makes it possible to use the APIs of many LLMs, e.g. GPT-4o, Gemini or MistralAI, which is used in Passed Pawn. This allows the RAG system to be implemented in the form of a RAG chain, i.e. a connected system

of steps. The best way to illustrate the steps performed within the "chain" is a code example in which the RAG chain of the presented implementation is defined:

```
rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough()
    }
    | RunnableLambda(build_prompt)
    | llm
    | StrOutputParser()
)
```

It is worth noting that the defined RAG chain consists of 4 stages, separated by "|" characters. The data flows through each stage in turn, in the order of definition. At the beginning, the question from the client application's user enters the "chain" in the form of a string. At the output, we receive the answer to the question, also as a string, which we send back to the client. This is how the data is processed step by step:

- Stage 1: The main role of this stage is to find the relevant documents in our vector database using the *retriever*. It finds the documents that best match the question semantically. Then the *format_docs* function joins them into a single string, thus creating the *context*. The *question* is simply the question we received at the input, passed on unchanged by *RunnablePassthrough*.
- Stage 2: This stage is responsible for transforming the object obtained from the previous stage into a ready prompt for the model. The *build_prompt* function combines special instructions concerning the "personality of the elephant" that our model is to play with the context and question created in the previous stage. The prompt prepared in this way is ready to be passed on to the next stage.
- Stage 3: Here we are dealing with a classic LLM query and the generation of an answer. On the basis of the prompt, it generates advice or a hint for the user.
- Stage 4: This stage is only a safeguard, thanks to which we can be sure that we will get a string as output. In most cases the model will already return the answer in the right format, but this does not always happen, and not with all models, so it is good practice to process the answer appropriately at the end of the chain.

The RAG chain used in the discussed implementation is fairly simple, but its modular structure means that it can be extended very easily and incrementally with further stages. LangChain is a powerful framework offering a great deal of functionality going beyond the scope of the problem discussed, but despite this, building a small RAG chain is relatively simple and uncomplicated.

7.3.3. Vector database

ChromaDB is a vector database created specifically for the needs of applications working with LLMs. It is designed to store *embeddings*, i.e. vector representations of natural language sentences. Thanks to this, the *retriever*, after vectorising the user's query (e.g. when building a chatbot), can compare the query with the documents in the database very efficiently and return the most similar

documents. The documents found can then be used as additional context when the LLM generates an answer.

7.3.4. API

FastAPI is a modern and efficient framework for building APIs in Python. It is easy to learn and allows fast development. It was used in the AI module to expose an API for the chatbot. For this functionality we do not need many endpoints or complicated logic, which is why it was important that the solution be simple and minimalistic, but at the same time effective.

7.4. Analysis of available AI models

The LangChain library offers integration with many LLM models, but it was decided to choose the models offered by the French company MistralAI. The discussed system uses 2 models:

- Mistral Large: the newest and most advanced model currently offered by MistralAI. According to benchmarks, it is the second-best model currently available via an API, after the OpenAI model.³²
- Mistral Embed: an embedding model, i.e. a model specialised in converting natural language sentences into *embeddings*. It is used to embed both the data about the application entered into the database and the users' questions, which are compared with the data in the database.³³

The main deciding factor in choosing these models was the favourable offer in the free tier. As the application develops, the models can easily be replaced with paid alternatives.

7.5. Sequence diagram of the AI module

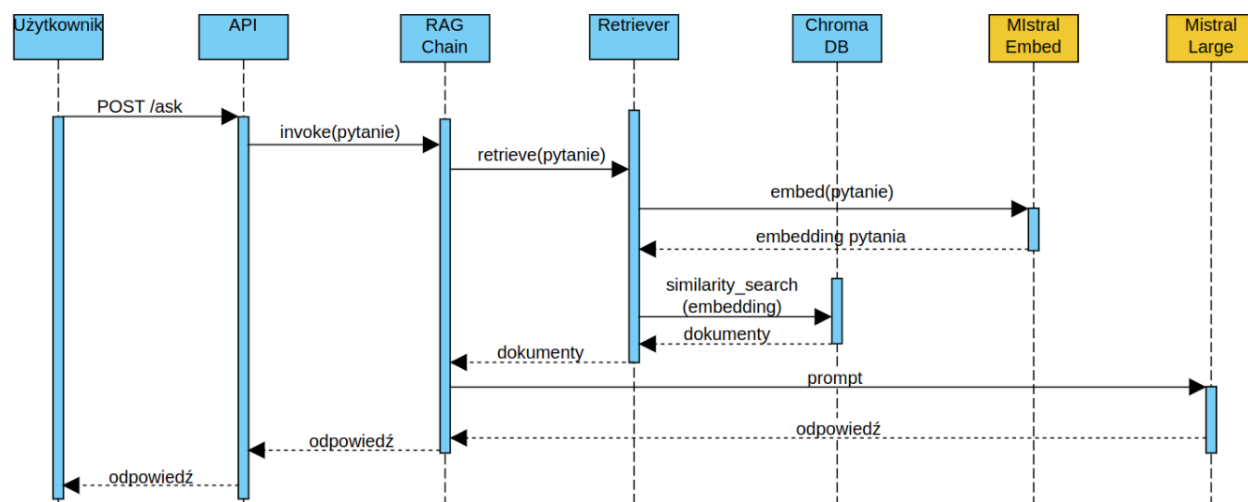


Figure 19: Sequence diagram of the AI module. Yellow denotes services outside the system.

Notes

31. LangChain. *Build a Retrieval Augmented Generation (RAG) App: Part 1*.

<https://python.langchain.com/docs/tutorials/rag/>. Accessed: 2025-06-24. 2024.

32. MistralAI. *Mistral Large, our new flagship model*. <https://mistral.ai/news/mistral-large>. Accessed: 2025-06-17. 2025.
33. MistralAI. *Models Overview*. https://docs.mistral.ai/getting-started/models/models_overview/. Accessed: 2025-06-17. 2025.

8. System infrastructure

The infrastructure of an IT system is the set of components necessary to operate and manage an enterprise's IT services and environments.³⁴ It consists of hardware elements, such as servers, network devices and data centres, and of software, including operating systems and applications, which work together to ensure efficient communication, performance and security of the system.³⁵

Proper planning and design of the infrastructure in line with the needs of the product is important, because it affects the stability, scalability and security of the system, and thus its business success and competitiveness.

8.1. Division of infrastructure into traditional/on-premises and cloud

Nowadays IT infrastructure is divided into two basic categories according to the level of control and the directness of access to resources: traditional (on-premises) and cloud.³⁶ Cloud solutions are gaining more and more popularity and are often the preferred choice due to their flexibility and scalability.

Traditional infrastructure

Traditional infrastructure is based on physical IT resources that are installed and managed locally at the company's premises or in its designated facilities. This type of infrastructure includes, for example, servers, network devices, desktop computers or even microcomputers such as the Raspberry Pi, which perform computing or control functions.

Advantages of traditional infrastructure

The advantage of traditional infrastructure is full and direct control over the hardware and data and the ability to adapt the system to the requirements of a specific organisation/project. In addition, on-premises solutions can offer lower latency and give greater independence from external service providers.

Disadvantages of traditional infrastructure

The disadvantages of traditional infrastructure include potentially high maintenance costs, including expenditure on hardware, energy and technical staff, and limited scalability compared to many cloud solutions. In addition, updating and expanding the infrastructure can be time-consuming and complicated.

Cloud infrastructure

Cloud infrastructure is a model of delivering IT resources over the Internet, in which users use services and resources without the need to have physical hardware on site. It is characterised by a wide spectrum of services with different levels of control over the infrastructure — from dedicated servers and VPSs, through data storage services such as Amazon S3, Azure Blob Storage and Google Cloud Storage, to multimedia platforms such as Cloundinary.

Advantages of cloud infrastructure

The advantages of cloud infrastructure are above all flexibility and the ease of scaling resources as needed, which allows cost optimisation and fast adaptation to changing system loads or business requirements. In addition, cloud solutions often offer advanced security mechanisms and high availability.

Disadvantages of cloud infrastructure

The disadvantages of cloud infrastructure are potential concerns about data security and privacy, as well as dependence on an external service provider. This can limit control over the system and the availability of services in the event of failures or connectivity problems. In addition, the costs of long-term cloud use can be difficult to predict and control.

8.2. Choice of an OVH dedicated server and a domain name

In the case of the Passed Pawn system, given the low maturity of the project, the optimal infrastructure is one that makes it possible to minimise the initial effort of operating and developing it as much as possible, while remaining flexible to modify and imposing as few specific solutions as possible. In the phase of building a project from scratch, the functional and non-functional requirements and the assumptions about the architecture and look of the project can change easily and quickly, and therefore flexibility and as much control over the system as possible work in its favour. At the same time, limiting work on the infrastructure as much as possible makes it possible to focus on the business logic of the system and thus to verify more quickly the assumptions about the requirements and architecture of the system and the business potential of the project itself.

In this case, infrastructure in the form of a dedicated server can be considered a good middle ground between ensuring maximum control and minimising the amount of work on the infrastructure. A **dedicated server** is a physical computer in a specific data centre whose hardware resources (processor, memory, storage) are intended exclusively for the use of one customer, giving them full control, high performance, reliability and privacy in using the hardware, without sharing it with other users. At the same time, it is a solution that abstracts the physical management of the hardware away from the user, which minimises the initial work of preparing the infrastructure. It is therefore a solution that meets the requirements set out for the Passed Pawn system.

8.2.1. Dedicated server offer and specification

For the needs of the project, the KS-4 dedicated server from OVH's range of budget/economy dedicated servers is used.

Specification

- **Processor** — Intel Xeon-E3 1230v6 - 4c/8t - 3.5 GHz/3.9 GHz
- **Memory** — 32 GB ECC 2133 MHz
- **Storage** — 2×450 GB SSD NVMe, Soft RAID

The offer includes a Service Level Agreement (SLA), which guarantees server availability of 99.9% of the time on a monthly basis.³⁷ The cost of renting the server is PLN 118.08 per month plus an additional, one-off PLN 94.71 for setting up the server. These prices include 23% VAT.

8.2.2. Domain name offer and specification

In order to provide access to the application without the need to enter the server's IP address, a domain name was purchased from OVH. For PLN 54.72 per year (including 23% VAT) the domain passed-pawn.com is registered, and the offer also includes the DNSSEC service, which protects the domain against cache poisoning vulnerabilities. In addition, the offer includes optional handling of the domain by OVH's DNS servers.

8.3. Dedicated server software

This section contains the specification and description of the most important elements of the server software at the operating system level, which were installed and configured to ensure the security, stability and functionality of the server environment. Below is a specific list of the most important technologies, tools and configurations used to operate the server.

- **Debian 12 (Bookworm) with the latest updates** — a stable and modern Linux distribution offering broad architecture support and a rich set of packages
- **Remote SSH access** — secured by allowing only authentication with an SSH key added on the server — increases security by eliminating password login
- **SSH login to the root account disabled** — an additional layer of security that forces logging in to a user account with sudo privileges
- **Fail2ban** — a tool for protecting against brute-force attacks, which monitors logs and blocks suspicious IP addresses
- **UFW (Uncomplicated Firewall)** — a firewall configured to block all incoming traffic except the SSH, HTTP, HTTPS and Kubernetes ports, while allowing all outgoing traffic
- **K3s** — a lightweight Kubernetes distribution that allows containers to be managed in a production environment
- **Helm** — a package manager for Kubernetes that makes it easier to install and manage containerised applications
- **Docker** — a platform for building, distributing and running containers, providing isolation and ease of application deployment
- **OpenTofu** — a tool that makes it possible to manage the configuration of infrastructure and resources in a declarative way

A server environment configured in this way provides stability, security and flexibility in managing applications and services on the dedicated server.

8.4. Kubernetes – the chosen system orchestration tool

Kubernetes (K8s) is an open-source platform designed by Google and certified by the CNCF (Cloud Native Computing Foundation) for orchestrating system resources, usually in the form of containers

but also virtual machines, which automates their deployment, scaling and management.³⁸ It is difficult to single out one key/main function of Kubernetes, which results from its versatile potential applications and the wide spectrum of value it delivers.

Advantages of Kubernetes³⁹⁴⁰

- **Process automation:** automates the deployment, scaling and management of containerised applications
- **Scalability:** enables automatic scaling of applications, both horizontal and vertical, which can adapt to changing load
- **High availability:** offers mechanisms that increase fault tolerance through automatic failover and load balancing
- **Self-healing:** allows containers that have failed to be restarted automatically, replaced, and to have traffic withheld from them until they are ready to receive it
- **Portability:** enables applications to be deployed in different environments: on-premises, in the public cloud and in hybrid environments
- **Configuration and secret management:** allows sensitive data and configuration to be stored and managed securely
- **Load balancing and service discovery:** can expose containers using DNS names or IP addresses and balance network traffic
- **Automated rollouts and rollbacks:** enables tightly controlled deployment of new application versions and easy rollback of changes
- **Extensibility:** allows new functionality to be added without modifying the source code, thanks to Custom Resources and Operators

Disadvantages of Kubernetes

- **Steep learning curve:** use in complicated, complex or large-scale business cases may require considerable knowledge and experience, which rules out use by beginners
- **Complexity:** Kubernetes is a complex system, which can lead to complicated management and troubleshooting in the case of ill-considered/poorly controlled cluster growth
- **Potentially inefficient for small projects:** the minimum level of knowledge needed to operate Kubernetes may be too high compared to the benefits it offers for small start-ups, small projects or prototypes

8.4.1. Flexibility of Kubernetes

One of the key features of Kubernetes is certainly its flexibility. Kubernetes enables dynamic management of resources and automatic scaling of containerised applications, and even virtual machines, in various environments: on-premises, on virtual private servers (VPS), on dedicated servers and as part of dedicated services from the largest cloud providers, such as AWS (Elastic Kubernetes Service — EKS), Azure (Azure Kubernetes Service — AKS) and Google Cloud (Google Kubernetes Engine — GKE). It is also able to handle mixed (hybrid) environments, where part of the application runs locally and part in the cloud. Kubernetes also offers significant extensibility potential, providing a standardised way of extending its core functionality using Custom Resources (CR) and Custom Resource Definitions (CRD) and the documented Operator pattern. It thus opens up a wide range of

possibilities and makes it easier to introduce good infrastructure management practices such as GitOps and Infrastructure as Code. An example of using part of this potential is CloudNativePG, which, as an Operator for PostgreSQL on Kubernetes, automates the management of database clusters in a declarative and auditable way, including replication, backups and high availability, all thanks to the use of CRs, CRDs and the Operator pattern.

8.4.2. Reliability and scalability of Kubernetes

Kubernetes strengthens system reliability thanks to mechanisms for automatically restoring services, load balancing and replication management, which minimises the risk of downtime and failures. It enables applications to be scaled both horizontally (by adding replicas and nodes) and vertically (by increasing the resources of existing instances), which makes it possible to adapt efficiently to changing demand. In addition, thanks to extensions such as Operators, Kubernetes also supports high availability for external components, e.g. databases, providing a solid and resilient infrastructure. As a result, this platform is a key element in building reliable and scalable production environments.

8.4.3. Justification for using Kubernetes

The choice of Kubernetes for the logical management of Passed Pawn's resources is justified by its flexibility, easy extensibility, declarativeness, auditability, free use and broad support from the open-source community.

As a whole, Kubernetes is an ideal ecosystem that makes it possible to iteratively improve the system's infrastructure in almost every respect — scalability, reliability, high availability, performance, portability, security and process automation. A key advantage is the possibility of working on the infrastructure in small steps, without the need to have broad knowledge straight away or to make all decisions at an early stage of system design.

Thanks to this, even with limited knowledge of the tool itself and its extensions, or with not entirely precise requirements for the system and its architecture, it can be introduced gradually, taking a proactive approach to challenges that may arise in the future. Kubernetes and its extensions offer ready-made, widely supported, proven and standardised solutions to many potential problems and needs.

8.5. K3s as the chosen K8s distribution

Kubernetes distributions are ready-to-use packages that make it easier to install and manage a Kubernetes cluster, often offering already configured components and various optimisations for specific uses. They differ from each other in their set of built-in features, size and resource requirements.

K3s is a lightweight Kubernetes distribution certified by the CNCF, designed with small size, low resource usage and simple use in mind.⁴¹ It is delivered as a single binary file smaller than 100 MB, which it achieves by not including potentially unnecessary modules from the original Kubernetes. The minimum requirements for running a K3s cluster are 2 GB of RAM and 2 CPU cores.⁴²

Advantages:

- **Lightness:** very small size and low resource usage, ideal for resource-constrained environments
- **Ease of installation:** simple installation and fast start-up
- **Full compatibility with Kubernetes:** full compatibility with the Kubernetes API, which allows standard tools and manifests to be used

Disadvantages:

- **Fewer features than full Kubernetes:** some features that may be needed in more complex production environments have been deliberately removed
- **Fewer configuration options:** less flexibility in configuring some components compared to the full distribution

The choice of K3s for Passed Pawn is justified by its lightness and the simplicity of installation and management. K3s makes it possible to deploy and manage a cluster quickly without overloading the infrastructure, while maintaining full compatibility with the Kubernetes API, which makes it possible to use all the standard tools, patterns and extensions in further improving the infrastructure.

8.6. The most important custom components of the cluster

8.6.1. Longhorn

In short, Longhorn is a tool for dynamic storage provisioning for Kubernetes. The need for and the problem of storage provisioning for Kubernetes stem from the fact that containers are by nature ephemeral and without dedicated storage their data disappears after they stop running, while Kubernetes has no built-in mechanism for dynamically providing cloud or local disk space. Kubernetes is, however, compatible with the CSI (Container Storage Interface) standard, i.e. an interface specifying how storage management systems are integrated with Kubernetes. Longhorn has its own built-in CSI driver and thanks to it can dynamically manage local and cloud disk storage for Kubernetes. In container environments, providing persistent volumes (PV) that can be dynamically created and managed is necessary so that stateful applications can store their data independently of the Pod life cycle.

Longhorn is the preferred choice for application resources, especially stateful ones, because, unlike the storage provisioning tool built into K3s (Local Path Provisioner), it offers features that take care of the reliability and security of data, and also provides the potential to implement mechanisms ensuring high availability of data.

The most important features of Longhorn⁴³

- Volume replication
- Volume snapshots
- Backups to external systems (S3, NFS)
- Disaster recovery
- Volume cloning
- Encryption
- Intuitive management dashboard

8.6.2. Nginx Ingress Controller

Before explaining what the Nginx Ingress Controller is, we should look at the concept of Ingress in the context of Kubernetes. Ingress is a resource type built into Kubernetes that manages external access to services in the cluster, usually via HTTP and HTTPS. Ingress acts as a router, directing traffic to the appropriate services on the basis of host and path rules.⁴⁴

The Nginx Ingress Controller is an Ingress controller that uses the Nginx server as a reverse proxy and load balancer, automatically configuring the Nginx server according to the declarations contained in Ingress resources.⁴⁵

Advantages:

- **Advanced routing:** extensive routing capabilities (host, path, headers)
- **Useful features:** TLS support and easy certificate management
- **High performance and reliability:** the components use relatively few resources while offering high-availability mechanisms that increase reliability

Disadvantages:

- **Complexity:** greater configuration complexity compared to a self-prepared nginx proxy
- **Higher resource usage:** higher baseline resource usage than lighter solutions

The Nginx Ingress Controller is essential for managing complex incoming traffic to the system and provides flexibility and scalability.

8.6.3. Cert-manager

Cert-manager is a Kubernetes controller following the Operator pattern that automates the management of SSL/TLS certificates, making it easier to obtain and renew them and to integrate them with Ingress resources.⁴⁶

Advantages:

- **Process automation:** the certificate life cycle can be automated
- **Useful integrations:** offers integrations with Ingress and popular certificate authorities (e.g. Let's Encrypt)
- **Centralised certificate management:** acts as a centralised place in which certificates are managed in an automated way

Disadvantages:

- **Complexity:** the initial configuration can be complicated

Cert-manager is key to ensuring secure communication and automating certificate management in Kubernetes.

8.6.4. Other custom elements of the cluster

- **Containerd:** a lightweight container runtime that manages the container life cycle

- **Flannel:** an overlay network for communication between Pods on different nodes
- **Network Policy controller:** a network policy controller that implements the rules of communication between Pods
- **Spiegel:** a tool for creating a local mirror of the container image registry, speeding up image pulls and reducing dependence on the Internet
- **ServiceLB:** the default load balancer in K3s, which makes it possible to use LoadBalancer services without needing a load balancer from an external cloud provider

8.7. Application development and deployment infrastructure

In this description, the infrastructure related to application development and deployment is understood as all the technical resources and tools necessary for creating, testing, integrating and deploying software. Its most important parts are source code management systems and process automation tools. Planning and identifying these elements in the context of a specific system is crucial, because it allows effective management of the software development process, minimises the risk of errors and speeds up the delivery of new features and fixes.

8.7.1. Jenkins server

From the perspective of infrastructure design, choosing Jenkins **provides a big benefit in the form of direct control** over the automation tool **and operation within the system's own chosen and controlled infrastructure**. This makes it possible to adapt the operation of the automation tool as much as possible to the individual and changing needs of the project. **More about the Jenkins tool itself is written in the context of process automation related to system development and the DevOps approach, i.e. in section 9.**

8.7.2. GitHub services

GitHub, despite alternatives that allow self-hosting, is the choice for Passed Pawn because of its **simplicity of use, rich functionality for managing stored source code and supporting automation, and because the Passed Pawn project team knows the platform well.**

The most important GitHub services and features used in Passed Pawn are:

- Hosting of Git repositories
- Pull/Merge Requests for code review and integration
- GitHub Actions for test automation
- An OCI registry for storing deployment artefacts
- Integration with code analysis tools — GitHub Copilot
- Mechanisms (branch rules) for enforcing restrictions on introducing changes to the source code and adherence to defined change protocols

It should be noted that these features are used in the form of **Software as a Service (SaaS)**, which translates into limited control over them and dependence on the service provider — GitHub.

8.8. Keycloak server

From the perspective of infrastructure analysis, Keycloak, like Jenkins, **provides a big benefit in the form of direct control** over the tool and **operation within the system's own chosen and controlled infrastructure**. This makes it possible to adapt identity and access management as much as possible to the individual and changing needs of the project. **More about the Keycloak tool itself is written in the context of system security, i.e. in section 10.**

8.9. Use of Infrastructure as Code (IaC) and GitOps good practices

8.9.1. Origins of IaC

Before IaC (Infrastructure as Code) practices appeared and gained recognition in the IT community, the traditional approach to managing the configuration of servers, networks and other infrastructure components was manual. It was characterised by a low degree of automation and declarative configuration and a high degree of manual and imperative management through step-by-step execution of prepared instructions by a human. This model was, and still is, associated with susceptibility to errors, inconsistent results and time-consumption. IaC emerged as a response to these problems, with the idea of moving infrastructure management processes to the programming layer by describing the desired characteristics in configuration files using languages such as YAML, JSON or specialised DSLs (Domain-Specific Languages).⁴⁷

8.9.2. Definition and significance of IaC

Grzegorz Gnych, an IT infrastructure specialist, defines Infrastructure as Code as: "[...] a fundamental change in the approach to IT infrastructure management, where technical resources are defined and managed through source code rather than manual configuration." A list of potential challenges and common problems:⁴⁸

Advantages:

- **Potential for automation** — managing infrastructure through code opens up wide possibilities for standardising and automating infrastructure management processes, which increases the speed and reliability of deployments
- **Repeatability** — increased repeatability and consistency of environments, which minimises human error and differences between environments
- **Versioning** — enables simple versioning of configuration, which makes it easier to track changes, audit them and quickly roll back undesirable modifications
- **Scalability** — ease of scaling the infrastructure and the ability to quickly create test and production environments
- **Easier collaboration** — improved collaboration between development and operations teams by making it easier to analyse the existing and proposed infrastructure, whose desired state is described in code

Disadvantages:

- **Complexity** — higher initial complexity of managing infrastructure code, requiring specialised knowledge and programming skills in the team
- **Security and risk of errors** — a bug in the code, combined with automation, can instantly destroy the entire infrastructure, e.g. by accidentally deleting a database
- **Additional state management problems** — in the case of tools such as Terraform, the stored infrastructure state can become corrupted, e.g. if individual infrastructure changes were made manually

Knowledge of these advantages and disadvantages is useful when analysing the degree of automation of deploying changes to the system's infrastructure, e.g. when considering automating the deployment of changes to critical points of the system that require maximum caution and appropriate auditing, such as the authorisation server or databases.

8.9.3. Scope of IaC practices used

The scope of IaC practices used for Passed Pawn includes:

- **Built-in Kubernetes resources (e.g. Deployment, StatefulSet, Service, Ingress, etc.)** — stored in YAML manifests and packaged using so-called Helm Charts, which additionally loosen the coupling of resources with selected configuration aspects
- **Longhorn** — managed using YAML manifests driving the official Longhorn Helm Chart
- **Nginx-Ingress-Controller** — managed using YAML manifests driving the official Nginx-Ingress-Controller Helm Chart from Kubernetes
- **Cert-Manager** — managed using YAML manifests driving the official Cert-Manager Helm Chart
- **Jenkins** — managed using YAML manifests driving the official Helm Chart for the official Jenkins Operator for Kubernetes
- **Keycloak** — managed using the Keycloak Provider extension for the HCL language (HashiCorp Configuration Language)

Thanks to such broad application of IaC practices in Passed Pawn, the management of the system's infrastructure is versioned, transparent, auditable and repeatable.

8.9.4. Origins of GitOps

At the time when Kubernetes was consolidating its position as a container orchestration platform, many organisations began to experiment in order to develop efficient methods of managing application configuration, deployment and automation while ensuring the stability and repeatability of the defined processes. In 2017 the GitOps concept was popularised by Weaveworks, which developed an approach based on Infrastructure as Code (IaC) practices in which Git acts as the one, single source of truth about the state of the system. This approach eliminated a range of problems resulting from the nature of manual infrastructure management and the deployment of clusters and applications. In addition, it simplified deployment management, making it easier to audit the changes introduced, thanks to central management of the code in Git.⁴⁹

8.9.5. Definition and significance of GitOps

Andrzej Ochman, an expert in DevOps engineering and solution architecture, defines GitOps as: "[...] an approach to managing infrastructure and applications in which Git acts as the single source of truth. This means that the entire system configuration — from Kubernetes resource definitions to security policies — is stored in a Git repository. Changes to the infrastructure are made exclusively by updating the code in the repository. They are also automatically synchronised with the actual state of the system."⁵⁰

Advantages:

- **Centralisation** — centralisation of infrastructure management through a single, auditable Git repository as the only source of truth
- **Transparency** — full transparency and control over changes thanks to version control and code review mechanisms (pull requests)
- **Automation** — automatic synchronisation of the infrastructure state with the repository ensures consistency and strengthens system reliability
- **Auditability** — easier validation and auditing of infrastructure code thanks to the change history and the ability to restore previous versions quickly.
- **Easier collaboration** — as in the IaC approach, support for collaboration on infrastructure work by increasing the transparency, auditability and analysis of the existing and proposed infrastructure

Disadvantages:

- **Adaptation** — requires the team's work processes to be adapted to the Git-based management model, e.g. by granting automation tools access to remote repositories
- **Complexity** — the initial complexity of deploying and configuring GitOps tools can be high
- **Synchronisation** — introduces a range of potential state synchronisation challenges, e.g. in the case of errors in automated processes

8.9.6. Scope of GitOps practices used

The scope of GitOps practices used for Passed Pawn covers the same parts of the system infrastructure as the IaC practices. Wherever the infrastructure is managed as code, it is also versioned, stored remotely in code repositories and continuously reflected in the state on the OVH dedicated server. The only exception is the infrastructure of the Keycloak authorisation server, where changes are not synchronised automatically but manually, although they are still stored as code. This is related to particular caution when dealing with the tool and the establishment of a requirement that changes be applied by a human, in order to make it easier to roll these changes back quickly, abandon their deployment entirely or minimise any damage caused.

Notes

34. IBM. *What Is IT Infrastructure?* <https://www.ibm.com/think/topics/infrastructure>. Accessed: 2025-06-23. 2021.

35. Ibid.

36. Ibid.

37. OVH. *Dedicated Server Service Level Agreement*.
<https://us.ovhcloud.com/legal/sla/dedicated-servers/>. Accessed: 2025-06-23. 2025.
38. Kubernetes technical documentation. *Kubernetes*. <https://kubernetes.io/>. Accessed: 2025-06-23. 2025.
39. Kubernetes technical documentation. *Concepts/Overview*.
<https://kubernetes.io/docs/concepts/overview/>. Accessed: 2025-06-23. 2024.
40. Dhaval Gajjar. *Why And Where To Use Kubernetes*.
<https://www.opensourceforu.com/2024/02/why-and-where-to-use-kubernetes/>. Accessed: 2025-06-23. 2024.
41. K3s technical documentation. *K3s - Lightweight Kubernetes*. <https://docs.k3s.io/>. Accessed: 2025-06-23. 2025.
42. K3s technical documentation. *Requirements*. <https://docs.k3s.io/installation/requirements>. Accessed: 2025-06-23. 2025.
43. Longhorn technical documentation. *Features*. <https://longhorn.io/docs/1.9.0/>. Accessed: 2025-06-23. 2025.
44. Kubernetes technical documentation. *Ingress*. <https://kubernetes.io/docs/concepts/services-networking/ingress/>. Accessed: 2025-06-23. 2025.
45. Kubernetes ingress-nginx repository. *ingress-nginx*. <https://github.com/kubernetes/ingress-nginx>. Accessed: 2025-06-23. 2025.
46. cert-manager technical documentation. *cert-manager*. <https://cert-manager.io/docs/>. Accessed: 2025-06-23. 2025.
47. Grzegorz Gnych. *Czym jest Infrastruktura jako kod (Infrastructure as Code)? – Kompendium wiedzy* (What is Infrastructure as Code? – A compendium of knowledge). <https://nflo.pl/baza-wiedzy/czym-jest-infrastruktura-jako-kod-infrastructure-as-code-kompendium-wiedzy/#Integracja-z-Tenable-i-Rapid7>. Accessed: 2025-06-25.
48. Ibid.
49. Andrzej Ochman. *GitOps cz. I. Wprowadzenie do GitOps i jego rola w konteneryzacji* (GitOps part I. Introduction to GitOps and its role in containerisation). <https://linuxpolska.com/pl/baza-wiedzy/blog/gitops-wprowadzenie-jego-rola-w-konteneryzacji-cz-i/>. Accessed: 2025-06-25. 2025.
50. Ibid.

9. Process automation, the DevOps approach

Manual management of software maintenance and development processes can involve challenges and problems such as human error and poor repeatability and scalability of processes. Practising IaC is a significant step towards minimising the imperative style of managing software infrastructure by manually executing many instructions. In the end, however, even Infrastructure as Code has to be applied at some point, and the process of applying it can be more or less complicated and consist of fewer or more steps. Automating this process is in fact the completion of the grand vision of eliminating manual management of software infrastructure. Appropriate automation can take the same advantages that IaC provides a step further — advantages whose essence lies precisely in eliminating the human factor from the whole equation. Alongside treating the repository as the single source of truth about the infrastructure, this is exactly one of the two main postulates of the GitOps practices presented in the previous chapter. This chapter, however, is concerned not only with automatic synchronisation of infrastructure code with the repository, but with the general automation of the development and maintenance processes of Passed Pawn.

Advantages of process automation:

- **Faster development in the broader perspective** — a significant reduction in the time needed to carry out deployments and changes thanks to the elimination of manual activities
- **Fewer errors and repeatable processes** — a reduced risk of human error through standardisation and ensuring the repeatability of processes
- **Consistency** — ensuring uniform environments and configuration through standardisation and ensuring the repeatability of processes
- **Scalability** — the ability to adapt quickly to growing requirements and changes in the infrastructure

Disadvantages of process automation:

- **Implementation costs** — potentially high time and financial outlays on the initial configuration and on training the people responsible for automation
- **Maintenance** — the need to constantly update and monitor automation scripts in order to avoid errors
- **Risk of errors** — in the case of deployments, the effects of errors in automation can be particularly severe and affect many environments at once
- **Security** — automation tools constitute a new attack vector and system vulnerability that must be taken into account and secured

9.1. Methodology of developing and maintaining the Passed Pawn application

The process of developing and maintaining the Passed Pawn application took place with the DevOps approach to software development in mind and with the application of CI/CD (Continuous Integration, Continuous Deployment) practices. The aim of this approach and of applying these practices is to enable members of the development team to manage the system's infrastructure effectively. Given the development background and experience of all members of the Passed Pawn team, it is

precisely making work with the operational part of the system easier that is the main object of improvements in the spirit of DevOps.

9.1.1. Application of DevOps practices

Since it was first formulated by Patrick Debois in 2009, the understanding of the term DevOps has become blurred, and many definitions of it can be found which can differ greatly from one another. The definition adopted in the process of designing the Passed Pawn system is the one coming from the author of the term. In the afterword to the second edition of the book "The DevOps Handbook", Patrick Debois defined DevOps as follows: "[...] everything you do to overcome the friction between silos. All the rest is plain engineering."⁵¹ The silos meant here are the software development domain (Dev) and the operations domain (Ops). The author adds that this definition emphasises that building technology alone is not enough to practise DevOps – one also needs a constant goal, which is to remove the point of friction. The author explains that this point of friction will move as the current blockers are removed. Continuously assessing the bottleneck in the software development process is key. He also notes that nowadays organisations constantly optimise pipelines and automation, but do not work on other points of friction that cause blockers, and that this is currently one of the main challenges of software development.⁵²

Examples of practices applied in the spirit of DevOps:

- Automation of the build, test and deployment processes
- Continuous integration and continuous delivery (CI/CD)
- Shared responsibility of development and operations teams for how deployments work
- Using infrastructure as code (IaC) to manage environments
- A culture of continuous learning and process improvement

Applying DevOps practices in the development and maintenance of the Passed Pawn project makes it possible to increase the efficiency of the team's work, shorten deployment times and improve the quality and stability of the application. In addition, it helps to build a developer-friendly methodology of working on the development of the system's infrastructure and on the processes of its deployment and maintenance.

9.1.2. Application of CI/CD practices

CI, or Continuous Integration, is an engineering practice that consists in frequently committing code changes to a shared repository, ideally several times a day, and automatically building these changes. These changes are compiled together with other parallel modifications in the system, which allows early detection of problems with integrating the work of the many programmers participating in the project. Build errors resulting from failed integration are treated as the highest priority for the whole team, and work usually stops until they are fixed.

Combined with automated testing, continuous integration also makes it possible to test the integrated version, thanks to which we can check not only whether the code still compiles correctly, but also

whether it remains functionally correct. It is also a best practice in building solid and flexible software systems.⁵³

CD, or Continuous Delivery, extends the concept of Continuous Integration (CI) by also testing deployments of the integrated code in an environment that is a replica of the target production environment. This allows early detection of unforeseen operational problems resulting from the changes introduced and the identification of gaps in the tests.

The aim of these practices is to ensure that the main branch of the code is always ready for deployment, which means that, if necessary, the current version can be taken from this branch and deployed to production.⁵⁴

Examples of CI/CD practices:

- Automatically running unit and integration tests when the code changes
- Automatically building and packaging the application
- Automatically deploying to test environments
- Verifying code quality and security in the CI/CD pipeline

Adopting CI/CD in the development of the Passed Pawn system, in the form of automated tests and automation of building and deploying the application and other system components, made the process of integrating changes from different parts of the system and from different team members easier. Automation makes it possible to increase the frequency of merging changes and testing code by relieving team members of manual activities. Automatic building and deployment of the application also helps with manual testing, because it ensures the existence of a development environment that always contains a version of the application synchronised with the latest changes. Thanks to this, developers have a greater and more frequent chance of finding bugs in the system.

9.2. Jenkins – the main process automation tool

Jenkins is an open-source automation server that makes it possible to build, test and deploy software. Its main task is to automate processes, which allows new features to be delivered faster and more reliably.

Advantages:

- **Open source and free** — available free of charge, which allows wide use without additional licensing costs
- **A huge number of available plugins** — allows integration with many tools and extension of functionality
- **Flexibility and the ability to adapt to different environments** — ease of configuration for the specific requirements of projects
- **Broad support for different programming languages and tools** — makes it possible to work with many technologies and frameworks
- **A large user community and extensive documentation** — makes it easier to solve problems and develop the system

Disadvantages:

- **Complexity of configuration and maintenance** — requires experience and time to set up and manage correctly
- **Potential security problems with incorrect configuration** — risk of security vulnerabilities if the system is not properly secured
- **Poor support for a large number of plugins and extensions** — some plugins may be outdated or poorly maintained
- **Limited base capabilities** — **heavy dependence on plugins and extensions** — basic functionality is limited without additional extensions

9.2.1. The most popular alternatives to Jenkins

GitLab Runner

GitLab Runner is an agent that executes CI/CD jobs within the GitLab platform. It makes it possible to run many jobs at the same time, supports various execution environments, including Docker and SSH, and offers automatic scaling on various clouds and hypervisors.

ArgoCD + Argo Workflows

Argo Workflows is a Kubernetes-native job orchestration engine that makes it possible to define complex CI/CD pipelines. ArgoCD, on the other hand, is used for continuous delivery and management of applications on Kubernetes. Together they form a modern solution for cloud and container environments.

Cloud services: Azure DevOps, AWS CodePipeline, CodeBuild, CodeDeploy, GCP Cloud Build, Cloud Deploy, Cloud Test Lab

These are the automation services of the largest cloud providers – Azure, AWS and GCP. They offer comprehensive, integrated CI/CD solutions in the cloud that make it possible to automate the building, testing and deployment of applications. They offer scalability, integration with other cloud services and support for various languages and platforms, which makes it easier to manage DevOps processes in multi-cloud and hybrid environments.

9.2.2. Scope of Jenkins usage

Jenkins is used mainly to automate the processes of building and deploying software. It monitors source code repositories and runs jobs in response to changes, which allows errors to be detected quickly and high code quality to be maintained. It works well both in small teams and in large organizations that need a flexible and extensive CI/CD tool.

9.2.3. Justification for choosing Jenkins

Jenkins is the main automation server for Passed Pawn because of its flexibility, full control over management, the wide availability of training materials and technical documentation, and its potential for automating tests with unusual and complicated needs. Despite certain drawbacks, such as compli-

cated configuration or declining plugin support, its maturity and continuing popularity make it a solid choice at the current stage of the system's development.

9.3. Division of responsibility for pipelines

In the proposed architecture, each repository is responsible for managing its own CI pipelines, which allows teams/team members to define and maintain automation processes independently according to their needs. At the same time, all pipelines must follow a common, centrally maintained template, which ensures consistency and standardisation of how CI pipelines are declared and stored, requiring them to be located at `/cicd/pipelines/ci.jenkins`.

Below is a diagram illustrating the division of responsibility for CI pipelines and how the automation system works in the process of detecting changes in repositories and running the pipelines responsible for building and pushing deployment artefacts to the OCI registries corresponding to the repositories.

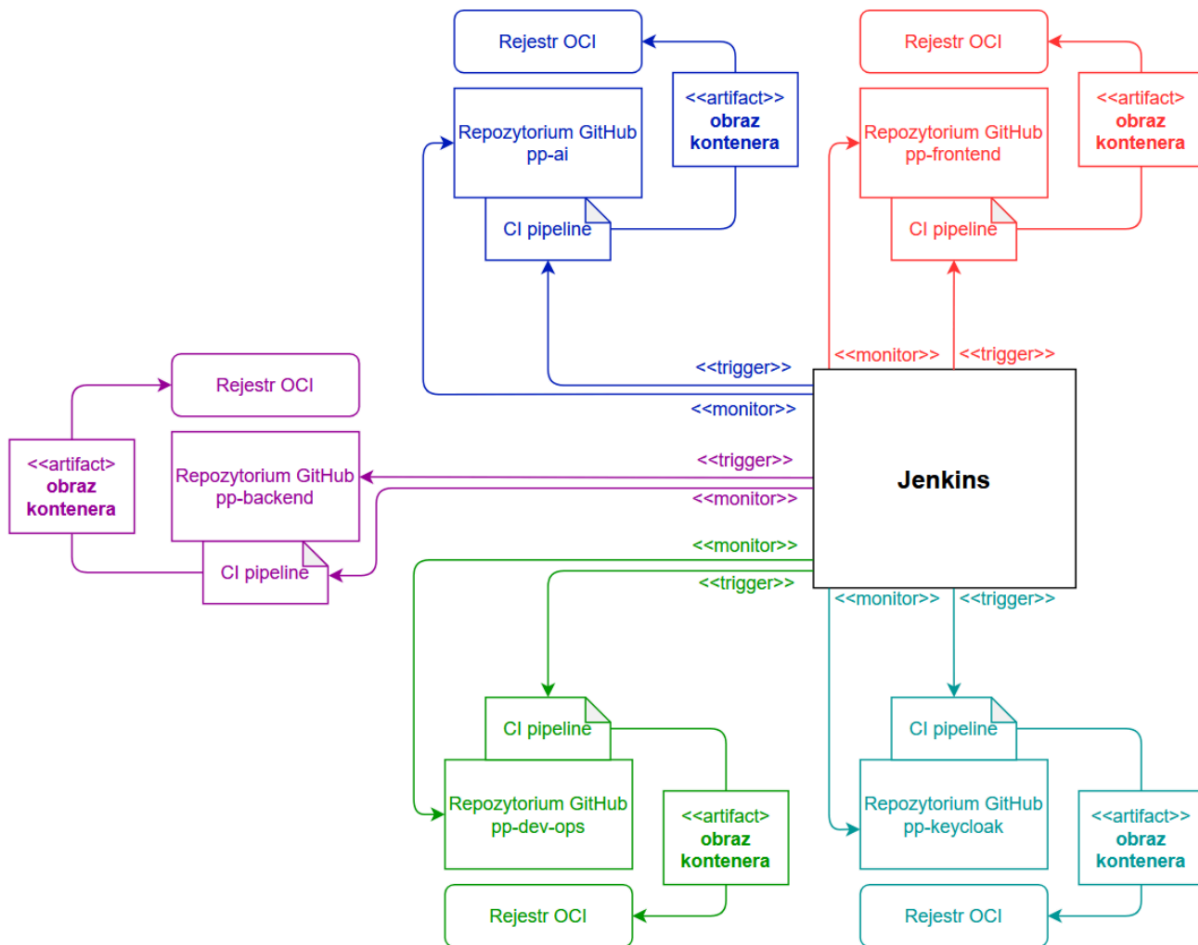


Figure 20: Diagram of the CI pipeline architecture

The Jenkins server periodically monitors the state of the repositories in order to detect changes on the main branch. If it finds changes, it runs the CI pipeline corresponding to the given repository, whose task is to build and push the corresponding deployment artefact to the corresponding OCI registry. The role of the people responsible for managing the given repository is to define the pipeline and make sure it fulfils its purpose. After the pipeline completes successfully, another pipeline is run, for whose maintenance the people managing the system infrastructure repository, i.e. pp-dev-ops, are responsible this time. This pipeline takes the newly built and pushed deployment artefact into account and builds and pushes the application's Helm Chart updated with the version of this artefact, i.e. a single deployment unit of the application intended for deployment on the Kubernetes cluster. All this ensures proper versioning of the individual components of the application and of the application as a whole. The version of the final deployment unit of the application has the following format:

```
<yyyy-MM-dd'T'HH-mm-ss>C<hash-of-the-last-commit-on-main-branch-pp-dev-ops>--  
<version-of-the-latest-frontend-application-artefact>--<version-of-the-latest-back-  
end-application-artefact>--<version-of-the-latest-ai-application-artefact>,
```

where the versions of the artefacts of the specific applications are likewise

```
<yyyy-MM-dd'T'HH-mm-ss>C<hash-of-the-last-commit-on-main-branch-of-the-repository>.
```

An example name of a deployment artefact from the pp-frontend repository (additionally prefixed with the registry name):

```
ghcr.io/passed-pawn-dev/pp-frontend/pp-frontend:2025-06-21T20-04-21C1efadfd
```

An example name of the deployment artefact of the whole application (also prefixed with the registry name):

```
ghcr.io/passed-pawn-dev/pp-dev-ops/charts/pp-app:2025-06-22T14-24-13C1436ff7--2025-  
04-13T20-03-05C259dfac--2025-06-22T14-21-23C6dcc528--2025-05-29T09-44-55Czda321h
```

The presented way of versioning the application as a whole and its individual elements separately makes it easy to analyse code versions, which is particularly useful in the event of failures or errors.

Below is a diagram presenting how the image of the whole application is built and pushed, which takes place after every successful run of a CI pipeline.

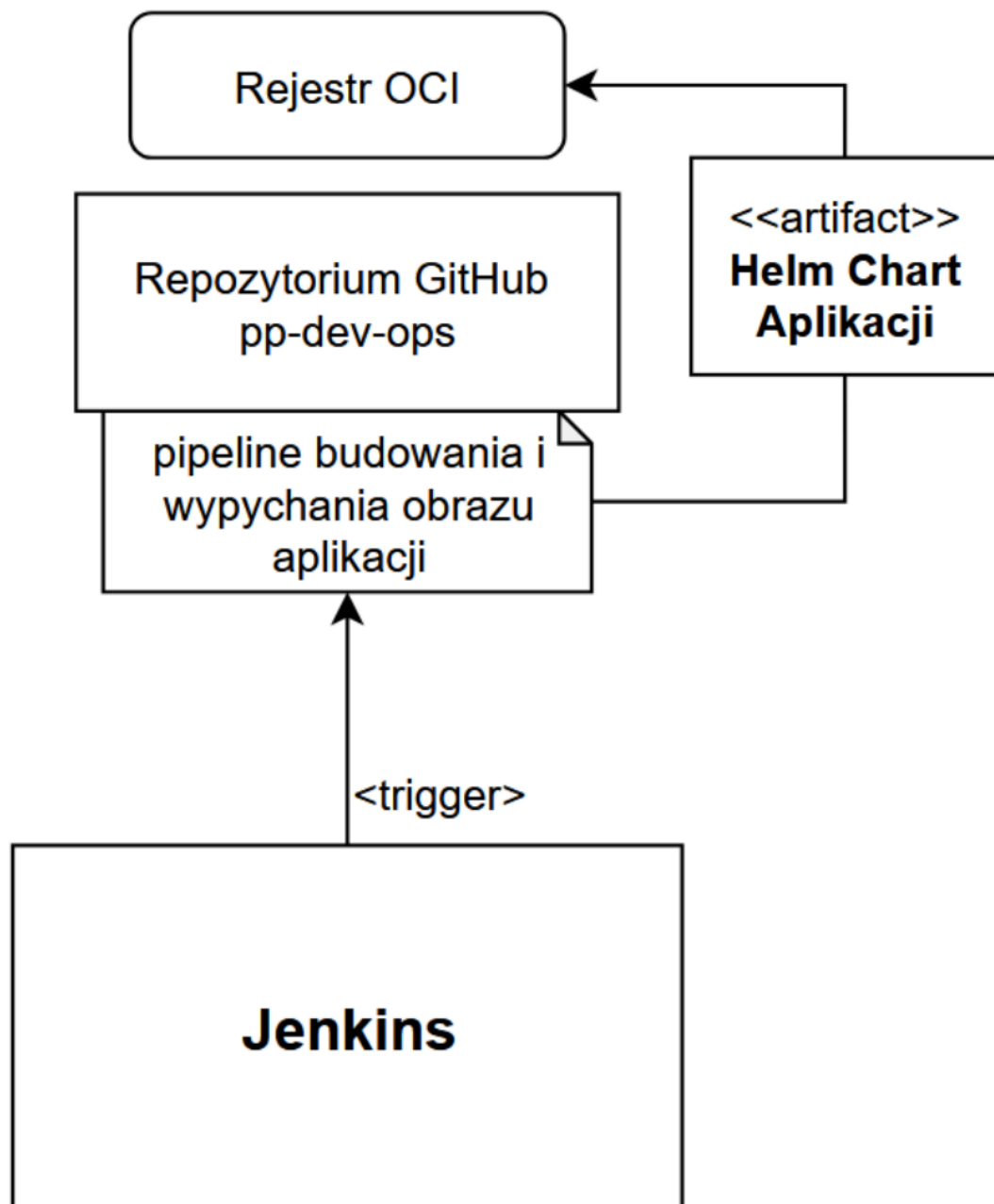


Figure 21: Architecture of building and pushing the application image in the form of a Helm Chart

Advantages of the presented architecture:

- **Team autonomy** — allows pipelines to be adapted quickly to the specific needs of the project without having to wait for central changes

- **Standardisation** — the central template enforces a uniform way of declaring CI pipelines in each repository
- **Scalability** — distributed pipeline management makes it easier to scale the system as the number of projects and teams grows
- **Easier maintenance** — updates and fixes to the template are propagated to all repositories, which reduces the effort of maintaining consistency

Disadvantages of the presented architecture:

- **Potential conflicts** — changes to the central template may require adjustments in many pipelines, which can generate additional work
- **Team discipline required** — teams/team members must be aware of and consistently follow the agreed standards

Justification: A division of responsibility in which each repository manages its own pipeline while using a central template combines the advantages of flexibility and standardisation. It allows teams/team members to respond quickly to changes and newly emerging requirements, while maintaining the consistency of the CI/CD process architecture across the whole system. Such an approach supports scalability and makes it easier to manage a large number of projects, which is key in view of the system growing or being split into smaller parts, e.g. as part of a migration to a microservice architecture. Introducing a central template as a point of reference minimises the risk of standards drifting apart and makes it easier to maintain and develop the automation infrastructure.

9.4. Jenkins — Kubernetes integration

Jenkins is implemented using the Jenkins Operator, which allows native management of Jenkins instances in the Kubernetes environment with a declarative and Infrastructure as Code (IaC) approach. The advantages of such a solution are automatic scaling, easier management of the Jenkins life cycle, built-in security practices and integration with Kubernetes and, optionally, public clouds (AWS, Azure, GCP). A drawback may be the greater complexity of the initial configuration and the need for knowledge of Kubernetes and operators. Another advantage is the set of preconfigured elements provided by the Operator, which effectively makes the work of people less experienced with Jenkins easier and shorter.

An alternative solution is to run Jenkins as an ordinary Kubernetes deployment, which is simpler but does not offer the full automation of management, scaling and updates that the operator provides, nor does it have similar IaC or GitOps capabilities. In a reasonable scenario, this approach also requires gaining the appropriate knowledge of and experience with Jenkins that would allow it to be secured and configured independently for individual needs.

Using the Jenkins Operator is justified by the GitOps and IaC approach, because the operator allows declarative management of the Jenkins configuration, which facilitates versioning, automation and repeatability of deployments. The Operator supports full life-cycle management of Jenkins, which fits in with GitOps practices, where changes to infrastructure and configuration are managed through code repositories and automated processes ensure the consistency of environments. In addition, as al-

ready mentioned, it offers a number of default configurations that provide security, reliability and broad automation functionality.

The most important built-in components of the Jenkins Operator

- **Jenkins Custom Resource (CR)** — defines a Jenkins instance in Kubernetes, enabling declarative management of the configuration and operating parameters of Jenkins, which makes it easy to maintain and replicate environments
- **Operator controller** — responsible for monitoring and managing Jenkins resources in the cluster; it automatically rolls out changes and updates while ensuring security thanks to an appropriate default configuration
- **Integration with Kubernetes and the cloud** — the operator supports backup, observability and security policy mechanisms, as well as integration with cloud services, which increases reliability and makes management easier in hybrid and multi-cloud environments

9.5. GitHub Actions – test automation

GitHub Actions is a tool for automating workflows in GitHub repositories, which makes it possible to define processes such as building, testing, packaging, releasing and deploying applications directly in the repository.

Advantages:

- **Integration with GitHub repositories** — integrated directly with GitHub, which makes configuration and management easier
- **Flexibility and cross-platform support** — support for many languages and environments, flexibility in defining workflows in YAML files
- **Control over the execution environment** — the option of using hosted runners or one's own infrastructure
- **Broad support and shared templates** — a large community and ready-made actions available for use

Disadvantages:

- **Limited functional capabilities:** less advanced capabilities compared to tools such as Jenkins, especially for very complex pipelines
- **Limited configuration capabilities:** limitations in the case of very large and complicated projects requiring extensive configuration

Alternatives to GitHub Actions

Popular alternatives include Jenkins, Azure Pipelines, GitLab CI and Argo Workflows. Jenkins, however, stands out as an open-source tool with a wide range of extensions and support for more complex pipelines. For this reason, should the system's testing requirements become more complicated or grow, the automated tests run in GitHub Actions can be moved to the Jenkins infrastructure that already exists in the system.

Scope of GitHub Actions usage

GitHub Actions is used in the project to automatically test the main server application before changes to it are approved.

Justification for using GitHub Actions

GitHub Actions is very practical for automating the process of validating and testing applications in projects of moderate complexity, because of its ease of use and integration with the GitHub repositories in which the system's code is currently stored. Should the remote code repositories on the GitHub platform be abandoned, or should the need arise to handle more complex testing pipelines, Jenkins, which is already prepared to adopt more complex processes and integrate with various platforms, offers better capabilities.

9.6. Git hooks – code validation

Git hooks are scripts run automatically in response to specific events in a Git repository, making it possible to customise and automate processes in the software life cycle.

Advantages:

- The ability to perform checks automatically before a Git operation is executed (e.g. commit, push).
- Ensuring code consistency and quality at an early stage.

Disadvantages:

- Hooks run locally and are not shared across the team by default, which requires additional configuration.
- They can be skipped or disabled by users.

9.6.1. Justification for using a pre-commit hook

A pre-commit hook makes it possible to run tests or linters automatically before changes are committed, which helps to detect errors and code problems even before they enter the repository. Despite limitations, such as the need for configuration on each developer's machine, pre-commit increases code quality and prevents errors from being introduced.

The pre-commit hook runs during work on the browser application project and runs, for example, so-called linters, which statically analyse the code before each commit, ensuring its consistency, quality and compliance with the adopted rules and conventions. Currently a total of more than 100 rules and conventions are configured, checked and enforced before every change introduced into the repository in the form of a commit.

9.6.2. Husky – a platform-agnostic wrapper for git hooks

Husky is a popular, platform-agnostic (i.e. not tied to a specific environment/operating system) wrapper for git hooks that makes them easy to manage in projects that use a `package.json` file. It works on different operating systems and integrates with popular linting and testing tools. Thanks to this, it allows code validation processes to be automated in a simple and convenient way.

9.6.3. ESLint – static code analysis

ESLint is a tool for static analysis of TypeScript and JavaScript code that identifies problems and inconsistencies with the coding style, helping to maintain high code quality. It makes it possible to configure rules and detect errors even before the application is run, which speeds up the development process and reduces the number of errors detected only at later stages of software testing.

9.6.4. Stylelint – static code analysis

Stylelint is a tool for static analysis of CSS code and other style preprocessors such as SCSS, which helps to detect errors and inconsistencies in styles, ensuring the consistency and quality of the application's visual layer. Thanks to it, formatting and correctness rules for style code can be enforced automatically, which makes the project easier to maintain.

To sum up, GitHub Actions and git hooks (in particular pre-commit with Husky) form an effective ecosystem for automating the validation and testing of the application, combining automation at the repository level with local code quality control. In the future, for more extensive pipelines, it is worth considering Jenkins as a tool offering greater configuration capabilities and scalability.

Notes

51. Gene Kim et al. *The DevOps Handbook*. 2nd ed. Accessed: 2025-06-25. IT Revolution Press, 2016.
52. Ibid.
53. Microsoft. *Continuous Integration and Continuous Delivery*. <https://microsoft.github.io/code-with-engineering-playbook/CI-CD/>. Accessed: 2025-06-25. 2024.
54. Microsoft, *Continuous Integration and Continuous Delivery*.

10. System security

System security can be defined not only as the technologies and technical mechanisms for ensuring the confidentiality and integrity of access to data and functionality, but also as the set of practices whose task is to protect the IT infrastructure. This is therefore a broad set of activities, methodologies and technical safeguards which together guarantee that the system operates as intended and is resilient to unexpected incidents and to the actions of actors with bad intentions.⁵⁵

System security is a critical aspect to take into account and design, because potential neglect in this area can result in financial losses, legal consequences or loss of reputation.⁵⁶

10.1. Keycloak central authorisation server

Keycloak, as open-source Identity and Access Management (IAM) software, is used in Passed Pawn to authenticate users and authorise their access to the main resources of the application.

The most important features of Keycloak:

- **User authentication:** including SSO login
- **Access authorisation:** access control to system resources
- **Protocol support:** support for e.g. OAuth2, OpenID Connect, SAML
- **User and role management:** the ability to define roles and permissions
- **Multi-factor authentication (MFA):** a higher level of security
- **Customisation and extensibility:** adaptation to specific needs
- **Possible integration with external identity providers:** Keycloak offers a simple way of implementing login with, for example, a Google, Facebook or GitHub account
- **Integration with LDAP and Active Directory:** potential synchronisation of user data

Advantages of Keycloak:

- **Open source and free of charge:** no licensing costs
- **Rich functionality:** support for modern industry standards
- **Easy integration:** compatibility with various applications and protocols
- **Strong support:** an active open-source community, regular updates and extensive documentation
- **Solid security mechanisms:** Keycloak offers recognised and proven security mechanisms, such as the authorization code grant flow with PKCE

Disadvantages of Keycloak:

- **Advanced configuration and complexity:** requires technical knowledge to set up optimally
- **Resource requirements:** can be resource-hungry with a large number of users
- **Adaptation to specific needs:** not all potential needs can be addressed by Keycloak

Alternative identity and access management solutions:

- **Okta:** a commercial cloud solution with broad support
- **Auth0:** an IAM platform with great flexibility and integrations

- **Microsoft Azure Active Directory:** popular in Microsoft environments

Justification for using Keycloak:

Keycloak was chosen because of its general flexibility, the control it offers and the wide range of functionality it provides. If one wants to keep the aspect of self-hosting and managing the authorisation server, it is difficult to name another free tool as rich in capabilities and as widely supported as Keycloak.

10.2. System security methodology

The design of the system's security is based on applying multiple layers of safeguards wherever possible, which is a solid approach to security. Such an approach increases the resilience of the system by minimising the various kinds of damage resulting from the potential exploitation of different attack vectors by bad actors, or even from their gaining some benefit or negatively affecting the operation of the system should one of the safeguards be broken. This type of approach is fundamental in solid systems, because it minimises vulnerability to 0-day attacks, which exploit unpatched security holes that, in the long run, cannot be avoided.

In addition, the system is designed with minimising the attack vector in mind, by limiting the surface exposed to potential threats. An example is limiting a certain range of functionality and access to the infrastructure to the necessary elements only, e.g. by restricting network traffic or access to system components from the Internet.

Another important element taken into account is the careful selection and extension of software dependencies. Unnecessary dependencies are avoided, and the libraries used are regularly updated and monitored in order to prevent the introduction of new security holes and the widening of the attack vector.

The principle of least privileged access is treated as an omnipresent mantra when planning access to system resources by end users, but equally by the system's own resources in the form of its components. It is consistently applied and considered when designing the system, so that users and processes have access only to the resources necessary to carry out their tasks. This limits the potential damage resulting from account compromise or user error and also minimises the potential attack surface of the system.

In addition, it is important to use current LTS (Long-Term Support) versions of the software used, which ensures stability and regular security updates. Thanks to this, the system is protected against known vulnerabilities and has vendor support.

Awareness of other, more detailed good security practices is also essential in more nuanced topics. Knowledge on this subject can be gained from recognised forums such as OWASP and by using code validation tools. An example of such a tool is **Popeye**, part of whose responsibility is to analyse Kubernetes cluster resources for an appropriate RBAC (Role-Based Access Control) configuration ensuring maximum security.

10.3. General security schemes and mechanisms

10.3.1. Same Origin Policy

The Same Origin Policy is a security mechanism built into browsers that restricts access to resources located on a domain other than the source page. It works by blocking the reading of the content of a server's response (or blocking the sending of the request to the server altogether) without explicit approval by the server of the origin that wants to read the response. For the sake of compatibility with old websites, the scope and the way requests are blocked are somewhat convoluted. As a rule, however, it can be assumed that in order to maximise the strictness with which the browser enforces security under the Same Origin Policy, no CORS configuration should be introduced on the server. CORS, or Cross-Origin Resource Sharing, is a mechanism that allows exceptions to the Same Origin Policy rules, controlled by the server's response headers to requests coming from the browser. So if there is no need to introduce exceptions to the security mechanisms offered by browsers, they should simply not be defined.

In browser <-> server communication, Passed Pawn uses a reverse proxy server in the form of nginx. All requests are directed to it, which creates a layer that, from the browser's perspective, provides a single origin for fetching the static files of the SPA application and, at the same time, for communicating with the main application server. This solution makes it possible not to configure CORS and not to allow cross-origin communication, which in effect protects the application's users against CSRF (Cross-Site Request Forgery) attacks, a large part of which rely on exploiting a CORS configuration set up more or less well by the server.

10.3.2. HTTP + TLS

TLS (Transport Layer Security) provides 3 aspects fundamental to the security of the user and the application, namely **confidentiality**, **data integrity** and **authentication**. Thanks to obtaining signed TLS certificates from the Let's Encrypt certificate authority, a user visiting the Passed Pawn website knows that they are on the right site and not on a site impersonating Passed Pawn. In addition, communication between the browser and the server is encrypted, which is precisely the aspect ensuring the confidentiality of communication. Moreover, thanks to the use of appropriate cryptographic mechanisms that are part of the functions provided by the TLS protocol, the browser and the reverse proxy server are able to notice when their communication has been disrupted and the transmitted data tampered with. TLS protects the user and the server against a spectrum of attacks in the MITM (Man in the Middle) category. In order to make maximum use of the potential of TLS, the reverse proxy server redirects every HTTP request, i.e. without the use of the TLS protocol, to HTTPS, i.e. with the use of TLS.

10.4. Client application security

10.4.1. Protection against XSS

Cross-Site Scripting (XSS) is a method that consists in placing unwanted code in a web application by an unauthorised person or entity. Such code can be used to steal the user's sensitive data or to perform undesirable actions on their behalf. Angular has built-in mechanisms that protect against this type of attack. One of them is automatic sanitisation of values inserted into the template. In the case of ordinary interpolation, HTML code is not interpreted at all, but converted so that it is displayed on the page in raw form. Code that should be interpreted, on the other hand, e.g. passed to the `innerHTML` or `style` attribute or to URL-related attributes, is automatically sanitised. All potentially dangerous fragments are removed, e.g. `<script></script>` tags. In the rare cases when we need to place unsafe code in the application, one of several available methods of the `DomSanitizer` service can be used, which allows us to bypass the safeguards.⁵⁷

10.4.2. Protection against video theft

Protecting the application against video theft is not understood as making it completely impossible to steal videos from the platform, because that is practically impossible. The point is essentially to make stealing videos harder and as inconvenient as possible for the average user. This is achieved by using appropriate HTML attributes on the video tag, namely attributes that remove the video download buttons from the user interface. In addition, in the script of the component responsible for displaying the video, the reaction to a right mouse click is overridden so that the user cannot open the menu containing the option to download the video. An equally important aspect is that the web application fetches the video from Cloudinary in the form of an attachment loaded directly into the browser's memory, instead of displaying it via a public URL. To download the video, a URL generated specifically for downloading the video attachment is used. The URL expires after a short time and cannot be reused. Moreover, it is used only in the page's script code, which makes it much less accessible to the average user than if it were embedded directly as the URL of a

- Finding the request to Cloudinary for the video attachment in the network tab of the browser's developer tools, downloading the binary file and then converting it to a human-friendly format
- Quickly finding and using the attachment download URL in the hard-to-read, bundled code of the page's script
- Using one's own script to extract the video as a binary file from the browser's memory and then converting it to a human-friendly format

- Recording the browser screen

In every scenario except the last one, the user must demonstrate above-average knowledge and skills in order to download a video placed on the site. These scenarios require more work and knowledge than the last one, but protecting the application against the last scenario is the hardest and requires the use of DRM (Digital Rights Management) tools and, optionally, steganography (as a mechanism for identifying people recording/downloading videos), which are complicated to use.

10.4.3. Access token management by the Keycloak server and the Keycloak adapter

In the client application, the Keycloak adapter stores 3 kinds of tokens:

- Access Token: a JWT token used for authorisation when accessing resources on the server. By default it lives for 5 minutes.
- Refresh Token: a token that makes it possible to refresh the access token after it expires without having to log in again.
- ID Token: a JWT token containing various information about the user. Some of this data is also available in the access token.

These tokens are stored directly in the browser's memory in order to avoid the risks associated with storing them in `localStorage`. However, even after the page is refreshed, when the tokens are lost, the user does not have to log in again. This is possible thanks to 2 mechanisms:

- *Session Cookie* — set by Keycloak at the moment of logging in. Because it is an HTTP-Only cookie set by Keycloak, the client application has no access to it at all, which prevents it from being stolen.
- *Silent Check SSO* — a mechanism that allows the Keycloak adapter to check the session silently. This takes place in a special hidden *iframe*, in which Keycloak sends a request for tokens to the server. The Session Cookie is automatically attached to the request. On its basis, the server decides whether the user is still logged in and, if so, sends the tokens.

Logging in through Keycloak works according to the *Authorization Code Flow* combined with *PKCE* (*Proof Key for Code Exchange*). When attempting to log in, the application first generates a *code_verifier*, i.e. a random string, and a *code_challenge*, i.e. a hashed version of the *code_verifier*. The application then sends a login request in which the *code_challenge* is transmitted. The server responds by redirecting the user to the Keycloak login page. If the credentials entered by the user are correct, the user is redirected back with the *authorization_code* received from the server. Using the authorisation code, the application immediately sends a request for tokens to the server, to which it also attaches the *code_verifier* generated at the beginning. If the code is correct and the hashed *code_verifier* is the same as the *code_challenge* sent at the beginning, the server sends a set of tokens in response. In the client application, this whole mechanism is implemented by the Keycloak adapter, i.e. Keycloak-js.⁵⁸

10.5. Server application security

10.5.1. SQL injection

The server application uses Entity Framework Core as its ORM. Queries to the database are never sent using hand-written SQL; LINQ queries are used instead. Such queries are parameterised, which makes it impossible for users to inject their own SQL.

10.5.2. Data validation

All data sent by the client is validated. The application uses both validators provided by ASP.NET Core and ones written by us. This guarantees that the user will not be able to send data that could damage the system by causing unexpected behaviour.

10.5.3. RBAC validation

All valid JWT tokens that users use for authorisation contain information about the user's role. For this reason, the server application is able to verify very simply and securely whether the user is a student or a coach. Most of the requests a user can make to the server are protected by such a role, which ensures that the user will never receive resources to which they do not have access.

10.6. System infrastructure security

This section presents the methods of securing the OVH dedicated server, the Jenkins automation server and its pipelines, general securing of the Kubernetes cluster, and securing the GitHub repositories and the Passed Pawn domain name.

- **SSH login:**
 - Access to the servers is possible only with SSH keys.
 - SSH keys are protected with an additional passphrase for increased security.
 - Password login is disabled (`PasswordAuthentication no`) in the SSH configuration.
- **Firewall (UFW):**
 - UFW (Uncomplicated Firewall) configured to block all incoming traffic.
 - Incoming traffic allowed only on the necessary HTTP, HTTPS, SSH and Kubernetes ports.
 - Monitoring and logging of unauthorised access attempts.
- **Fail2Ban:**
 - Automatic blocking of IP addresses after multiple failed login attempts are detected.
 - Prevention of brute-force attacks and port scanning.
- **Security of the organisation, repositories and OCI registry on GitHub:**
 - Use of granular *deploy keys* for repositories, limiting access to the necessary resources only.
 - A CI/CD server account with limited permissions, using a *Personal Access Token* (PAT) with the minimum required permissions.

- Introduction of strict rules and protocols for introducing changes to the repository, e.g. *branch protection rules*:
 - Enforcing *code review* before merging changes.
 - Restricting the ability to push directly to the main branches.
 - Automated tests and review by GitHub Copilot before changes are accepted.
- **Built-in RBAC system and Network Policies in Kubernetes:**
 - Defining roles and permissions (Role-Based Access Control) in accordance with the principle of least privilege.
 - Restricting access to Kubernetes resources to authorised users and services only.
 - Implementation of *Network Policies* restricting communication between pods and external network traffic.
- **Least privileged access and isolation of the Jenkins automation environment:**
 - Configuration of Jenkins accounts with the minimum permissions needed to perform tasks.
 - Encapsulation of the Jenkins environment through containerisation and network isolation using Namespaces, Network Policies and Kubernetes RBAC.
 - Restricting access to sensitive secrets and keys in Jenkins.
- **Building and pushing artefacts:**
 - Use of Kaniko to build container images securely without the need to run Docker in root mode and without the need to mount the container runtime socket into Jenkins agents.
 - Automatic pushing of artefacts to the GitHub OCI registry using secure access tokens.
- **Securing the domain name with DNSSEC:**
 - Ensures the integrity and authenticity of DNS responses through digital signatures.
 - Protects against DNS spoofing and cache poisoning attacks, preventing the forgery of DNS records.
 - Deploying DNSSEC for the purchased domain increases user security and improves trust in the service.

Notes

55. Oxari Team. *Bezpieczeństwo systemów IT – czym jest i jak je skutecznie zapewnić?* (IT system security – what it is and how to ensure it effectively). <https://www.oxari.com/blog/bezpieczenstwo-systemow-it-czym-jest-i-jak-je-skutecznie-zapewnic>. Accessed: 2025-06-25. 2025.
56. Ibid.
57. Angular. *Preventing cross-site scripting (XSS)*. <https://angular.dev/best-practices/security>. Accessed: 2025-06-24. 2025.
58. Keycloak Team. *Securing applications and services with OpenID Connect*. <https://www.keycloak.org/securing-apps/oidc-layers>. Accessed: 2025-06-26. 2025.

11. Organisation of work on the project

Effective organisation of a development team's work is a key element of the success of every IT project. This chapter presents the methods and tools that were used during the development of the application, including teamwork techniques, the task management system and the means of communication that made it possible to achieve the project goals efficiently and with high quality.

11.1. Pair programming

Pair programming is a programming technique in which two programmers work together on one task, sitting at one computer. Recently, remote pair work has been gaining popularity, which consists in sharing one's screen with the other developer through dedicated applications.

The advantage of this teamwork technique is above all producing better software and finding errors at an earlier stage of the project's development. Working in pairs forces developers to hold a lively, continuous discussion, which leads to finding the best possible solution, and also allows valuable knowledge to be shared, which leads to a continuous improvement of the team members' programming competences. There are 3 main pair programming techniques:⁵⁹

- **Ping Pong** — a pair programming technique linked to Test-Driven Development. One developer writes tests, the other produces software that successfully passes these tests. The developers swap roles, which makes the technique effective and stable.
- **Driver-Navigator** — a pair programming technique based on a driver–navigator relationship. One developer writes the code (the driver), following the instructions and hints of the other person (the navigator). The driver has the opportunity to question and oversee the content received, which naturally leads to discussion and finding a common solution.
- **Unstructured Pairing** — the third pair programming technique, occurring when the teamwork practised does not fit any of the above forms. It is distinguished by a looser structure combining the Ping-Pong and Driver-Navigator techniques. Swapping the developers' roles is not rigidly imposed; it happens at a convenient, sensible moment.

While building the chess application, the development team worked mainly individually on the challenges set before it. Pair programming was used mainly when working on complex and high-priority tasks that required two pairs of eyes to solve the problem efficiently and with high quality. Over the long road of developing the chess portal, the team mainly used the unstructured pair programming technique; thanks to its loose and natural form, the foundations of the application were built effectively and enjoyably. This teamwork technique brought fruitful results in developing the chess logic and in integrating the access management system — Keycloak — with the chess service.

The advantage of using this technique is its effectiveness. The team held many intensive discussions about the architecture of the system and the solutions to problems concerning the application, which translated into producing efficient and less defective software. The process of involving an additional team member in work on the aforementioned complex tasks was very simple: if solving a task alone

was difficult or took too much time (which consequently threatened the smooth development of the chess application), the team decided to work in pairs.

11.2. Code review

Code review is a process in software development whose goal is to check the written code and detect potential errors. There are several types of code review, including the Pull Request Review, a feature used in Git-based version control systems (GitHub, GitLab). Developers create a pull request with the proposed code changes, which are then checked by the rest of the team members.

The chess application repository uses this feature, which makes it possible to control the code being pushed and to detect errors at a very early stage before the changes are merged into the main branch. Code review also forces team members to familiarise themselves with the solutions to tasks carried out by other team members, thanks to which knowledge about the project is distributed and dependence on a single person is limited.

Another advantage of this feature is the exchange of knowledge within the team and the possibility of discussing the proposed solution, which leads to making joint and, consequently, better design decisions. During the development of the chess portal, many implementation errors that were detected at the code review stage were fixed, which shows that the process works effectively.⁶⁰

11.3. Kanban

The development of the chess application involved a large number of tasks requiring effective management, which is why the team decided to use a dedicated tool — a Kanban board in the Trello application. Kanban is a project management framework that uses Agile principles to plan, visualise and organise project tasks. One of the tools offered by the framework is the Kanban board, which makes it possible to visualise all the tasks that individual team members are working on. The board consists of columns that represent specific stages of work (e.g. "To do", "Code review", "Done"). Thanks to this, the current state of the whole team's work can be presented clearly.

Using the framework enabled consistent and effective development of the project by the team. Every bug encountered while working on the implementation of the chess portal was immediately reported and recorded on the board, which contributed to keeping the team's work organised. The key improvement that Trello provided is precisely order, i.e. a sorted and readable list of tasks.

Using a project management tool is a foundation in the world of today's software development. Managing a complex project with a large number of tasks requires a system that provides transparency and control over progress; without it, chaos and a lack of coordination come easily.⁶¹

Table 8: Comparison of working with a Kanban board and without a task structure

Aspect	Working with Kanban (e.g. Trello)	Working without a defined task organisation
Task visibility	Clear, transparent visualisation of all tasks	No overall view, tasks are often scattered

Aspect	Working with Kanban (e.g. Trello)	Working without a defined task organisation
Division into work stages	Tasks assigned to specific stages (To Do / In Progress / Done)	No clear division — hard to tell what is in progress
Task prioritisation	Ability to set priorities, labels and deadlines	Priorities are often unclear or forgotten
Team collaboration	Easy tracking of progress and assignments	Difficulties in dividing responsibilities
Progress tracking	Simple overview of who is working on what	Progress is not clearly visible
Eliminating bottlenecks	Quick identification of overloads at specific stages	No mechanism for detecting problems in the workflow
Time management	Helps to plan and control delivery time	Often leads to delays and chaos
Motivation and order	Systematic work, motivation from moving cards	Risk of losing or forgetting tasks

11.4. Brainstorming

Before writing the first line of code, one needs an idea, an outline of the project, an MVP that will be implemented in the future. Every project begins with a carefully worked-out plan of action and the preparation of the application's basic features. A tool is needed that allows the team's ideas to be documented.

Work on the chess portal project began by gathering the ideas of the team members, and brainstorming was used for this — a teamwork technique based on creative thinking, whose aim is to generate as many ideas as possible for solving a specific problem. It is an ideal solution for quickly documenting the numerous, often chaotic visions and suggestions of team members, which, once combined, grew into a coherent model of the chess application. The ideas varied, some of them even absurd, but intensive and constructive discussion within the team made it possible to arrive at a solid outline of the project.

The process of generating good ideas began with each developer individually gathering and writing down ideas and visions in the form of short descriptions on the Miro platform. This was followed by a team discussion, during which the proposals were discussed and the best solutions were jointly selected. The process went very smoothly, and once a coherent whole had taken shape, development work began.⁶²

11.5. Communication in the team

Communication is an indispensable element of a successful IT project. It is often underestimated and neglected by busy, isolated developers focused solely on their assigned task. Communication is the key to efficient teamwork; it allows lively discussion and joint problem solving.

The team developing the chess application had no problems exchanging information; the relaxed atmosphere made it easy to communicate about difficulties encountered and the need for consultation in case of doubt. The foundation of the team's communication was simplicity; a clear goal and clarity

of the information conveyed were essential. The focus was on the quality of what was said, not its volume.

The main channel for conveying information was text messages. After pushing changes to the repository and creating a pull request, a developer informed the rest of the team on a designated text channel that their code was ready for review. The message contained helpful links to the repository and the Kanban board, which made it easier to find one's way quickly through the thicket of numerous tasks and provided programming context.

When working on complex tasks requiring consultation with the rest of the team, voice channels were used, which enabled efficient transfer of information and served as a tool for pair programming. The use of dedicated applications (Miro, Trello) made it possible to organise the team's work, which, combined with good communication, turned into fast and stable development of the project.⁶³

Notes

59. Codementor. *What is pair programming?* <https://www.codementor.io/pair-programming>. Accessed: 2025-06-19. 2025.
60. BrowserStack. *What is Code Review?* <https://www.browserstack.com/guide/what-is-code-review>. Accessed: 2025-06-19. 2025.
61. Coursera Staff. *What Is a Kanban Board?* <https://tinyurl.com/mryvypab>. Accessed: 2025-06-19. 2024.
62. Miro. *Jak przeprowadzić burzę mózgów* (How to run a brainstorming session). <https://miro.com/pl/brainstorming/what-is-brainstorming/>. Accessed: 2025-06-19. 2025.
63. Ayoub Abidi. *The Role of Communication in Software Development*. <https://ayoub3bidi.medium.com/the-role-of-communication-in-software-development-2db6cbe38c>. Accessed: 2025-06-19. 2023.

12. System testing and validation

12.1. API tests

The API is tested mainly in two ways: manually and with automated unit tests.

12.1.1. Manual tests

Every time a change is made to the code, the related part of the application is immediately tested manually. Two tools are used for manual testing:

- **Postman:** A popular tool for testing APIs. Almost all endpoints are saved, and new ones can easily be added as new features are created. After a change is made, before it is deployed to production, requests are sent to the relevant endpoints in a variety of ways. For example, it is checked whether the application behaves correctly for a student, a coach and an anonymous user. Testing is also carried out on different data sets, both for valid cases and for ones that should be rejected. Although these tests do not give a full guarantee of correct operation, they make it possible to catch the most obvious errors quickly.
- **DevTools:** The API is also tested together with the frontend, directly in the browser. The application is used in the way a user would use it, and network traffic is monitored in the "Network" tab of DevTools. It is verified whether all requests to the API have the correct structure and whether the responses meet expectations. These tests provide a high probability that the vast majority of users will be able to use the application without errors.

12.1.2. Automated tests

Automated tests have a certain drawback — when the code changes, the related tests often have to be modified as well, which can be time-consuming. Since manual tests already provide a high level of confidence, it was decided that the additional certainty offered by automated tests does not justify the effort needed to maintain them. Therefore, automated testing covers only those parts of the application that contain complex logic or are most frequently used by users.

The xUnit framework is used for automated tests — a popular tool in the .NET ecosystem intended for testing applications of every kind, including APIs. In addition, the Moq library is used, which makes it easy to create mocks of services.

All services inside the API are designed as interfaces that are injected into the components that need them. One of the advantages of this approach is that they can easily be mocked. Thanks to this, controllers can be tested independently of the logic contained in the services.

The project is divided into three main parts: Controllers, Services and Validators:

- **Controllers:** The controllers are tested, in particular the endpoints most frequently used by users. These tests assume that the services work correctly and check only whether the controllers react properly. They provide high confidence, but because the controller code is modified frequently, the test code also has to be updated often. Each test follows the pattern:
 - **Arrange:** The appropriate mocks are prepared, which are to respond in a defined way.

- Act: The method to be tested is called.
- Assert: The result of the method is checked: the most important are the response status and the returned object, which must correspond to the assumed test cases.
- Services: Only those services that contain complicated logic are tested. This is logic that is rarely modified but difficult to test manually because of the multitude of cases that must be taken into account. These tests enabled a Test-Driven Development approach, in which the tests were written first and only then the implementation. Thanks to this, each iteration showed which cases still needed to be handled.
 - ClaimsPrincipalServiceTests: The logic of finding a user in the database on the basis of the JWT token is tested.
 - CourseServiceTests: The service responsible for creating and modifying courses is tested. The most important thing is testing the addition and reordering of lessons, because each lesson has an Order field. A guarantee is required that there are no duplicate Order values and that adding a lesson between others shifts the remaining ones accordingly.
- Validators: Validators are responsible for verifying the correctness of the data entered by the user. They are usually simple, with the exception of one validator that checks whether a given FEN (a string representing the state of the chessboard) has the correct format. In this case an advanced regular expression (regex) is used, which requires thorough testing. Numerous cases of both valid and invalid FENs are tested.

12.1.3. GitHub Actions

Every time changes are pushed to GitHub, a so-called GitHub Action is run. GitHub Actions have many uses, but in the project repository they are used to run the tests automatically and quickly.

The whole process is performed automatically every time new code is pushed to the repository. Thanks to this, it is possible to check quickly whether the application works correctly after the changes made.

While a GitHub Action is running:

- A virtual environment with the appropriate dependencies is created,
- The application is built,
- All automated tests are run.

On the basis of the result of these steps, the system automatically reports success or failure. When another person reviews the code, the status of the last run of the action is visible — thanks to which it can be checked very quickly whether all tests have passed.

In the event of errors (e.g. failing tests), the person responsible for the changes is notified by GitHub by e-mail that the Action has failed. Thanks to this, the problem can be noticed and fixed quickly, before the changes are merged into the main branch of the project.

12.1.4. Rider

Rider is an IDE produced and supported by JetBrains. Every programmer responsible for the project's API uses this IDE, as it provides many different tools and programming aids. Many of these tools

support testing.

- Unit Tests tab: The IDE automatically finds all tests in the project and displays them in a dedicated tab. Here the programmer can run all of them, or only specific ones, with one button, and then see which pass and which do not.

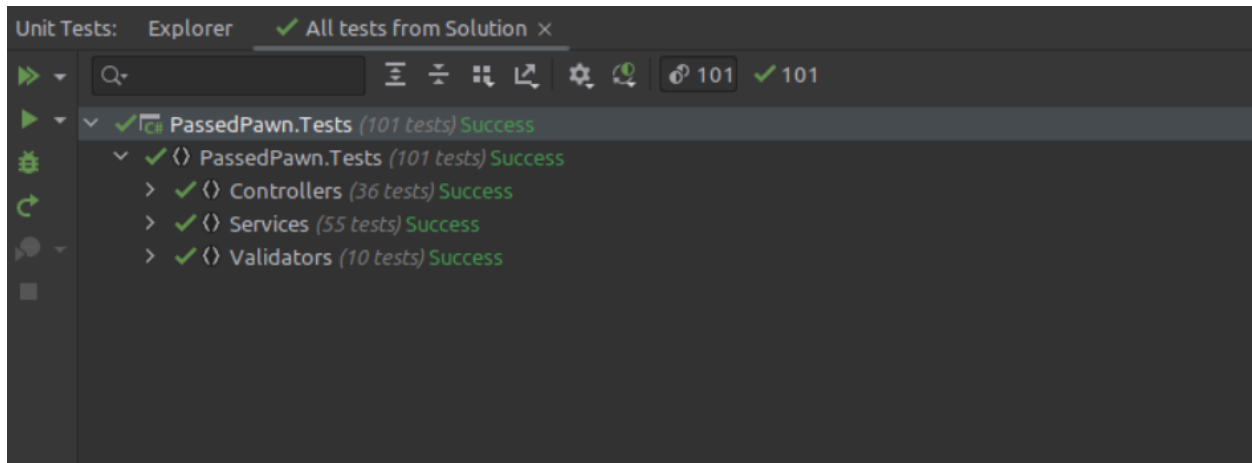


Figure 22: Graphical presentation of test status

- DotCover: A tool that, after the tests are run, calculates which parts of the application are tested and which are not. It makes it easier for the programmer to analyse which parts of the application need more tests.

Controllers	27%	380/523
> CourseController	100%	0/5
> StudentController	100%	0/13
> CourseReviewController	93%	2/27
> CourseStudentController	43%	37/65
> CourseCoachController	42%	56/97
> LessonController	38%	51/82
> CoachController	0%	50/50
> CourseExampleController	0%	30/30
> CoursePuzzleController	0%	41/41

Figure 23: Graphical presentation of code coverage by tests

12.2. End-to-end tests

End-to-end testing is a testing technique aimed at checking the entire flow of the application — from beginning to end. This type of test verifies every component of the system and how they work together. Manual end-to-end tests were carried out in the Passed Pawn application. Scenarios were pre-

pared that describe in detail how each test should be performed. Below is an example scenario concerning the purchase of a course by a student.

Table 9: Example end-to-end test scenario

Scenario name	Purchase of a course by a user
Actors	User logged in as a student
Preconditions	• The user has a student account in the system • The user is logged in to the account • The user has a payment card with available funds
Initial state	The logged-in user is on the "My Courses" page
Flow of events	1. The user clicks the "Available Courses" button in the navigation bar 2. The user searches for a course of interest using the search engine 3. The system displays matching courses 4. The user clicks the "Buy" button on the tile of one of the courses 5. The system displays a form for payment card details 6. The user fills in the card details 7. The user clicks the "Pay" button 8. The system displays a notification that the payment has started 9. The system displays a notification that the payment has succeeded
Success postconditions	• The user gains access to the course in the "My Courses" tab and to all the lessons and elements it contains • A student account was created with the user's data

13. Project summary and potential for further development

The presented system meets the basic functional and non-functional requirements that were set. In its current form it makes it possible to create chess courses containing elements dedicated to chess that support the learning process — quizzes, examples and puzzles, which contain interactive chessboards with visual aids. At the same time, it offers improvements over competing solutions in the form of an intuitive and simple user interface, a clear view of course content for students, an AI assistant, and open access to publishing content on the platform for interested tutors.

Thanks to the use of DevOps, IaC, GitOps and CI/CD practices, the work of maintaining, modifying and developing the system is highly auditable and stable, while still being efficient. This is a testament to the effectiveness of the practices applied and of the thoughtfully designed system architecture. Despite considerable growth, the system remains efficient and approachable to manage. In its current form the application is a solid product that can be extended with additional features, such as:

- Learning progress tracking — elements of a course that have already been visited by the user will be marked accordingly. In addition, the course tile will show what percentage of the elements has already been visited
- Experience points that the student will be able to exchange for further courses — they engage the user in further interaction with the platform and offer a reward system that motivates them to achieve further goals
- Integration with external APIs, e.g. the Chess.com platform — for an example with a specific position, the user will be able to see all their games in which this position occurred and analyse their mistakes.
- Hot content — a panel highlighting popular course elements that are well received and very popular among users (it would help to highlight content worth attention).
- Translation of courses using artificial intelligence — this will make it possible to reach more students, for example those who do not know English well. It will also enable coaches from many countries, who prefer to create courses in their native language, to create popular courses. The artificial intelligence used for this purpose is developing very rapidly and already enables high-quality translations.
- Individual lessons with a coach — a feature providing lessons with a chess tutor in the form of a video call with access to an interactive chessboard and training materials. The student would also be able to play chess with the coach on a virtual chessboard enriched with visual support in the form of arrows and highlighted squares.
- Final exam generation — a feature that would make it possible to automatically create an exam for the student at the end of a completed course. It would help to check and consolidate the knowledge gained.
- A ranking of users with the most experience points — it could encourage regular learning and more frequent use of the application.
- Adding other users as friends — this will make it possible to display the user's progress against the people around them. In addition, it may encourage the student to promote the platform among their friends and family.

Bibliography

1. Chess.com. *How Many Chess Players Are There In The World?*
<https://www.chess.com/article/view/how-many-chess-players-are-there-in-the-world>.
Accessed: 2025-06-23. 2017.
2. Chessly. *Chessly main page*. <https://chessly.com/>. Accessed: 2025-06-23. 2025.
3. Canva. *How to Use Canva: A Beginner's Guide*. <https://www.canva.com/learn/how-to-canva-beginners-guide/>. Accessed: 2025-06-19. 2025.
4. Figma. *Browse component properties* (Polish: *Przeglądaj właściwości komponentów*).
<https://www.canva.com/learn/how-to-canva-beginners-guide/>. Accessed: 2025-06-19. 2025.
5. Hitesh Sahni. *Canva vs Figma comparison: which design tool is right for you?*
<https://hypegig.com/canva-vs-figma-comparison/>. Accessed: 2025-06-19. 2025.
6. prof. WSEI dr hab. inż. Jan Werewka. *Rola i znaczenie architektury w obszarze IT* (The role and importance of architecture in IT). <https://magazyn.wsei.edu.pl/rola-i-znaczenie-architektury-w-obszarze-it/>. Accessed: 2025-06-21. 2024.
7. Marcin Sobieraj. *Service Oriented Architecture - SOA*.
<https://drogaarchitektait.pl/2020/08/service-oriented-architecture-soa/>. Accessed: 2025-06-21. 2020.
8. Pramitha Jayasooriya. *Monolithic vs. SOA vs. Microservices Architecture: A Java Perspective*.
<https://medium.com/@lpramithamj/monolithic-vs-soa-vs-microservices-architecture-a-java-perspective-6d3d9fb26ac7>. Accessed: 2025-06-21. 2024.
9. Sehban Alam. *Single Page Applications (SPAs) vs. Multi-Page Applications (MPAs): Advantages and Challenges*. <https://medium.com/@sehban.alam/single-page-applications-spas-vs-multi-page-applications-mpas-advantages-and-challenges-df06bee3fed1/>. Accessed: 2025-06-17. 2024.
10. Edgemesh. *Why Time to Interactive (TTI) is the Most Consistent and Valuable Metric*.
<https://edgemesh.com/blog/time-to-interactive-and-conversion-rate#heading-1>. Accessed: 2025-06-17. 2025.
11. BrowserStack. *Angular vs React vs Vue: Core Differences*.
<https://www.browserstack.com/guide/angular-vs-react-vs-vue>. Accessed: 2025-06-18. 2025.
12. Stack Overflow. *Tag Trends*.
<https://trends.stackoverflow.co/tags=reactjs%2Cvue.js%2Cangular%2Csvelte%2Cangularjs%2Cvuejs3>. Accessed: 2025-06-19. 2025.
13. Gara Mohamed. *Angular vs. React (Part 1): style abstraction and encapsulation*.
<https://medium.com/@gara.mohamed/angular-vs-react-part-1-style-abstraction-and-encapsulation-f55b336e0eaa>. Accessed: 2025-06-19. 2018.
14. Sotiris Kourouklis. *SASS, CSS, or Tailwind: Which One Should You Choose?*
<https://dev.to/sotergreco/sass-css-or-tailwind-which-one-should-you-choose-3c00>.
Accessed: 2025-06-19. 2025.

15. PrimeNG. *PrimeNG Documentation*. <https://primeng.org/>. Accessed: 2025-06-17. 2025.
16. Miłosz Rutkowski. *Wszystko co musisz wiedzieć o Dependency Injection w Angularze* (Everything you need to know about Dependency Injection in Angular). <https://angular.love/pl/wszystko-co-musisz-wiedziec-o-dependency-injection-w-angularze>. Accessed: 2025-06-18. 2024.
17. Angular. *Forms in Angular*. <https://angular.dev/guide/forms>. Accessed: 2025-06-18. 2025.
18. Angular Minds. *Data Binding In Angular: Everything You Need to Know*. <https://www.angularminds.com/blog/data-binding-in-angular>. Accessed: 2025-06-18. 2025.
19. Angular. *The RxJS library*. <https://v17.angular.io/guide/rx-library>. Accessed: 2025-06-17. 2023.
20. RxJS. *RxJS Overview*. <https://rxjs.dev/guide/overview>. Accessed: 2025-06-17. 2025.
21. Angular University. *Angular Change Detection - How Does It Really Work?* <https://blog.angular-university.io/how-does-angular-2-change-detection-really-work/>. Accessed: 2025-06-19. 2025.
22. Angular. *Understanding communicating with backend services using HTTP*. <https://angular.dev/guide/http>. Accessed: 2025-06-19. 2025.
23. Keycloak JS. *Keycloak JS npm package page*. <https://www.npmjs.com/package/keycloak-js>. Accessed: 2025-06-17. 2025.
24. Mauricio Vigolo. *Keycloak Angular npm package page*. <https://www.npmjs.com/package/keycloak-angular>. Accessed: 2025-06-17. 2025.
25. Angular. *CanActivateFn Documentation*. <https://angular.dev/api/router/CanActivateFn>. Accessed: 2025-06-19. 2025.
26. Raghuvardhan Karanam. *Route Guards in Angular*. <https://raghuvardhankaranam.medium.com/route-guards-in-angular-c2c01fe6167b>. Accessed: 2025-06-19. 2023.
27. Roberts Tech. *Code a Chess Game with Stockfish API – JavaScript Tutorial*. <https://youtu.be/fJIsqZmQVZQ?si=6JttY01vB8CtVCzJ>. Accessed: 2025-06-30. 2024.
28. Roberts Tech. *Code a Chess Game with Stockfish API – JavaScript Tutorial*. <https://github.com/awesomeCStutorials/chess-game>. Accessed: 2025-06-20. 2024.
29. davehorner. *license?* <https://github.com/awesomeCStutorials/chess-game/issues/4>. Accessed: 2025-06-20. 2025.
30. LangChain. *Build a Retrieval Augmented Generation (RAG) App: Part 1*. <https://python.langchain.com/docs/tutorials/rag/>. Accessed: 2025-06-24. 2024.
31. MistralAI. *Mistral Large, our new flagship model*. <https://mistral.ai/news/mistral-large>. Accessed: 2025-06-17. 2025.
32. MistralAI. *Models Overview*. https://docs.mistral.ai/getting-started/models/models_overview/. Accessed: 2025-06-17. 2025.
33. IBM. *What Is IT Infrastructure?* <https://www.ibm.com/think/topics/infrastructure>. Accessed: 2025-06-23. 2021.

34. OVH. *Dedicated Server Service Level Agreement*.
<https://us.ovhcloud.com/legal/sla/dedicated-servers/>. Accessed: 2025-06-23. 2025.
35. Kubernetes technical documentation. *Kubernetes*. <https://kubernetes.io/>. Accessed: 2025-06-23. 2025.
36. Kubernetes technical documentation. *Concepts/Overview*.
<https://kubernetes.io/docs/concepts/overview/>. Accessed: 2025-06-23. 2024.
37. Dhaval Gajjar. *Why And Where To Use Kubernetes*.
<https://www.opensourceforu.com/2024/02/why-and-where-to-use-kubernetes/>. Accessed: 2025-06-23. 2024.
38. K3s technical documentation. *K3s - Lightweight Kubernetes*. <https://docs.k3s.io/>. Accessed: 2025-06-23. 2025.
39. K3s technical documentation. *Requirements*. <https://docs.k3s.io/installation/requirements>. Accessed: 2025-06-23. 2025.
40. Longhorn technical documentation. *Features*. <https://longhorn.io/docs/1.9.0/>. Accessed: 2025-06-23. 2025.
41. Kubernetes technical documentation. *Ingress*. <https://kubernetes.io/docs/concepts/services-networking/ingress/>. Accessed: 2025-06-23. 2025.
42. Kubernetes ingress-nginx repository. *ingress-nginx*. <https://github.com/kubernetes/ingress-nginx>. Accessed: 2025-06-23. 2025.
43. cert-manager technical documentation. *cert-manager*. <https://cert-manager.io/docs/>. Accessed: 2025-06-23. 2025.
44. Grzegorz Gnych. *Czym jest Infrastruktura jako kod (Infrastructure as Code)? – Kompendium wiedzy* (What is Infrastructure as Code? – A compendium of knowledge). <https://nflo.pl/baza-wiedzy/czym-jest-infrastruktura-jako-kod-infrastruktura-as-code-kompendium-wiedzy/#Integracja-z-Tenable-i-Rapid7>. Accessed: 2025-06-25.
45. Andrzej Ochman. *GitOps cz. I. Wprowadzenie do GitOps i jego rola w konteneryzacji* (GitOps part I. Introduction to GitOps and its role in containerisation). <https://linuxpolska.com/pl/baza-wiedzy/blog/gitops-wprowadzenie-jego-rola-w-konteneryzacji-cz-i/>. Accessed: 2025-06-25. 2025.
46. Gene Kim et al. *The DevOps Handbook*. 2nd ed. Accessed: 2025-06-25. IT Revolution Press, 2016.
47. Microsoft. *Continuous Integration and Continuous Delivery*. <https://microsoft.github.io/code-with-engineering-playbook/CI-CD/>. Accessed: 2025-06-25. 2024.
48. Oxari Team. *Bezpieczeństwo systemów IT – czym jest i jak je skutecznie zapewnić?* (IT system security – what it is and how to ensure it effectively). <https://www.oxari.com/blog/bezpieczenstwo-systemow-it-czym-jest-i-jak-je-skutecznie-zapewnic>. Accessed: 2025-06-25. 2025.
49. Angular. *Preventing cross-site scripting (XSS)*. <https://angular.dev/best-practices/security>. Accessed: 2025-06-24. 2025.
50. Keycloak Team. *Securing applications and services with OpenID Connect*.
<https://www.keycloak.org/securing-apps/oidc-layers>. Accessed: 2025-06-26. 2025.

51. Codementor. *What is pair programming?* <https://www.codementor.io/pair-programming>. Accessed: 2025-06-19. 2025.
52. BrowserStack. *What is Code Review?* <https://www.browserstack.com/guide/what-is-code-review>. Accessed: 2025-06-19. 2025.
53. Coursera Staff. *What Is a Kanban Board?* <https://tinyurl.com/mryvypab>. Accessed: 2025-06-19. 2024.
54. Miro. *Jak przeprowadzić burzę mózgów* (How to run a brainstorming session). <https://miro.com/pl/brainstorming/what-is-brainstorming/>. Accessed: 2025-06-19. 2025.
55. Ayoub Abidi. *The Role of Communication in Software Development*. <https://ayoub3bidi.medium.com/the-role-of-communication-in-software-development-2db6cbe38c>. Accessed: 2025-06-19. 2023.

List of figures

1	Coach use case diagram	14
2	Student use case diagram	15
3	Use case diagram of features shared by the coach and the student	16
4	Landing page of the chess application	21
5	View of the course offer and the proposed learning model	22
6	Landing page, questions section	23
7	View of the puzzle creation component	24
8	View of the quiz creation component, stage 3	25
9	Course review component	26
10	Component presenting the average course rating	27
11	Input field component for the course difficulty level	27
12	Component, diagram presenting the content of a chess course	27
13	Control panel and the system of interactive arrows on the chessboard	28
14	AI assistant	29
15	UML component diagram of the general architecture	38
16	ERD diagram of the application database	40
17	Diagram of the client application modules	54
18	Diagram of how RAG works, published by LangChain	64
19	Sequence diagram of the AI module. Yellow denotes services outside the system.	66
20	Diagram of the CI pipeline architecture	84
21	Architecture of building and pushing the application image in the form of a Helm Chart	86
22	Graphical presentation of test status	104
23	Graphical presentation of code coverage by tests	104

List of tables

1	Tool comparison: Canva and Figma	19
2	Comparison: custom UI design vs ready-made UI library	29
3	Comparison of architectures	37
4	Framework comparison: Angular, React and Vue	45
5	Comparison of component styling: Angular and React	46
6	Comparison of frontend styling systems	47
7	Example REST requests	59
8	Comparison of working with a Kanban board and without a task structure	99
9	Example end-to-end test scenario	105